

讲义 · 网页版

工程优化

从实践者视角看问题、解与方法

周艳杰

郑州大学

<https://ieyjzhou.github.io>

<https://ieyjzhou.github.io/lab>

ieyjzhou@zzu.edu.cn



郑州大学
ZHENGZHOU UNIVERSITY

第 零 篇

课程简介



工程优化不只是算法的集合，而是一种思维方式。它是工业界的“全科医生”：
先读懂整个系统，再决定该用哪一类专科手段。

本教材的逻辑框架与篇章关系

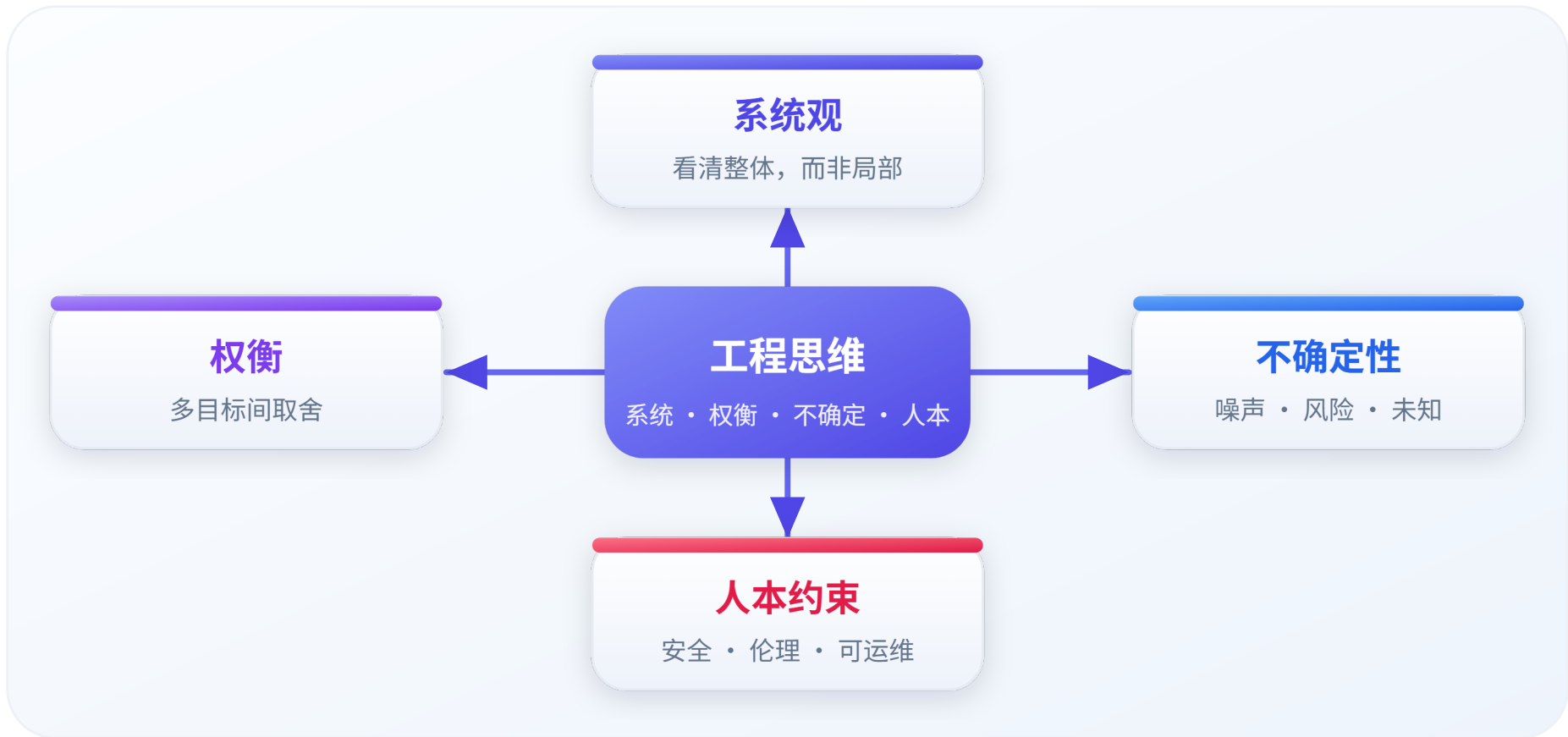
全书的五篇构成一条主线：先建立工程思维与问题意识，再掌握通用方法论，继而把实际问题归入问题谱系、匹配相应方法体系，最后在工程中落地。



逻辑主线：第零篇（思维）→ 第二篇（方法论）→ 第三篇（问题分型）→ 第四篇（方法体系）→ 第五篇（落地）；应用中的新难题再反哺问题与方法的演进，形成“认知→基础→问题→方法→应用→再认知”的闭环。

工程优化培养的是工程思维

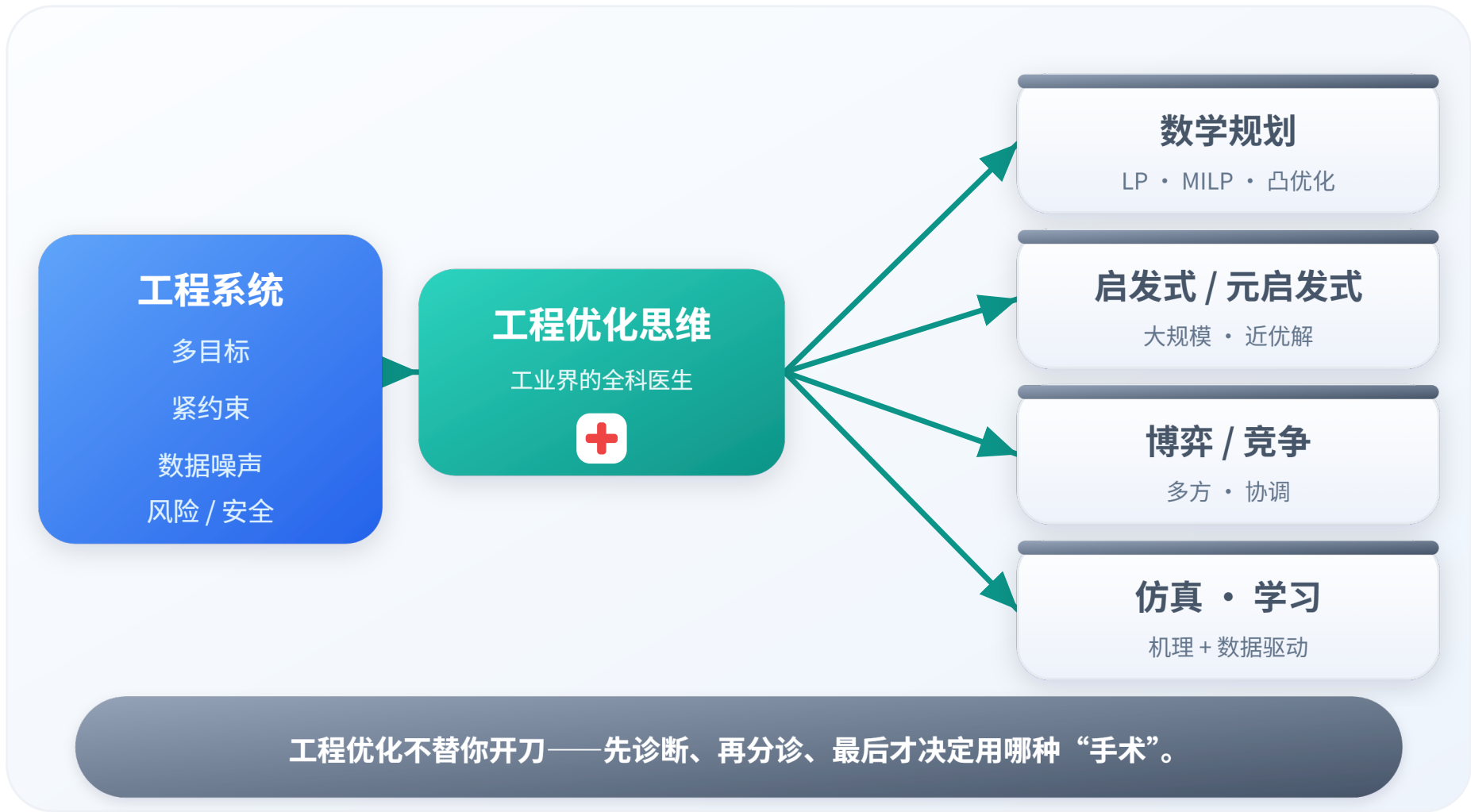
全书的六篇（第零篇至第五篇）构成一条主线：先建立工程思维与问题意识（第零篇），继而打牢形式化与建模的基础（第一篇），再掌握通用方法论（第二篇），把实际问题归入问题谱系（第三篇）、匹配相应方法体系（第四篇），最后在工程中落地（第五篇）。



逻辑主线：第零篇（思维）→ 第一篇（基础）→ 第二篇（方法论）→ 第三篇（问题分型）→ 第四篇（方法体系）→ 第五篇（落地）；应用中的新难题再反哺问题与方法的演进，形成“认知→基础→问题→方法→应用→再认知”的闭环。

工业界的全科医生

面对一个陌生的工程问题，你往往先要当“全科医生”：读懂整个系统，再判断该请哪一类“专科手段”出手。



从主诉到分诊：工程优化思维把问题导向最合适的专科方法。

工程问题有多庞大、多复杂

一个真实的工程问题，从来不是一道孤立的习题：它牵涉成百上千的设施、变量与约束，且彼此**强耦合**——牵一发而动全身。



一个工程问题把设施、运输、制造、能源、库存、成本、市场、风险等子系统缠成一团，且始终处在不确定与动态之中。

工程问题有哪些类型

问题千差万别，**没有一种算法能包打天下**。先看清它落在哪几根“轴”上，才能选出对的工具——这正是全科医生的分型思路。



从变量、目标、信息、时间、主体到数据——六个维度把问题“分型”，方法才能对症。

求解方法：又多，又复杂

工程问题的求解方法同样庞杂：同一类问题往往有多条技术路线，每条路线下又有大量具体算法。下面只是一张“方法地图”的缩影。

数学规划

- LP · QP · SOCP
- 凸优化
- 非线性规划 NLP

整数 / 组合

- 分支定界
- 割平面 · 列生成
- Benders 分解

启发式 / 元启发式

- 遗传 GA · 模拟退火
- 禁忌搜索 Tabu
- 蚁群 ACO · 粒子群 PSO

多目标优化

- 加权 · ϵ -约束
- NSGA / MOEA
- Pareto 前沿

不确定优化

- 随机规划
- 鲁棒 / 分布鲁棒
- 机会约束

动态 / 在线

- 动态规划 DP
- 模型预测 MPC
- 强化学习 RL

博弈 / 多主体

- 纳什 / Stackelberg
- 机制设计
- 拍卖 · 协调

数据驱动

- 代理模型 Surrogate
- 贝叶斯优化
- 仿真优化 · ML

问题分型 → 方法分型：本书涉及 5 大类、数十种算法——先分型，再选刀。

从数学规划到启发式、从鲁棒到博弈、从仿真到学习——方法之丰富，正对应问题之丰富。

第一部分

基础

我们如何形式化一个问题、为何要分解它、“解”意味着什么、存在哪些方法，以及将它们联系在一起的种种权衡。

本篇逻辑框架

本篇奠定工程优化的语言基础：从“问题是什么”出发，到如何分解、何为解、有哪些方法，以及贯穿始终的权衡。

本篇 5 章 · 按阅读顺序



逻辑主线：问题 → 分解 → 解 → 方法 → 权衡，五章层层递进，构成全书的方法论地基。

主线：从问题到分解，到解，到方法，到权衡；五章依次构建方法学基础。

逻辑主线：问题 → 分解 → 解 → 方法 → 权衡，五章层层递进，构成全书的方法论地基。

CHAPTER 1

问题

在优化任何东西之前，我们必须先说清“问题”是什么——以及为何有些容易、有些困难、有些不可能。

审视每个问题的三种视角

每个工程优化问题，本质上都是一个关于**选择**的问题：哪一种设计、计划或控制决策最优？

决策问题

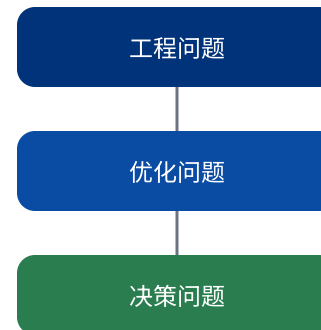
一个是非题。“是否存在满足所有截止期的调度方案？”——只是一位信息。

优化问题

要求**最优**的值或最优点——而不只是“是否存在一个好的解”。

工程问题

我们最终想要解决的、混乱的真实世界问题——集装箱码头、电网、供应链等。



(固定阈值 → 伪装下的决策问题)

一条层级链：工程问题被建模为优化问题；固定一个阈值后，它又变成决策问题。

决策问题与可判定性

决策问题的回答永远是“是”或“否”。每个输入恰好映射到两种输出之一。



决策问题把每个输入映射到两种输出之一（是或否）。

可判定

存在一种算法，总能在有限步内停机并给出正确的“是/否”回答。例如“DFA 是否接受串 w ？”——在 M 上运行 w ，时间为 $O(|w|)$ 的线性。

不可判定

对所有输入都不存在这样的算法。经典的例子是 *停机问题*：图灵证明，对任意 M 都不存在能判定它的算法。

数独（“是否存在合法的 9×9 填数？”）虽可判定，但一般情形下是 NP 完全。可判定性问的是有没有算法；可处理性问的是代价多大——这是两个不同的问题。

数独：一个决策问题实例

Wikipedia 经典例题

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

自动填充答案

检查

重置

换一题

1

2

3

4

5

6

7

8

9

< 擦除

点击格子并填入 1-9，或用下方数字键；按「自动填充答案」可看完整解。
实际例子：下面是一张真实的数独题，试着把它填完——这正是“找出一个合法填数”的优化版本。

问：“是否存在一个合法填数？”——答案是单一的是/否位。

- **可判定**：回溯搜索总会停机并给出正确答案。
- 但一般尺寸的 9×9 数独是 **NP 完全** 问题；推广到任意网格尺寸后尚无多项式算法。
- 同一张网格还有 **优化版本**（找出一个合法填数），这正是它又回到优化视角的原因。
- **动手试**：点左图格子填 1-9，或按「自动填充答案」立刻看到完整解——这就是“优化版本”要找的答案。

数独的数学建模：两种整数规划方案

上面那张游戏是一张“决策/优化”实例。下面把它写成严格的整数规划。Williams（2026）用数独对比了两种建模思路，核心教训是：描述问题最自然的变量，往往不是产出有效线性模型的那一个。

方案 A — 二进制分配模型

引入 729 个 0/1 变量：令 x_{ijk} 表示“数字 k 放在格子 (i,j) ”。所有规则都变成简单的线性等式。

$$\sum_{k=1}^9 x_{ijk} = 1 \quad (i,j) \in \{1,\dots,9\}^2$$

(每格恰一个数字)

$$\sum_{j=1}^9 x_{ijk} = 1 \quad i=1,\dots,9, k=1,\dots,9$$

(每列含数字 1-9 各一次)

$$\sum_{i=1}^9 x_{ijk} = 1 \quad j=1,\dots,9, k=1,\dots,9$$

(每行含数字 1-9 各一次)

$$\sum_{i,j \in \{a+b, a+b+1, \dots, a+b+8\}} x_{ijk} = 1 \quad k=1,\dots,9$$

(每宫含数字 1-9 各一次)

若 $a+b \leq 9$ 为已知 i,j

目标为 $\sum_{i,j,k} x_{ijk}$ (只要可行解)。变量虽多，但约束全是线性等式，极易求解，也便于扩展到 Killer Sudoku (笼子和约束)。

Python 实现 (python-mip + CBC 开源求解器)

方案 B — 自然整数变量 + big-M

更直观：只设 81 个整数变量 x_{ij} 表示每格的值。但“每行/列/宫内互不相同”必须把两两不等写成线性析取，用 big-M 松弛：

$$x_{ij} - x_{st} \leq M \cdot (1 - y_{ijst})$$
$$x_{st} - x_{ij} \leq M \cdot (1 - y_{stij})$$
$$y_{ijst} + y_{stij} \leq 1$$

对同行/同列/同宫的每一对 $(i,j), (s,t)$ 各加一对不等式与 1 个辅助二元变量 y_{ijst} 。共 810 个辅助二元 + 1620 个 big-M 不等式——变量少了，约束却暴涨。AMPL 的 alldiff 只是把这套繁琐约束藏在了高层算子里。

方案 A 的分配模型（下面列出的 givens 即本页图的同一道题）：

```
# Formulation A – binary assignment model
# (the givens below are the same puzzle shown in the figure)
from mip import Model, xsum, BINARY, minimize

N = 9
givens = [[6, 0, 8, 5, 0, 0, 0, 0, 0], [0, 0, 3, 6, 7, 9, 0, 0, 0], [4, 0, 0, 0, 3, 0, 0, 0, 7], [2, 0, 4, 0, 0, 0, 0, 8, 0], [0, 0, 0, 0, 0, 9, 0, 0, 0], [0, 0, 0, 0, 0, 0, 0, 0, 0], [0, 0, 0, 0, 0, 0, 0, 0, 0], [0, 0, 0, 0, 0, 0, 0, 0, 0], [0, 0, 0, 0, 0, 0, 0, 0, 0]]

m = Model(solver_name="CBC") # open-source COIN-OR CBC
x = [[[ m.add_var(var_type=BINARY) for k in range(N) ]
        for j in range(N) ] for i in range(N) ]

# each cell gets exactly one digit
for i in range(N):
    for j in range(N):
        m += xsum(x[i][j][k] for k in range(N)) == 1
# each row / column contains each digit exactly once
for k in range(N):
```

多解的方案：no-good 割循环枚举

给定题可能有多个合法解。每求出一个解，就追加一条“排除这个完整填法”的割，再求解；直到无解为止，即得全部解（下面演示取前若干）。

在上一个模型之后继续：反复禁止已求得的解，枚举多解。

```
# continue after the model above
solutions = []
while len(solutions) < 6:
    m.optimize()
    if m.num_solutions == 0:
        break
    sol = [[ next(k + 1 for k in range(N) if x[i][j][k].x >= 0.5)
             for j in range(N) ] for i in range(N) ]
    solutions.append(sol)
    # forbid this exact completion (no-good cut)
    m += xsum((1 - x[i][j][k]) if sol[i][j] == k + 1 else x[i][j][k]
```



```
for i in range(N) for j in range(N) for k in range(N)) >= 1
print("number of solutions found:" len(solutions))
```

多解的可视化

Givens

6		8	5					
		3	6	7	9			
4				3				7
2		4					8	
				9			1	
	9			2	4			
			7			8		2
7					5			
						3		

Solution 1

6	7	8	5	4	1	9	2	3
1	2	3	6	7	9	4	5	8
4	5	9	2	3	8	1	6	7
2	1	4	3	5	6	7	8	9
3	6	5	8	9	7	2	1	4
8	9	7	1	2	4	5	3	6
5	4	1	7	6	3	8	9	2
7	3	2	9	8	5	6	4	1
9	8	6	4	1	2	3	7	5

Solution 2

6	7	8	5	4	1	9	2	3
1	2	3	6	7	9	4	5	8
4	5	9	2	3	8	1	6	7
2	1	4	3	5	6	7	8	9
3	6	5	8	9	7	2	1	4
8	9	7	1	2	4	5	3	6
5	4	6	7	1	3	8	9	2
7	3	2	9	8	5	6	4	1
9	8	1	4	6	2	3	7	5

左：题目所给线索；中、右：该给定题的两个不同合法解。琥珀色高亮的格子即两种解取值不同的位置——这些“自由度”的存在，正是数独出现多解的根源。

参考文献：A. Williams, “When Natural Variables Are Not Enough: Teaching Integer Programming with Sudoku,” arXiv:2606.30227v1 [math.OC], 29 Jun 2026.

建模启示

方案 A 用 729 个“不自然”的变量换来了干净的线性约束；方案 B 用 81 个“自然”变量，却要为“互不相同”付出 1620 条 big-M 不等式的代价。选变量，就是在选约束的复杂度。

两个实例：可判定 vs 不可判定

可判定——路径可行性（最短路）

给定城市道路网与每段路的长度，问“是否存在一条从仓库到客户、总路程不超过 L 的路径？”只需跑一遍最短路算法（Dijkstra），时间关于路网规模为多项式。这是运筹学里最轻松的可判定问题之一。

不可判定——平面铺砖（Tilings of the plane）

给定一组正方形瓷砖，每条边被染成有限种颜色之一；平铺时相邻边必须同色。问“这组瓷砖能否铺满整个无限平面？”Berger（1966）证明它**不可判定**：该问题可以编码任意图灵机的计算过程，从而归约到停机问题——不存在能对所有瓷砖组给出判定的算法。比 NP-hard 更“负面”：根本没有精确算法。

可判定性与可处理性是不同的问题：前者问“有没有算法”，后者问“代价多大”。最短路可行性可判定且高效；而平面铺砖能否铺满无限平面，连“有没有算法”都做不到——这正是工程里“能建模”与“能求解”之间的鸿沟。

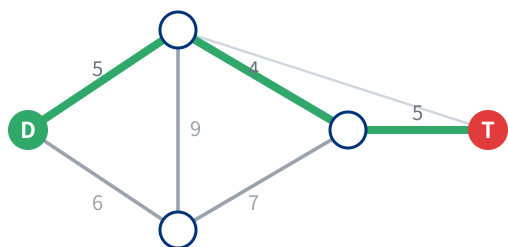
两个具体实例（带图示）

可判定 · 具体实例：道路网最短路可行性

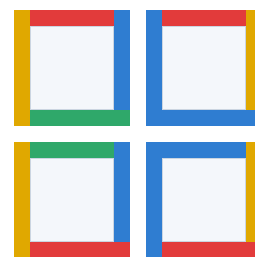
给定图 G 、边权 $w(e)$ 与阈值 L ，问“是否存在仓库 D 到客户 T 的路径总长 $\leq L$ ？”对所有节点跑一遍 Dijkstra，得到 D 到每点的最短路；若 $\text{dist}(T) \leq L$ 则存在。**时间 $O((V+E) \log V)$** ，关于路网规模多项式——一个必定在有限步内停机并给出正确答案的算法。

不可判定 · 具体实例：Wang 瓷砖能否铺满无限平面

给定一组正方形瓷砖，每条边染成有限种颜色（图中①②③④）之一；平铺时相邻边必须同色。问“这组瓷砖能否铺满整个无限平面？”Berger（1966）证明该问题**不可判定**：它能编码任意图灵机的计算过程，于是归约到停机问题——没有算法能对所有瓷砖组给出判定。



最短路径 $D \rightarrow A \rightarrow C \rightarrow T = 5 + 4 + 5 = 14 \leq L = 15$ ，存在；判定在多项式时间内完成。



■ ① ■ ② ■ ③ ■ ④

相邻边同色即匹配（图中已满足）。问题是：能否这样无限延伸下去？——不可判定。

一口气说清 P 与 NP

P 类

可由确定性算法在**多项式时间**内求解的决策问题——也就是我们能高效求解的问题。

NP 类

一旦给出证明证书，其“是”的回答能在多项式时间内被**验证**的决策问题；等价地，可由非确定性机器在多项式时间内求解。

- 因为任何求解器也能验证自己的答案，故 **P 与 NP 对决策问题定义**。
- 理论计算机科学的核心未解问题：**P 与 NP 是否成立？**
- 关键在于，**P 与 NP 只对决策问题定义**——这正是复杂度理论反复回到决策视角的原因。

为何重要

难度结论（NP-hard、NP-complete）都是针对决策版本给出的；证明决策版本困难，也就证明了优化版本同样困难。

NP-hard 与 NP-complete 的区别

NP-hard

若 **每一个** NP 中的问题都能在多项式时间内归约到它，则该问题是 NP-hard——即“至少和 NP 中最难的问题一样难”。若任一 NP-hard 问题有多项式算法，则 **所有** NP 问题都有多项式算法。NP-hard 问题不必是决策问题，也不必属于 NP（如 TSP 的优化形式）。

NP-complete

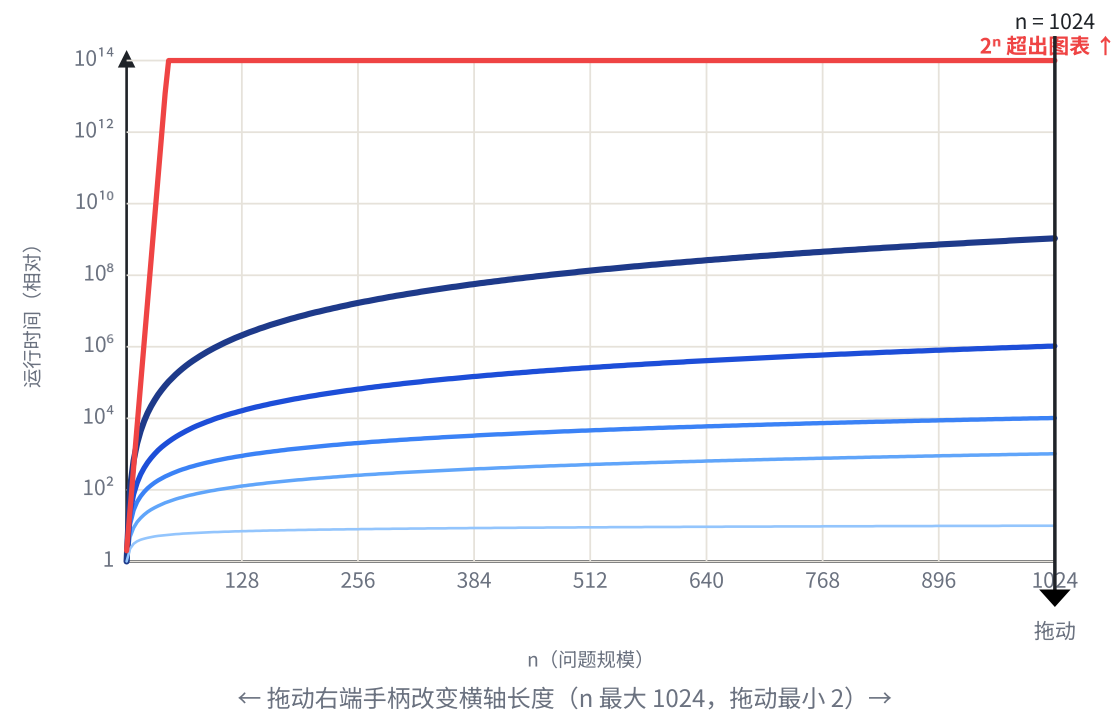
既属于 NP，又 NP-hard 的决策问题——即其“是”的回答在给出证书后可在多项式时间内被验证。

为何复杂度栖身于决策视角

旅行商问题（TSP）的优化形式是 NP-hard，却不在 NP 中（它是“函数值”问题）；而其决策版本（“是否存在长度 $\leq L$ 的环路？”）才是 NP-complete。这正是复杂度理论只针对决策版本的原因。

时间复杂度：多项式 vs. 非多项式

拖动横轴右端的手柄，把横轴拉长（或缩短），观察 n 变大时各函数如何增长。蓝线属于“多项式时间”，红线属于“非多项式（指数）时间”。



函数	在 $n = 1024$
$\log_2 n$	10
n	1024
$n \cdot \log_2 n$	10,240
n^2	1,048,576
n^3	1.1e9
2^n	∞

纵轴随 n 自动增长：但 2^n 的攀升速度远超任何多项式——即便坐标轴不断拉长，指数曲线也会迅速“顶满”整张图，这正是 NP-hard 问题在规模变大时“算不动”的根源。

优化问题

优化问题要求最优的 *值* 或 *最点*——例如“在满足所有截止期的调度方案中，哪一个使总成本最小？”

标准形式：

$$\begin{aligned} & \min && c^T x \\ & \text{s.t.} && Ax \leq b \\ & && x \in X \end{aligned}$$

x

c

A

b

X

x ：决策变量 · c ：目标 · A, b ：约束 · X ：可行集。优化问题就是一个 *附带量化目标* 的决策问题。

示例——0-1 背包（一个 MIP）

$$\begin{aligned} & \min && \sum_{i \in I} c_i x_i \\ & \text{s.t.} && \sum_{i \in I} a_i x_i \leq b \\ & && x_i \in \{0, 1\} \end{aligned}$$

二值变量 x_i 决定是否装入物品 i ；整数与连续变量混合，正是它成为 MIP 的原因。

我们在 X 中寻找使 f 最小的点。

示例：0-1 背包（一个 MIP）

实例：背包容量 **C = 100**，8 件候选物品（重量 *w*，价值 *p*）。每件最多取一次。

物品	w	p	密度 p/w
1	23	92	4.00
2	31	57	1.84
3	29	49	1.69
4	44	68	1.55
5	53	60	1.13
6	38	43	1.13
7	63	67	1.06
8	85	84	0.99

目标：在重量不超过 100 的前提下，选出价值之和最大的子集。

数学建模（混合整数规划 MIP）：

$$\sum_{i=1}^8 w_i x_i \leq C$$
$$\sum_{i=1}^8 p_i x_i \rightarrow \max$$
$$x_i \in \{0, 1\}$$

```
# 0-1 knapsack modelled as a MIP
from mip import Model, xsum, BINARY, maximize

items = [(23,92),(31,57),(29,49),(44,68),
          (53,60),(38,43),(63,67),(85,84)] # (weight, value)
C = 100 # capacity

m = Model(solver_name="CBC") # open-source COIN-OR CBC
x = [m.add_var(var_type=BINARY) for _ in items]

m += xsum(w * x[i] for i, (w, _) in enumerate(items)) <= C # capacity
m.objective = maximize(xsum(p * x[i] for i, (_, p) in enumerate(items)))
m.optimize()

chosen = [i + 1 for i in range(len(items)) if x[i].x >= 0.5]
print("value", round(m.objective_value), "items", chosen)
```

求解器返回精确最优；二进制变量 *x_i* 是 MIP 的核心。

MIP 结果：最优价值 **217**，装入物品 {1, 2, 4}，使用重量 98 / 100。

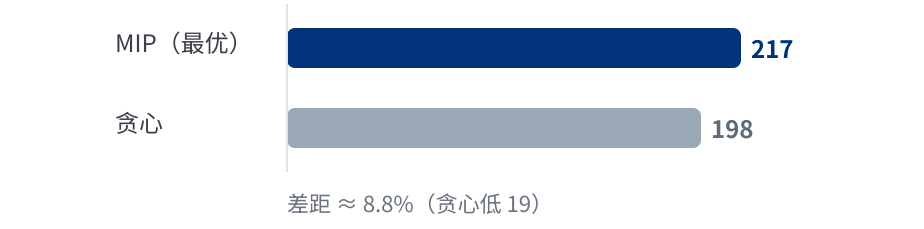
同例：贪心启发式 vs MIP

启发式：按价值密度 p/w 降序，能装就装（不回溯）。

```
# Greedy heuristic: best value-density first
items = [(23,92),(31,57),(29,49),(44,68),
         (53,60),(38,43),(63,67),(85,84)]
C = 100

order = sorted(range(len(items)),
               key=lambda i: items[i][1]/items[i][0],
               reverse=True)
cap, value, chosen = C, 0, []
for i in order:
    w, p = items[i]
    if cap - w >= 0:
        chosen.append(i + 1); cap -= w; value += p

print("greedy value", value, "items", chosen)
# -> greedy value 198, items [1, 2, 3], weight 83 / 100
```



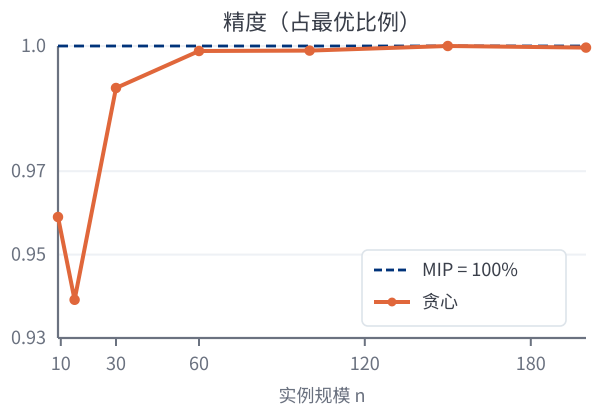
方法	物品	重量	价值
MIP (CBC)	1,2,4	98	217
贪心	1,2,3	83	198

运行时间 $O(n \log n)$ ，但局部最优不保证全局最优。

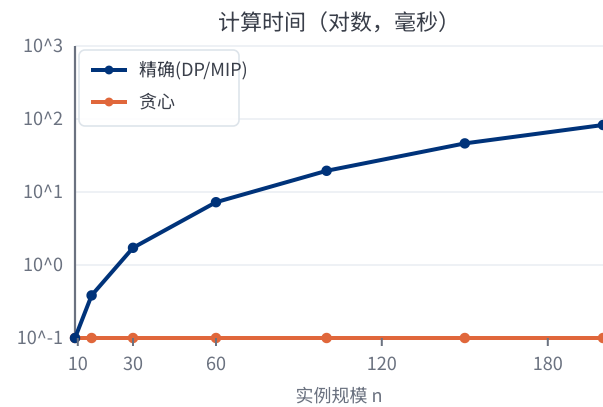
权衡：贪心仅得 198（物品 1,2,3），比最优低约 8.8%，却只需 $O(n \log n)$ ；MIP 保证最优，但最坏情况指数级。这正是“精确 vs 启发式”的典型取舍。

规模增大时：精度 vs 时间

同一组随机算例（容量≈45% 总重），贪婪与 MIP(CBC) 各跑一遍。横轴为物品数 n。



精度：MIP 恒为 100%；贪婪随 n 增大在 80%–98% 间波动，无法保证最优。



时间：贪婪几乎贴地 (<0.1 ms)；精确法(DP)随 n 上升更快——背包因有伪多项式 DP 而例外，一般 MIP(分支定界)在更大规模时耗时显著增长。

n	贪婪值	精确值	精度	贪婪 t(ms)	精确 t(ms)
9	351	366	95.9%	0.00	0.08
15	633	674	93.9%	0.00	0.38
30	1179	1191	99.0%	0.01	1.72
60	2516	2519	99.9%	0.01	7.23
100	3700	3704	99.9%	0.02	19.52
150	6244	6244	100.0%	0.03	46.19

n	贪婪值	精确值	精度	贪婪 t(ms)	精确 t(ms)
200	7975	7978	100.0%	0.04	82.71

附录 · 可直接复现

数据格式（与上一页代码直接兼容）

上一页的贪心与 MIP 代码只需一组 **物品列表** 与 **容量 C**。本页 7 组实例统一采用下列 JSON/Python 字典格式——把它展开成 `items` 与 `C`，两段代码即可原样运行。

```
# 与上一页完全兼容的数据格式（以 n=9 为例）
instance = {
    "n": 9,
    "capacity": 209,                # 背包容量 C
    "weights": [49, 43, 30, 22, 50, 74, 89, 37, 71],  # w_i
    "profits": [36, 50, 94, 98, 74, 2, 100, 32, 85],  # p_i
}
# 展开为上一页代码所期望的形式
items = [(w, p) for w, p in zip(instance["weights"], instance["profits"])]
C = instance["capacity"]
# — 上一页的贪心 / MIP(CBC) 代码原样可用 —
```

7 组测试数据（n = 9 → 200，可复现）

每个实例由 `random.seed` 固定生成：重量 w_i 、价值 p_i 各自独立取自 1-100 的均匀整数，容量 $C = \text{round}(0.45 \cdot \sum w_i)$ ；复制下面种子即可逐字节复现。上面 7 行结果表即由这些数据算出。

► 展开 7 组完整数据（JSON）

实验设置

精确最优由**动态规划**（伪多项式， $O(nC)$ ）求出，即 MIP 会返回的真值；贪心为按价值密度 p_i/w_i 降序装入的启发式。实例随机生成： w_i 、 p_i 各自独立取自 1-100 的均匀整数， $C = \text{round}(0.45 \cdot \sum w_i)$ ；每组由固定 random.seed 复现，计时用 time.perf_counter（多次运行取平均）。

工程问题

工程问题是一种真实世界的问题，我们必须从中选出最优的设计、计划或控制决策。**工程优化**借助模型与算法来做出这一选择。

大多为 NP-hard

调度、路径、装箱、网络设计——尚无已知的多项式精确算法。超出一定规模后，精确方法便不实用。

许多是在线的

决策随新数据陆续到达而顺序做出；待实例完整时，决策时机早已过去。需用竞争分析来评价。

一个有益的循环

真实系统*提出问题* → 理论*提供工具* → 解*反馈*到更好的设计，又引出新的（更难的）问题。

正因如此，我们能部署的*最好解*几乎从不是理论上的*最优解*——也正因为如此，即使用启发式或学习来求解，形式化模型依然不可或缺。

工程问题 $\neq$ 优化问题 $\neq$ 决策问题

落到工程师桌上的真实问题，承载了远超数学镜头的内容。模型是强大工具，但只是更复杂整体的一个简化。

- **目标存在争议**：“最优”很少被给定，不同干系人各执一词。
- **人与组织语境**：合同、法规、安全规范从不以等式出现。
- **隐式约束**：物理定律、不成文规则、经验。
- **不确定性与动态**：许多问题是“在线”的，边解世界边变。
- **杂乱数据**：稀疏、含噪、有偏、来得快。
- **风险、安全、伦理**：纸面上可行最优的点，可能不安全或不公。

成熟的工程师：先解模型，再问“模型遗漏了什么”。

在线问题 vs 离线问题

离线（批量）问题

在**任何**决策做出之前，实例已完全给定：所有数据、参数、完整输入都已知，求解可以从容进行。

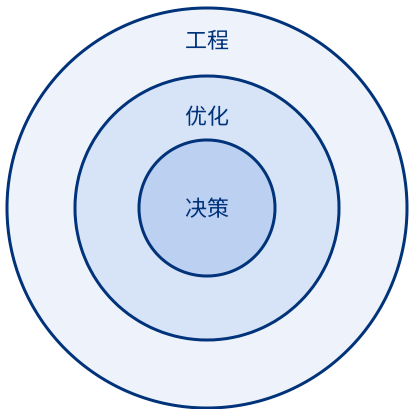
在线问题

数据逐步揭示；每个决策必须在后续数据到达前做出，且先前的决策通常无法撤销。

如何评价在线算法

用**竞争分析**：将其代价，与一位“全知”的离线最优（它提前知道一切）相比。例：新订单陆续到达、机器突发故障——待数据完整时，决策时机早已错过。

一条层级链：工程 → 优化 → 决策



三个集合按包含关系嵌套；最外层最广。

优化与决策在多项式意义下等价：

引理——优化可归约为决策

若某优化问题可在时间 T 内求解，其决策版本可在正比于 T 的时间内求解：固定阈值 ϵ ，解该优化问题，再把最优值与 ϵ 比较。



每一步都比前一步更精细；决策版本最为简单——复杂度理论也栖身于此。

用决策“预言机”二分搜索还原最优值

相反方向也成立：若我们拥有一个**决策预言机**——“是否存在可行 使 ？”——就能通过在区间上**二分搜索** 来求出最优值。这正是引理之逆，二者共同证明决策与优化**多项式等价**。



若目标为整数且在已知区间 $[L, U]$ 内，只需约 $\lceil \log_2(U - L) \rceil$ 次查询即可锁定最优值。

为何要建数学模型？

在选择方法之前，先回答那个关键问题：**这个问题能否被建模为一个数学模型？**

- **沟通。**一种建模即是一种通用语言，能消除关于“优化什么、在什么条件下优化”的歧义。
- **评价基准。**模型的最优值是每个启发式、学习方法或仿真都必须超越的标杆。
- **教学。**同一问题的多种建模方式成为教学资产——培养变通而审慎的思维。
- **借助学习扩展。**机器学习利用结构特征来缓解求解器的局限，使 MILP 得以应对更大、更真实的问题。

当建模失败时

目标存在争议、约束无法写出、系统理解不清，或数据太快/太乱 → 退而求其次，改用学习、启发式与仿真。

可建模 vs. 不可建模

可建模 → 数学方法

组成：目标 、决策变量 、约束。求解器：Gurobi、CPLEX（商用）；GLPK、CLP、SCIP、CBC（开源）；BARON（全局 NLP）。

不可建模 → 直接学习或搜索

这条边界并非绝对：今天只能部分建模的问题，明天或许就能完整建模。那个诚实的第一问——“我们能把它写下来吗？”——决定了此后整套工具箱的去向。

MIP 是 NP-hard, 线性规划属于 P

MIP——NP-hard

整数/二值约束使得一般混合整数规划成为 NP-hard (Karp, 1972)。目前没有已知的多项式精确算法；规模变大时，求解器退而使用分支定界 / 分支切割 / 启发式。

LP——属于 P

只有连续变量与线性约束。Karmarkar (1984) 给出了在多项式时间内求解 LP 的内点法 → 。(单纯形法实践中很快，但最坏情形是指数级。)

但复杂度说的是增长率

即使 LP 也可能有数千万个变量；这类实例仍需数天乃至数周 (Klotz & Newman, 2013)。而且线性规划无法在多项式时间内求解大型 NP 完全问题 (Hofman, 2006)：松弛求解很快，却给不出精确的整数解。

正是这条边界——存在可证明的精确方法 (LP) vs. 可证明不存在 (MIP) ——促使本书在整数情形上投入分解、学习与启发式。

翻箱问题：一个可交互的 NP-hard 游戏

按优先级 1,2,3,⋯ 依次取走箱子。若目标箱子被压住，必须先把它上方的箱子挪到别的堆——每次挪动都算一次“翻箱”。目标是用最少的翻箱次数取完所有箱子。

规则：① 每列箱子由下往上堆叠，有最大高度上限；只能挪动每列最顶层的箱子，放到另一列的最顶层（满列不可放）。② 按固定取箱顺序 1→2→3⋯（即每次取当前全局最小优先级）依次取走：目标已在顶部→直接取走（不计翻箱）；被压住→先把它上方的阻挡箱逐一挪到别处，每挪一个 +1 次翻箱。③ 成绩 = 翻箱次数（越少越好）；另按堆场参数统计挪动距离 / 时间供参考。④ 规则变体：受限（只允许挪动当前目标的阻挡箱）/ 不受限（可提前挪动任意顶箱以优化后续）。

操作方式：用鼠标按住最顶上的箱子，拖到另一个堆即可挪动；要取走时把它拖到右侧「取货区」（仅当该箱子是当前全局最小才允许）。

3

1

A

4

2

B

C

取货区

↶ 撤销

重置

换一题

🚚

取走

当前目标 = 1，被压在堆 A 下面，上方还有 1 个阻挡箱。需先把它们逐一挪到别处（每次 +1 次翻箱），直至目标到顶。已翻箱 0 次。

您的操作（每步）

最优解

点击「计算最优解」查看最少翻箱次数与步骤。

计算最优解

播放动画

暂停

您的成绩

挪动次数	0
挪动距离	0 m
挪动时间	0 s

最优解基准

难度：简单中等困难

规则：受限不受限

堆场参数（可设置）

贝位间距 (m)

吊装耗时 (s)

每米运输 (s/m)

最少挪动	—
最少距离	—
最少时间	—

通往 NP-hard 的漫漫长路：集装箱翻箱

2006 · Kim & Hong

首次定义了该问题，并给出分支定界算法——*保证*返回最优解。方法精确，但运行时间随规模急剧增长。

2012 · Caserta 等

给出数学建模，并*证明*该问题为 NP-hard。形式化难度结论，比工程界开始研究它晚了六年。

先求解，后理论化

工程中常见的顺序是：先交付一个可用的方法（启发式、分支定界、学习策略），*之后*才问“是否有形式化难度结论能解释它为何难”。NP-hard 标签是**回顾性确认**，而非前提；没有证明，从不应阻止工程进展。

参考文献

1. Kim, K. H., & Hong, G.-P. (2006). A heuristic rule for relocating blocks. *Computers & Operations Research*, 33(4), 940–954.
2. Caserta, M., Schwarze, S., & Voß, S. (2012). A mathematical formulation and complexity considerations for the blocks relocation problem. *European Journal of Operational Research*, 219(1), 96–104.

CHAPTER 2

问题分解

大问题很少能作为一个整体被求解。我们拆分、局部求解，再协调。

为何要分解？

- **规模**：2000 辆车路径规划作为一个单体问题难以求解。
- **结构**：松耦合的子系统只通过少数共享量相互作用。
- **并行性**：相互独立的子问题可并发求解。
- **专业性**：不同团队负责不同的子问题。

陷阱

分解画起来容易，协调起来昂贵——切口必须落在耦合最弱处。

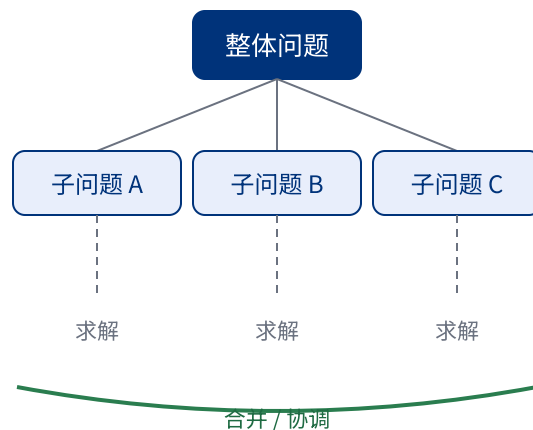
分治法 与 分而治之

西方：分治法

递归地把问题拆成同类子问题，求解后再**合并**——且合并代价被显式计入（归并排序，快速排序会退化到 $O(n^2)$ ）。

中国：分而治之

“分而治之。”侧重何时分、以及各部分如何保持整体——分与合是一种往复运动（分久必合，合久必分）。



拆分 → 求解 → 合并。代价（与风险）正藏在合并这一步。

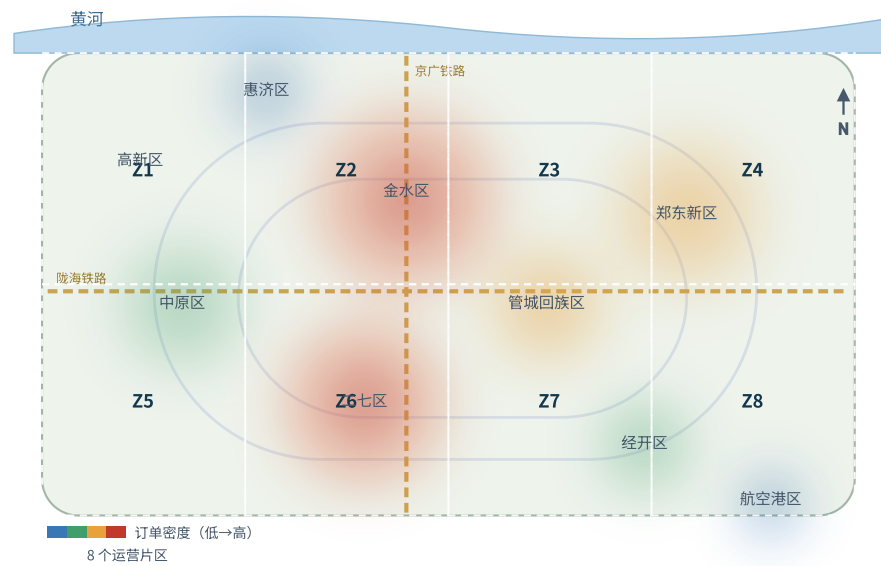
真实工程案例

- **建筑与拆除 (C&D) 物流**: 按空间 + 按阶段分解 (收集 → 转运 → 分拣 → 处置); 回收厂是瓶颈 → 紧耦合协调器。
- **大型建设工程群调度**: 按主体 / 区域分解; 共享塔吊、井道、场道造成紧耦合 → 由项目级调度器协调。
- **晶圆厂排程**: 递归 / 层级分解 + 再入流; 工具组是瓶颈 → 由 fab 级调度器协调。呼应第四篇的 Benders / Dantzig-Wolfe 方法。

一个分解实例：城市配送（以郑州为例）

把“全市当日配送”这个整体问题，按 **空间** 与 **时间窗** 两刀切下去。下面以郑州主城区为例。

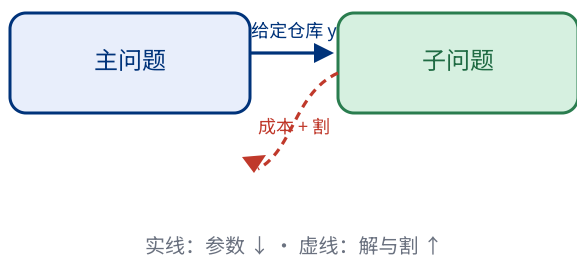
- **按空间**：把郑州划成 8 个运营片区，每区约 250 单。
- **按时间窗**：每区再按上午 / 下午切成子批。
- **并行求解**：16 个子问题各自独立跑路径算法，耗时从“数小时”降到“几分钟”。
- **协调**：跨区订单与共享车队由上层协调器兜底。



郑州主城区：热力图=订单密度，白色虚线=8 个运营片区；黄河横亘北部，京广 / 陇海铁路十字交叉。

分解的一个数值小例

以“仓库选址—配送”为例：主问题决定开哪些仓，子问题在给定仓库下做配送路由；二者通过一次迭代达成协调。



主问题把“开哪些仓”钉死，子问题回报成本与一条可行性割，主问题据此更新决策。

迭代 0

主问题猜“只开仓库 A”。子问题发现 B 区订单无法在时限内送达 → 记为不可行，并回传一条割：必须再开一个仓库。

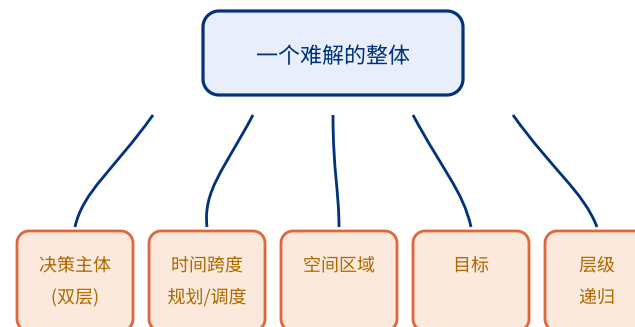
迭代 1

主问题加上割，改猜“开 A 且开 B”。子问题求得成本 80 且全部可行 → 协调完成，得到整体可行解。

分解的五类常见切口

每个问题都不同，但反复出现的就是那么几种切口；认出它们，后面才能对症下药选协调机制。

- **按决策主体**——分给相互独立的行动者（双层 / Stackelberg）。
- **按时间跨度**——规划 / 调度 / 控制，尺度不同。
- **按空间区域**——弱耦合的子系统（泊位、堆场、车辆）。
- **按目标**——先标量化再协调（目标规划）。
- **按层级 / 递归**——一直分解到每片叶可解（Benders、Dantzig-Wolfe）。

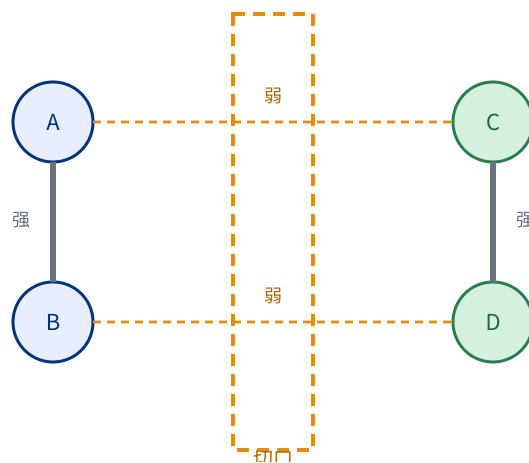


一个难解的整体，可沿五种反复出现的维度切分；选哪一刀取决于问题结构。

切口怎么选：画一张耦合图

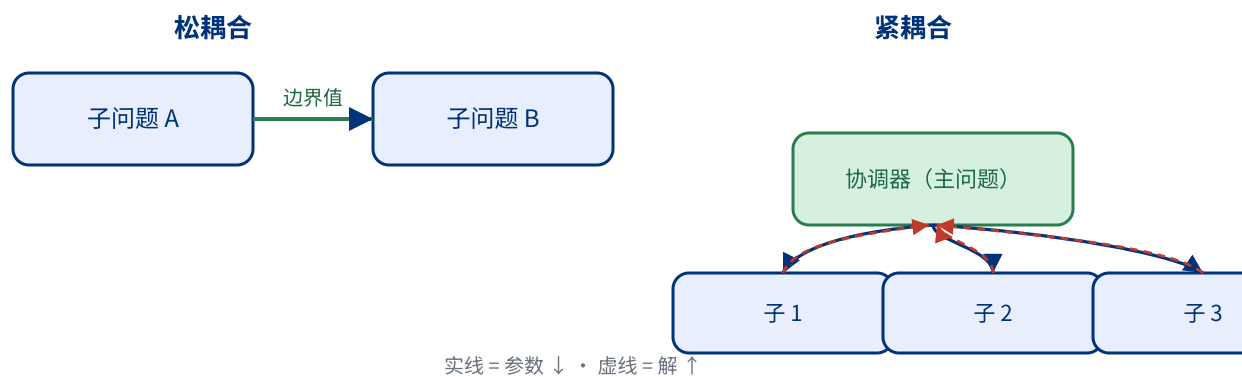
分解的第一步不是“怎么切”，而是“**在哪切**”。把系统画成一张图：节点是子系统，连线是它们之间的依赖与数据交换，连线的“粗细”就是耦合强度。

- **弱耦合处下刀**：切口应穿过最细的连线——被切断的依赖越少，协调器要背的锅越小。
- **切口越短越好**：切口越长，跨边界交换的变量越多，协调越贵。
- **切口要平衡**：一侧过大仍难解，应在“可独立求解”与“耦合最弱”之间折中。



强耦合留在同一侧（A-B、C-D），让细弱的跨边界连线成为唯一的协调通道。

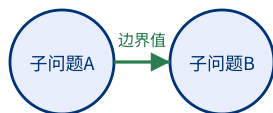
两种协调模式



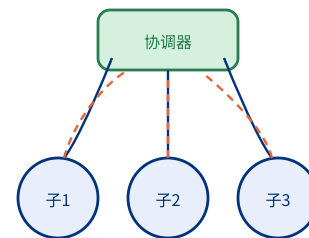
松耦合：一次边界值交换。紧耦合：协调器与每个子问题反复迭代参数与解。

两种协调模式：松耦合 vs 紧耦合

切分会产生耦合约束——把子问题拴在一起的共享变量。一个可行解不仅要满足自身约束，也要满足耦合。



松耦合：交换少量边界值即可



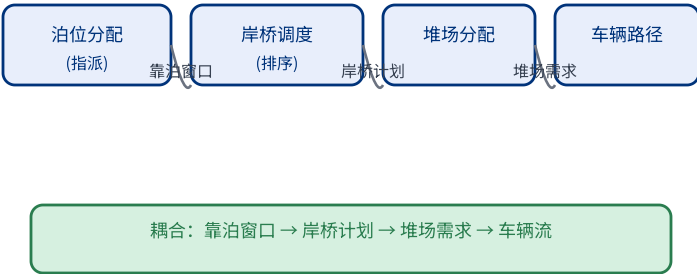
紧耦合：需协调器驱动至一致

判断

松耦合只需交换边界值；紧耦合（共享量是瓶颈）需要协调器——主问题或迭代式方案——驱动子问题收敛到全局一致解。

集装箱码头：四个弱耦合子问题

一个集装箱码头，至少要做四件事：分配泊位、排布岸桥、堆放堆场、调度车辆——每一件都有自己的变量、约束与最优方法。



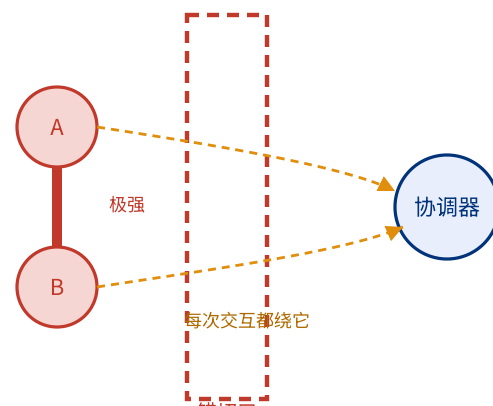
四个弱耦合子问题，只通过少数共享量相互作用

整体从不被一次性求解，而是作为一组可解部件的联邦来求解；共享量被反复协商直到一致。

一个反例：切口选错，不如不分解

回到 s-c2a 的提醒：分解画起来免费，协调起来昂贵。若切口落在**耦合最强**处，协调器会变成吞吐瓶颈，迭代次数爆炸，通信与重算的成本盖过分解省下的求解时间。

- **错切口**：把一对本应同处一边的强耦合子系统切开 → 它们之间每一次交互都要绕协调器。
- **后果**：协调器既算得慢、又成了单点故障；整体反而比单体更慢、更脆弱。
- **对策**：先画耦合图（s-c2i），让切口穿过最弱连线；必要时宁可
不分解，也不要硬切。



把强耦合的一对切开，等于逼着它们每说一句话都先经过协调器。

分解的代价与风险

- **一致性**：各子问题各自最优 $\neq$ 整体最优；割裂的局部解在边界处互相打架。
- **协调开销**：紧耦合时迭代次数可能爆炸，通信与重算成本超过分解带来的好处。
- **失去全局视野**：看不到跨子问题的权衡（例如安全 vs 成本），易做出局部好看、整体糟糕的决策。
- **切口选错**：切在耦合最强处，协调器立刻成为瓶颈——比不分解更糟。
- **收敛迟缓**：迭代可能很慢才收敛；工程中常“够好就停”，接受次优。

一句话

分解是手段，不是目的——切口选错，比不分解更糟。

分解得到的是“真最优”吗？

不一定。这取决于协调器能否证明收敛。

精确分解

Benders / Dantzig–Wolfe 等：主问题与子问题反复交换割与界，收敛后可证明——分解解 = 整体全局最优。分解只是算法手段，不改变答案。

启发式分解

贪心切分、固定迭代次数、或“差不多就停”的 ADMM：得到的是可行解 / 近优解，没有最优性证书。它便宜、快，但你要清楚自己放弃了保证。

一句话判据

协调器若能证明收敛（割收敛 / 对偶界闭合）→ 分解解 = 整体最优；否则只是可行或近优。第 2 章的仓库选址例子属于前者，城市配送的“16 个子问题各自求解”若不再协调则往往属于后者。

精确分解

收敛 →

= 全局最优

Benders / D-W

启发式分解

停手 →

可行 / 近优

贪心 / 固定迭代

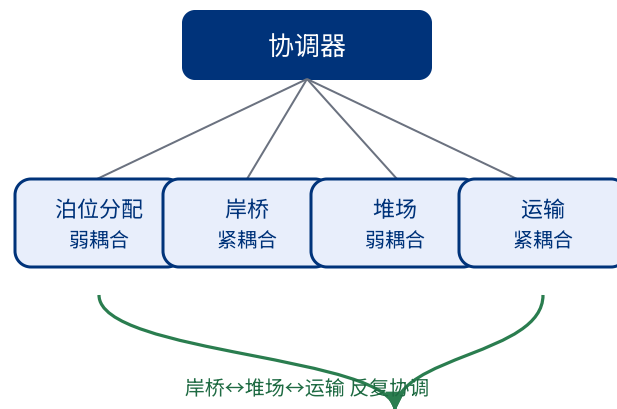
同样是“分解”，答案的确定性截然不同。

本章要点

- 大问题靠**分解**；分解靠**切口**；切口要落在耦合最弱处。
- 西方“分治法”显式计入**合并**代价；中国“分而治之”强调分合的**往复**与整体。
- 松耦合：一次边界值交换。紧耦合：协调器与子问题反复迭代。
- 分解不是免费午餐：一致性、开销、全局视野都要付出代价。
- 本书后续（Benders / Dantzig-Wolfe / 启发式）都是“如何切、如何协调”的具体答案。

案例：集装箱码头的四子问题分解

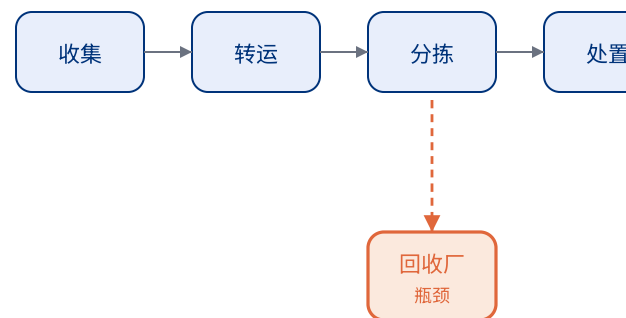
- **泊位分配**：到港船应在哪一段岸线停靠。
- **岸桥调度**：每台岸桥服务哪些箱位。
- **堆场计划**：进口/出口箱落位何处。
- **水平运输**：AGV/集卡在岸桥与堆场间转运。



切口落在最弱耦合处：泊位/堆场可独立，岸桥—堆场—运输需紧密协调。

案例：建筑废弃物（C&D）物流的分解

- **按空间：**收集点、转运站、分拣中心、处置场各在不同的地理位置。
- **按阶段：**收集 → 转运 → 分拣 → 处置，是一条串行流水线。
- **瓶颈：**回收厂吞吐受限 → 成为紧耦合的协调器。



串行流水线 + 一个瓶颈协调器

把整条物流链按空间与阶段两刀切下；回收厂吞吐限制使其成为全局瓶颈。

CHAPTER 3

解

“解”并非单一概念。可行、局部、全局、近优——每一种都有精确的含义。

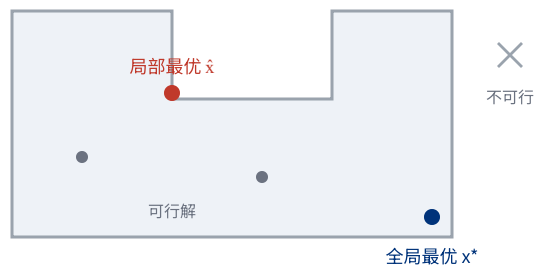
可行 · 局部 · 全局

可行集（表 3.1 记号）：

可行解
：满足全部约束。
局部最优（最小化）
$\star$ ε $\star$ $\circ$
全局最优（最小化）
$\star$ ，即 $\star$ $x \in X$ $\circ$

默认：最小化。最大化时不等号反向，并将 min↔max 互换。

可行集 X（非凸）中的点分类



灰色阴影为可行集 X；红点为局部最优（邻域内最低，却非全局），蓝点为全局最优。区域外的叉号表示不可行解。

近优解

近优解 (ϵ -最优, 最小化)

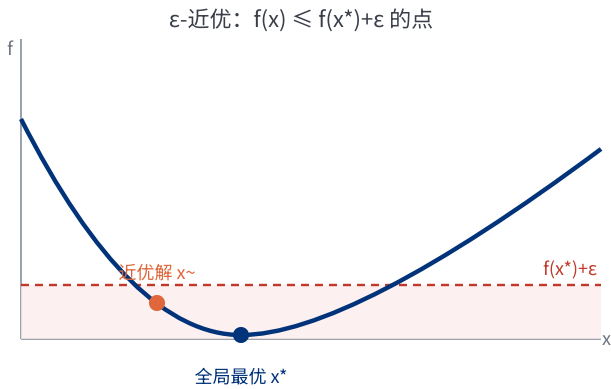
★

★

工程中我们很少能 *证明* 全局最优；通常退而求其次，接受一个可证明的近优解——而那份证书（对间隙的界）往往正是实际部署决策所需要的。

注

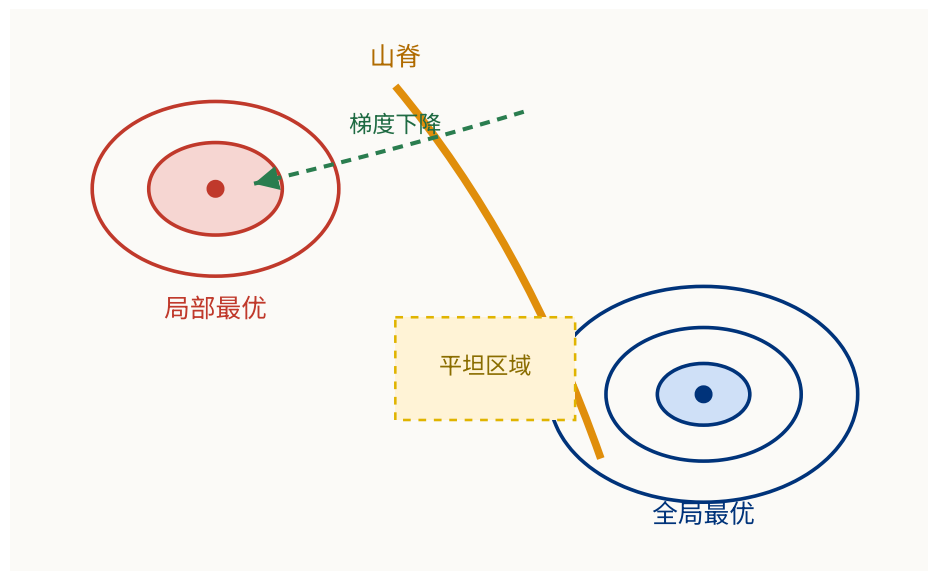
最大化： ★ ○



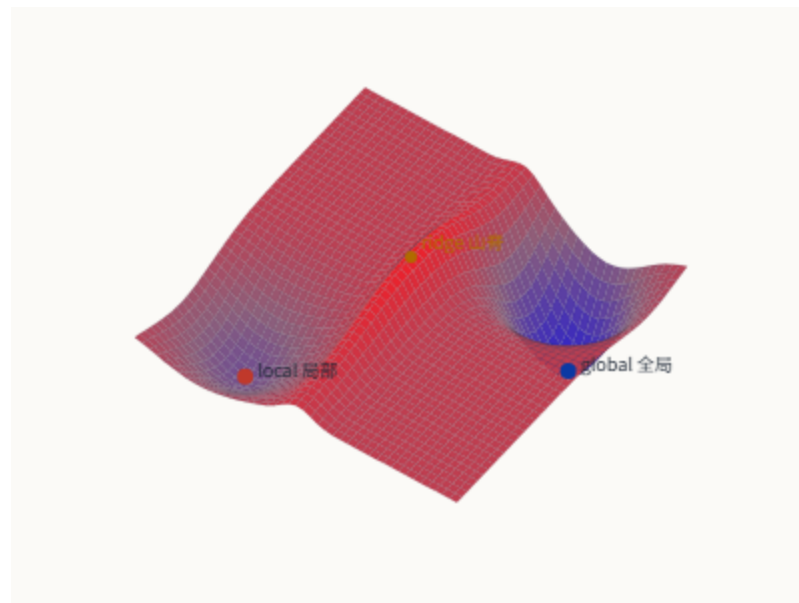
红色虚线下方（浅红带）是 ϵ -近优集：其中任意点的目标值都不超过全局最优再多 ϵ 。工程上常接受这样一个 $x_{\sim}$ ，并把“间隙 $\leq \epsilon$ ”作为交付证书。

非凸地形：二维等高 vs 三维 relief

同一座非凸地形，俯视是嵌套等高线，侧看才是真正的“坑有多深”。三维 relief 让“更深的全局谷、更浅的局部谷、横贯的山脊、平坦区域”一目了然——这些正是梯度法会被困住、停滞或走“之”字的根源。



俯视等高：蓝盆=全局、红盆=局部、暖色山脊、黄色平坦区域；绿色虚线为梯度下降轨迹。



自动旋转

线框

标注最优点

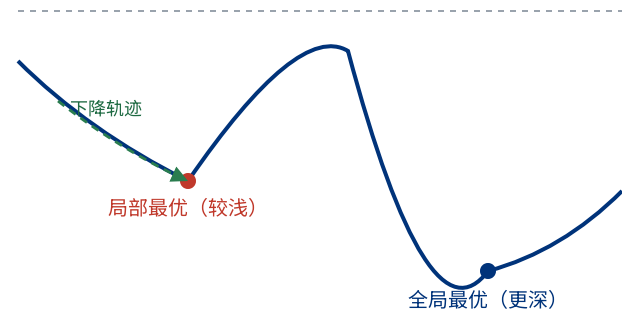
复位视角

拖动曲面可用鼠标旋转观察；蓝点=全局最优 (2.0,-1.5)，红点=局部最优 (-1.8,1.8)，暖色山脊横贯中部，右上肩为近恒定平坦区域。

一个数值小例：局部 ≠ 全局

考虑一维目标函数 $f(x) = \sin(x)$ ，在 $[0, 2\pi]$ 上求最小值。

- **局部最优**：从 $x = \frac{\pi}{2}$ 出发沿下坡走，停在 $x = \frac{3\pi}{2}$ ， $f(x) = -1$ 。
- **全局最优**：同一函数在 $x = \frac{7\pi}{2}$ 也取到 -1 —— 本例两者值相同，但高维时局部点往往更差。
- 启示：梯度类方法“目光短浅”，容易停在第一个谷底；要找全局，需要多起点、随机性或全局搜索。



从左侧出发的下降法会停在第一个（较浅的）谷底——它并不知道右边还有更深的全局谷。

凸性为何重要：凸 $\rightarrow$ 局部 = 全局

上一节看到：非凸地形里，梯度类方法会停在第一个谷底。凸性是一道分水岭——它让“局部最优”与“全局最优”成为同一件事。

凸问题

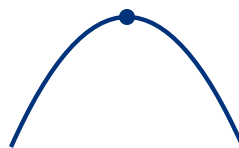
可行集 是凸集，且目标 是凸函数（向下开口的碗）。任何局部最优都自动是全局最优——无需担心“有没有漏掉更低的谷”。

非凸问题

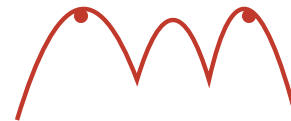
存在“坑”与“山脊”，局部最优 $\neq$ 全局最优。这正是 $LP \in P$ 而 MIP 是 $NP\text{-hard}$ 的根源之一：线性规划是凸的，混合整数规划不是。

凸：一个谷

非凸：多谷



局部=全局



局部 $\neq$ 全局

凸地形里无论怎么滑，都只会到一个谷底；非凸地形里滑到哪个谷，取决于起点。

凸集与凸函数：凸性的两个构件

上一节说“凸问题让局部 = 全局”。凸性由两个独立概念构成，都需要形式化定义。

凸集 X

任取两点，其连线仍在集合内：

1

2

1

2

。

凸函数 f

任取两点，弦在函数图像之上：

1

2

1

2

。



“凸函数”的“凸”指函数图像向上开口（碗形），弦在图像之上。线性函数既凸又凹——这是 LP 易解、MIP 难解的另一面。

如何从局部走向全局

既然梯度法“目光短浅”，要找全局就需要拓宽视野。下面是一组常用策略，核心思想只有两个：**多试几个起点**，或**偶尔允许向上走**。

- **多起点 / 随机重启**：从多个随机起点各跑一次梯度法，保留最好的谷。简单、并行友好，但无法保证找到全局。例：生产排程中对多个初始排程各跑一次局部改进，取最优者。
- **模拟退火**：以一定概率接受“暂时变差”的步，借温度逐步下降逃出局部谷；温度 schedule 决定探索与利用的权衡。例：芯片布局布线中“受热”接受暂时更差的摆放，避免卡在局部拥挤解。
- **进化 / 群体算法**：维护一“种群”，靠选择、交叉、变异在解空间里广度搜索；适合黑箱、非凸、高维。例：车辆路径（VRP）常用遗传算法在多车分配上广度搜索，求得近优配送方案。
- **精确法兜底**：若问题可建模为凸或小规模 MIP，分支定界 / Benders 能 *证明* 全局最优，无需上述“赌运气”的策略。例：航空维修排班这类结构化 MIP，用求解器直接给出带间隙证书的最优排班。

记住

梯度法给出“局部最优”是 *保证*；全局最优通常需要多起点、随机性或精确法的配合——这正是第 4 章“方法”要展开的。

最优性条件：怎样才算“最优”

前面讲清了“局部”与“全局”的定义。但拿到一个候选点，我们怎样判断它是否最优？答案是一组可检验的**条件**。

无约束（一阶必要）

* : 最优点处无下降方向可言。

约束问题（KKT）

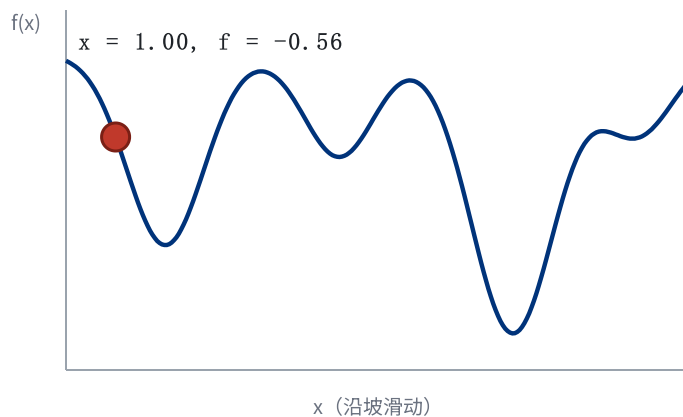
对 $\min f^*$ s.t. $g_j(x) \leq 0$ $i \in I(x)$ ，最优解满足：平稳性、原始可行、对偶可行、互补松弛。这是判断候选点的“体检表”。

关键洞见

对**凸问题**，KKT 条件不仅是必要的，更是**充分的**——满足它的点一定是最优的。这正是“ $\text{凸} \Rightarrow \text{局部} = \text{全局}$ ”的形式化根基，也是求解器能开具最优性证书的依据。

动手试：拖小球，看它滚进哪个谷

下面的曲线与模拟退火演示（第 53 页）所用的是同一个多峰函数（[点击查看函数表达式](#)），含多个局部极小、仅一个全局最小。用鼠标**拖动小球**放到任意位置，松手后它会“顺坡下滑”到最近的谷底。换几个起点，看它是否总能找到真正的全局最低。

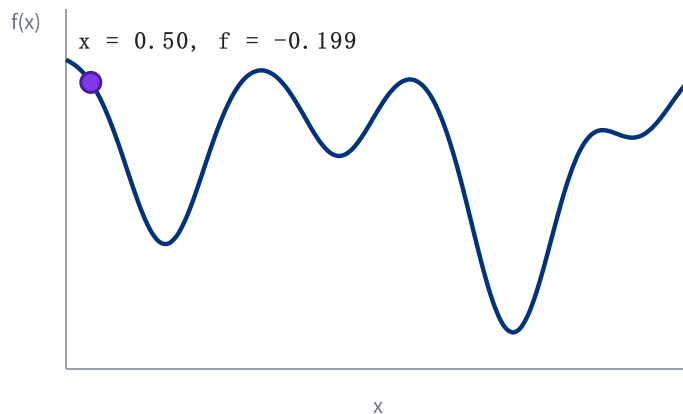


松手即下滑；起点不同，停下的谷也不同——这正是“局部最优 $\neq$ 全局最优”的直观演示。

重置

动手试：多起点与模拟退火

下面的曲线有多个局部极小，其中 $x \approx 9$ 处是最深的全局极小。点击「跑多起点」从多个随机起点各降一次；点击「跑模拟退火」允许偶尔向上跳——小球会“受热”跳出浅谷（带跳跃弧线），看它能否最终落到更深的全局谷。



多起点把所有落点都标出（橙），其中最低者标蓝（全局）；模拟退火的小球会“受热”跳出浅谷，最终偏向更深的全局谷。

▶ 播放

退火 0% · $T = 1.20$

跑多起点

重置

倍速

0.5×

1×

2×

4×

记号与可行集

在求解任何优化问题之前，我们需要一套共享的记号。

符号	含义
$x \in \mathbb{R}^n$	决策变量向量
$f(x)$	目标函数（极小化 / 极大化）
$h_i(x) = 0$	等式约束
$g_j(x) \leq 0$	不等式约束
X	可行集（ x 的定义域）
x^*	一个最优解

可行集收集了满足所有约束的点：

$$x \in X \text{ 且 } \begin{cases} h_i(x) = 0, & i = 1, \dots, m \\ g_j(x) \leq 0, & j = 1, \dots, p \end{cases}$$

为何在实践中证明最优很难

证明最优的机制——光滑凸问题的 KKT、离散问题的界与最优性间隙——都假设问题是固定、已知、且评估廉价的。真实工程同时违反这三点：

- **非凸、粗糙的地形**——大量局部极小、平台、山脊；没有 KKT 证书，也未必有易处理的松弛。
- **规模下的 NP-hard**——调度、路径、装箱随规模组合爆炸；证明最优（间隙 = 0）在稍大之后便不可行。
- **在线、漂移的数据**——等你解完，你解的那个问题早已不是你面对的问题。
- **模型与测量误差**——真实目标只是代理；有噪声或偏差的数据会让“已证明”的最优离真正的好决策很远。
- **不确定性与风险**——在期望上最优的点，在最坏情况下可能灾难性。
- **时间、数据与算力预算**——决策时刻有截止期；等待证书本身就是一种成本。

结论

实践中我们几乎从不确定“最优”。我们给出界（gap），接受近优或启发式解，并在新信息到来时重解。

实践中的最优解：工具箱

两个结构性事实迫使我们追求“实践中最好的解”：多数工程优化问题是 NP-hard，而数据往往来得太晚、等不及最优。我们靠一套工具箱来获得它：

启发式与元启发式

遗传算法、模拟退火、禁忌搜索、蚁群——数秒给出强可行解；无最优性保证。

数学启发式

把精确求解器嵌入元启发式：兼得精确的质量与启发式的速度。

近似算法

多项式时间，带可证明的最坏情况界（近似比）。

滚动时域

只解当前窗口，提交早期决策，数据到来后再解下一窗口。

基于学习/反应式

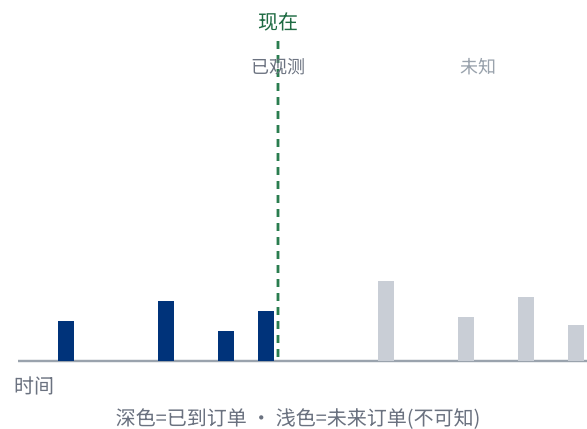
从问题特征预测有希望的区域或好的初值；对突发情况即时反应。

原则

给定容差，跑到截止时间或达到 gap 目标为止，接受手头最好的可行解。

案例：在线订餐调度（无离线最优）

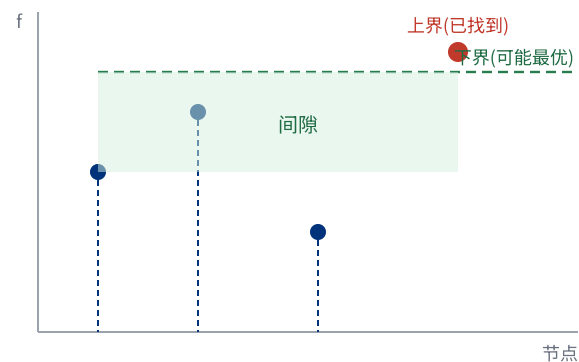
- 订单**随时间到达**：调度器在 t 时刻只知道已到的单。
- 离线最优需**预知未来**——这在现实中不可能。
- 因此工程上求**在线近似最优** + 可证明的竞争比。



离线最优需要看到全部订单——但未来订单尚未到达，故只能做在线决策。

案例：最优性间隙证书

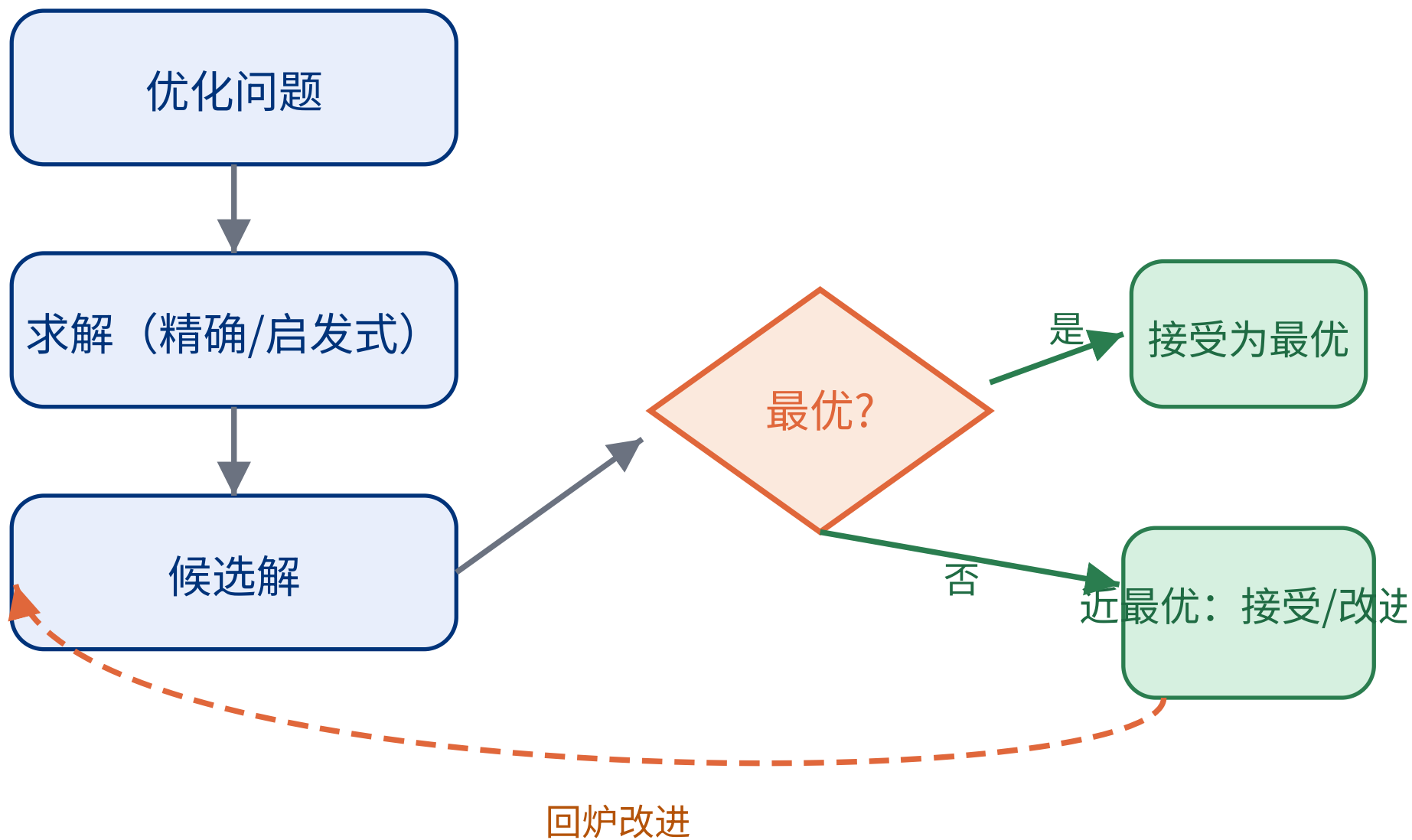
- 分支定界维护两条界：**上界**（已找到的最佳解）与**下界**（剩余子问题的最佳可能）。
- **间隙** = 上界 - 下界；间隙为 0 即证明全局最优。
- 工程部署要的是这份**证书**，而非无限逼近。



当上界（红）与下界（绿）重合，间隙归零，全局最优得证——这就是部署所需的证书。

解的生周期

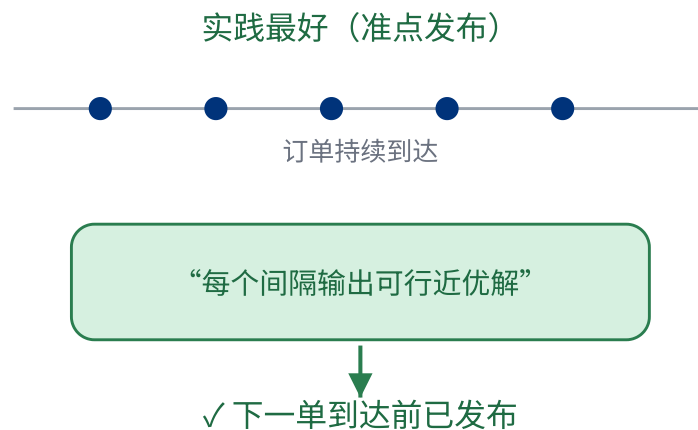
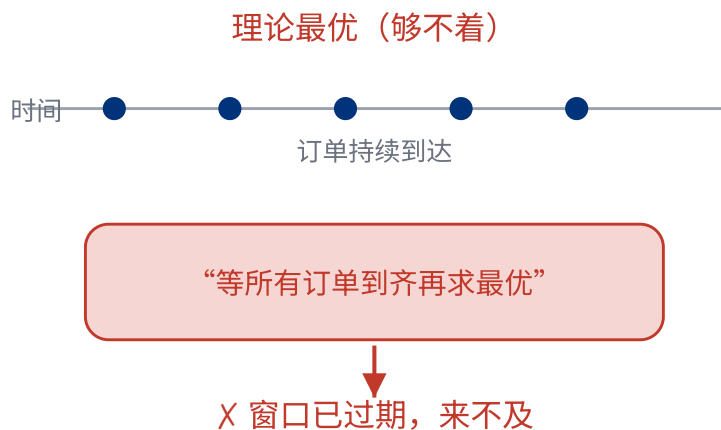
一个解从“问题”走到“被接受”，是一条会自我修正的流水线：建模求解得到候选，检验是否最优；是最优就接受，否则作为近最优接受或回炉改进。



“是否最优”是分流阀：是则开具最优性证书，否则作为近最优接受或回炉。多数工程问题到此停在“近最优”。

理论最优 vs 实践最好

订餐调度这类问题“最优解”在理论上是存在的，但新订单源源不断到达（在线）。等到所有订单到齐再求最优，送达窗口早已过期——调度员只能交付“此刻可得的最好解”。



“最优解”与“最好解”之间的差距不是方法的失败，而是实时工程问题的必然代价。

CHAPTER 4

方法

精确、启发式，还是近似——各自在不同约束下有其位置。

面向优化理论的方法

方法	适用情形	优势	劣势
精确法 (分支定界、分支切割、DP、 Benders、D-W)	忠实模型、LP 或小规模/结构化 MIP、需要证明	可证明最优；最优性间隙证书； 求解器成熟	大型 MIP 可能爆炸；需要精确 闭式模型
启发式与元启发式	NP-hard、高维、精确法太慢	快、灵活，可处理黑箱 / 非凸	无最优性保证；结果随运行与 调参波动
近似算法	需要带界多项式时间	多项式时间；可证明最坏情况界	界可能松散；并非所有问题都 存在
学习式方法 (神经 / 强化学习)	数据丰富、模型不全、在线 / 高维模 式	推理快；能抓复杂模式；可端到 端	需数据；sim-to-real 差距；可 解释性差

面向真实工程问题的方法

方法	作用	权衡
建模与抽象	由粗到精的简化	保真度 vs. 可解性
基于仿真	评价而非优化	昂贵；无梯度
从数据学习	代理模型 / 策略	需数据；存在 sim-to-real 差距
多解与鲁棒选择	对冲不确定性	保守；模型更难
人在回路	验证与修正	慢；能捕捉建模错误

如何选方法：一张决策清单

- 问题够小、模型精确 → 用精确法（MILP 求解器、分支定界）。要最优性证书就选它。
- NP-hard、规模大、要快 → 用启发式 / 元启发式（遗传、禁忌、模拟退火）。接受“好但不保证最优”。
- 需要最坏情况保证、又要多项式时间 → 用近似算法，并盯紧近似比。
- 模型不准 / 数据缺失 / 在线 → 用仿真、学习、人在回路补足模型的不足。

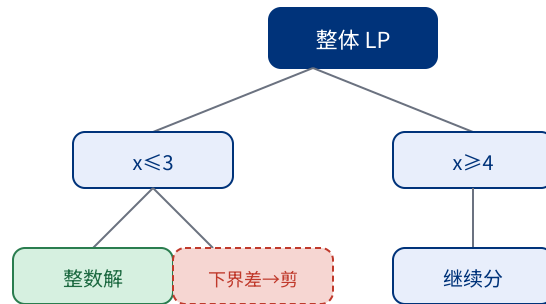
经验法则

先问“这问题多大、我要多确定的答案”；答案往往直接指向方法，而非反过来。

精确法内部：分支定界如何搜索

精确法不是“试遍所有解”，而是**系统地搜索 + 用界剪枝**。以混合整数规划为例，分支定界做三件事：

- **分支**：放松整数约束先解 LP 松弛；若解仍非整数，就按某个变量劈成两枝（ $\leq k$ 与 $\geq k+1$ ）。
- **定界**：每节点用 LP 松弛值给出该子树最优值的下界。
- **剪枝**：若某节点的下界已差于已知最好整数解（上界），整棵子树无需再搜——这正是它比穷举快得多的原因。



红色虚线节点：其下界已劣于已知整数上界，整枝剪掉，不再搜索。

案例：分支定界解 0-1 背包

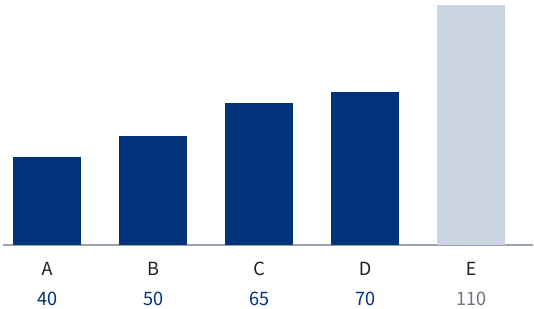
问题描述

救灾卡车容量 15（重量单位），有 5 个候选物资箱 A-E，各有权重与价值。决定装哪些，使总价值最大且不超重。

数学模型

数据：A(2,40) B(3,50) C(4,65) D(5,70) E(9,110)，容量 W=15。

结果展示



最优装载 A,B,C,D：总价值 225，容量 15 用 14。分支定界仅展开 11 个节点（穷举需 32）。

Python 代码实现

```
# Branch-and-bound for the 0-1 knapsack
items = [('A',2,40),('B',3,50),('C',4,65),('D',5,70),('E',9,110)]
W = 15
names=[t[0] for t in items]; w=[t[1] for t in items]; v=[t[2] for t in items]
n = len(items)
best = {'val': -1, 'take': None}
nodes = 0

def bound(i, cw, cv):
    bw, bv = cw, cv
    for j in range(i, n):
        if bw + w[j] <= W:
            bw += w[j]; bv += v[j]
    # greedy upper bound on remaining items
```

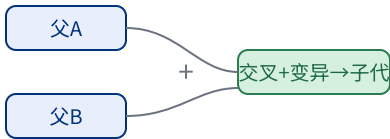
```
    else:  
        bv += v[j] * (W - bw) / w[j]; break  
return bv
```

元启发式一瞥：GA 与模拟退火

当问题 NP-hard、非凸或黑箱，且“好解即可”时，元启发式用**受控的随机性**在解空间里大范围搜索，放弃最优性保证换取速度。

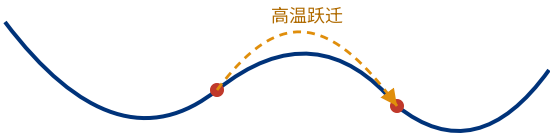
遗传算法 (GA)

维护一群解（种群）：按适应度**选择**优良个体，两两**交叉**产生后代，再**变异**注入多样性。一代代演化，整体趋优。适合组合/调度类问题。



模拟退火 (SA)

像物理退火：以温度 控制“接受变差步”的概率 $-\Delta/T$ 。高温时敢跳出局部谷，随 下降渐稳于低处。温度 schedule 决定探索与利用的权衡。



案例：遗传算法解同一背包

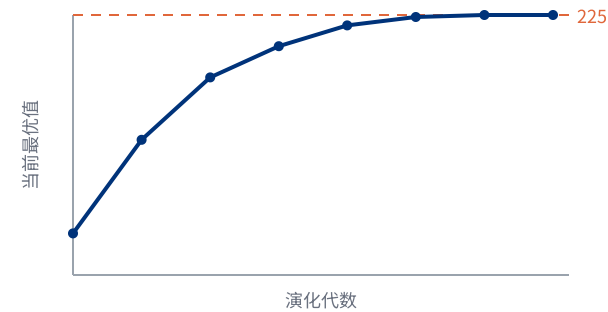
问题描述

同一道题（见上一页）：容量 15，箱 A-E。元启发式不保证最优，但用“受控随机性”在大空间里快搜，规模放大仍可秒级给出近优解。

数学模型

j j j j j j j

结果展示



GA 在种群演化中收敛到 225 (= 精确最优)，但这是“大概率”而非“保证”；规模放大后它仍可在秒级给出近优解。

Python 代码实现

```
# Genetic algorithm for the 0-1 knapsack
import random
items = [('A',2,40),('B',3,50),('C',4,65),('D',5,70),('E',9,110)]
W = 15
w=[t[1] for t in items]; v=[t[2] for t in items]; n=len(items)

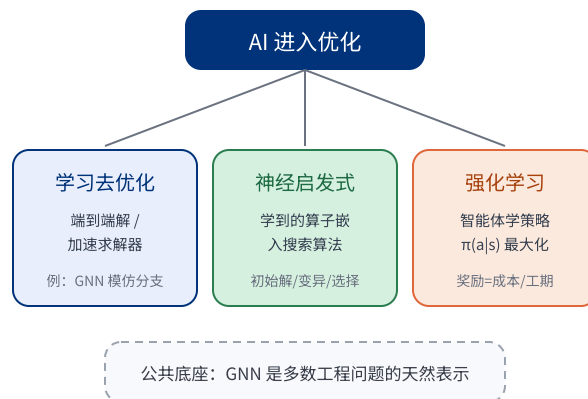
def fitness(g):
    wt = sum(w[k] for k in range(n) if g[k])
    return 0 if wt > W else sum(v[k] for k in range(n) if g[k])

def rnd(): return [random.randint(0,1) for _ in range(n)]

pop = [rnd() for _ in range(30)]
for gen in range(80):
    pop.sort(key=fitness, reverse=True) # rank by fitness
    new = [pop[0][:], pop[1][:]] # keep the two best (elitism)
```

学习式优化方法：AI 如何进入优化

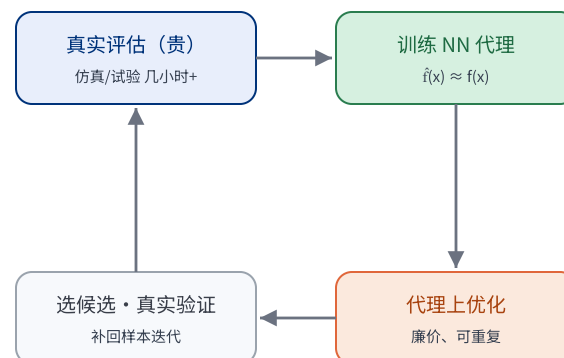
- **学习去优化 (L2O)**：训练模型直接产出解，或加速精确求解器——例如用 GNN 模仿分支定界中的“强分支”。
- **神经启发式**：把学到的算子（初始解、变异、邻域选择）嵌入遗传 / 禁忌搜索，替代手工规则。
- **强化学习 (RL)**：智能体与环境交互，学出决策策略 $\pi(a|s)$ ，最大化累积奖励（如工期、成本）。
- **图神经网络 (GNN)** 是天然底座：网络、排程、分子等大多工程优化问题本就是图。



AI 不取代优化，而是加速与增强它：三种方式互补，都用“速度换一部分最优性保证”。

案例：神经代理模型加速昂贵优化

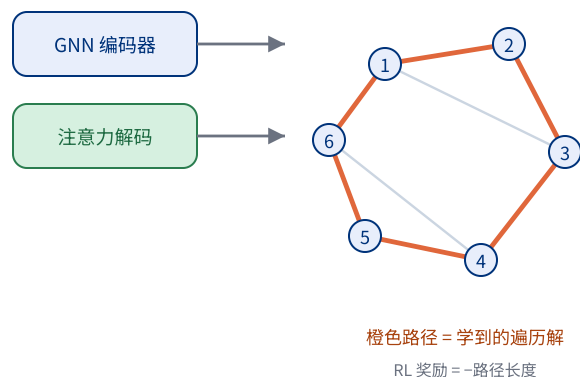
- 真实仿真 / 试验很贵（一次评估几小时到几天），直接优化不现实。
- 先抽样少量点，训练一个神经网络当“廉价代理” $\hat{f}(x) \approx \text{真实 } f(x)$ 。
- 在代理上快速优化，挑出有希望的候选，再用真实仿真校验。
- 把校验结果补回样本，迭代——用极少真实评估逼近好解。



代理把“昂贵的真实评估”替换成“廉价推理”，只在关键候选上才调用真实仿真——典型可省 90%+ 评估。

案例：图神经网络 + 强化学习解组合优化

- 把组合优化实例（如路径 / 排程图）喂给 GNN，得到每个节点的嵌入表示。
- 解码器用**注意力**机制逐个挑选元素，逐步构造解（如一条遍历路径）。
- 用强化学习训练（奖励 = 路径长度 / 工期），**端到端**，无需手工特征。
- 同一模型可泛化到不同规模实例，秒级出解，逼近经典元启发。



左：GNN 编码 + 注意力解码；右：一个六节点路径实例上学到的遍历解（橙色）。

动手选：方法推荐器

回答下面四个问题，看推荐器给出哪种方法。这不是铁律，而是第 4 章清单的可玩版。

问题规模？

小（可精确求解）

大（成千上万变量）

需要最优性证书吗？

是，要可证明最优

否，好解即可

模型是否精确可写？

是，可写清数学模型

否，黑箱/非凸/数据驱动

是否在线 / 实时？

否，离线一次性

是，边运行边决策

请在每行选一项 ↑

理论 vs 工程：并排比较

两类方法家族回答不同的问题，也带来不同的风险。

维度	优化理论	工程实践
已知什么	精确的数学模型	真实、部分建模的系统
目标	算出最优 x^*	交付一个可行的决策
典型保证	最优性 / 界	可行、鲁棒、可接受
主要工具	单纯形、B&B、DP、元启发	仿真、学习、由粗到精、迭代
何时使用	模型忠实且可解	模型不全、数据丰富、变化快

要点

理论提供引擎；工程提供方向盘。

一个码头，两类方法家族

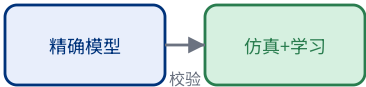
一个集装箱码头说明：单靠任何一类方法家族都不够。

理论

精确的泊位分配与岸桥调度模型（混合整数规划）；只要到港时间、吊速、堆规已知，分支定界即可求得最优。

工程

离散事件仿真在真实靠泊拥堵与设备故障下检验该计划；学习模块对实时扰动做出反应。



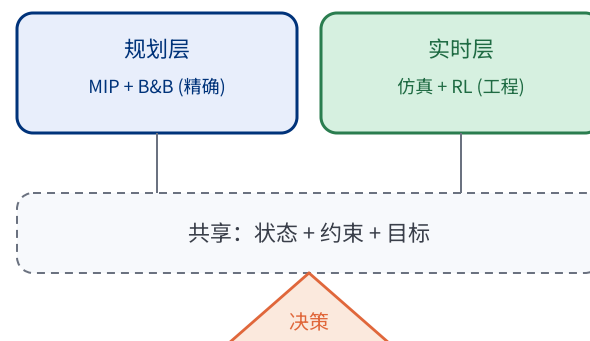
理论给出最优；工程验证并应对现实

闭环

工程闭环才是理论遇见码头的地方。

案例：同一码头，两类方法

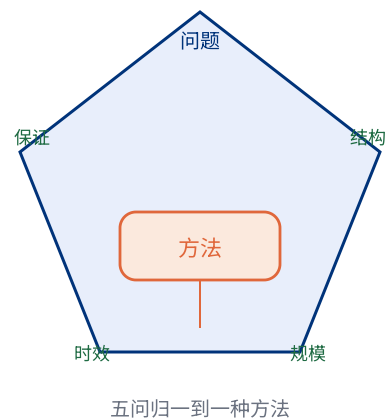
- **规划层（精确）**：MIP + 分支定界求次日泊位/岸桥计划，规模小、要最优。
- **实时层（工程）**：仿真 + 强化学习调度 AGV，规模大、要快与稳。
- 两类方法不是对手，而是**同一系统的两层**。



规划层给出可证明近优的蓝图，实时层在不确定性中执行；二者通过共享状态与约束耦合。

案例：方法推荐器

- 先看**结构**：连续、整数、组合、随机？结构决定哪些方法可用。
- 再看**规模与时效**：能否承受精确求解的代价？
- 最后看**所需保证**：要最优证书，还是只要快而稳？



把“结构 / 规模 / 时效 / 保证”逐层过滤，问题空间被收敛到一族可行方法。

“方法”背后藏着的两种问题

方法一词下藏两个问题：优化理论关心“怎样解这个模型”，工程实践关心“怎样解真实系统”。两者互补，本书分别用“方法”与“应用”两大部分展开。

优化理论

怎样解“模型”？

- 给定干净数学模型
- 求 x^* 使 f 最优
- 提供最优性证明
- 单纯形/B&B/DP

工程实践

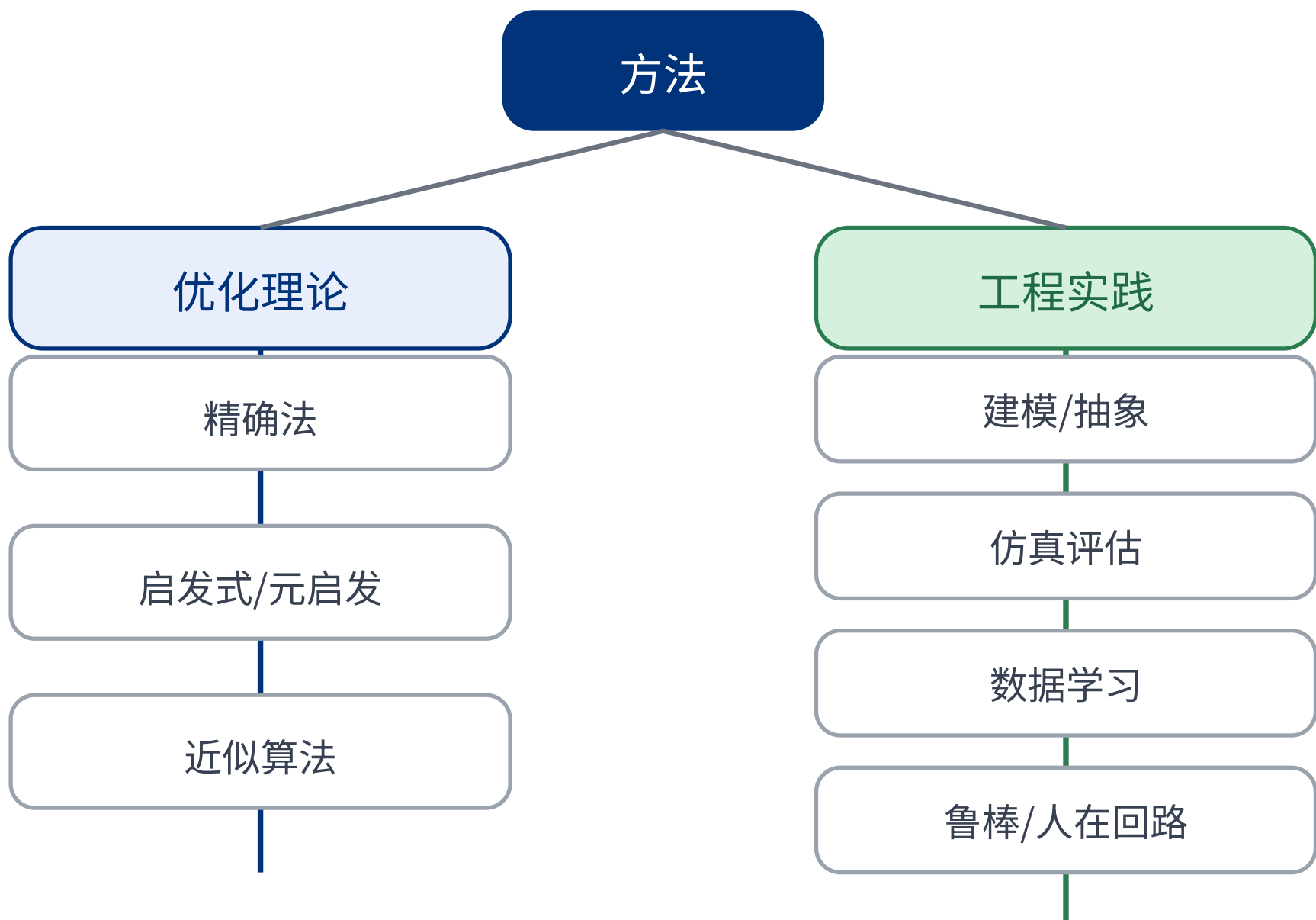
怎样解“真实系统”？

- 模型只捕捉部分现实
- 时间/数据/组织约束
- 交付可部署决策
- 仿真/学习/人在回路

理论告诉你“怎么解一个模型”；工程告诉你“解哪个模型、何时相信它、解不动时怎么办”。

方法谱系：从“保证”到“速度”

方法可按“承诺”划分：优化理论一端要可证明最优（精确法），或退而求有界近似（近似算法），或放弃保证换速度（启发式）；工程实践则用建模、仿真、学习、鲁棒与人在回路去应对真实系统。

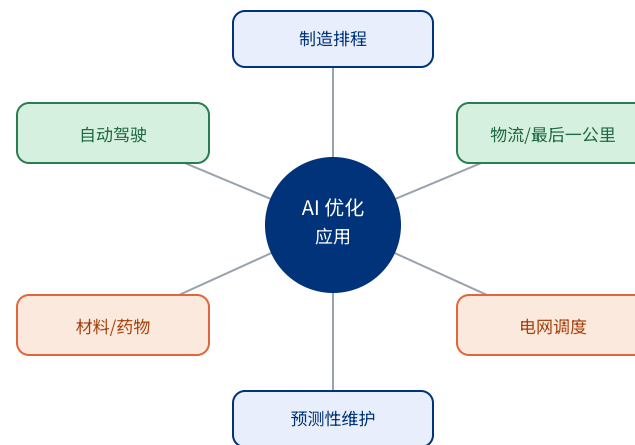


保证最优 $\leftrightarrow$ 放弃保证换速度；模型 $\leftrightarrow$ 真实系统

选择方法本质是“在怎样的保证下、用多快的速度、解决怎样的对象”——这就是第 5 章要讲的权衡。

人工智能算法在现代工程优化中的应用

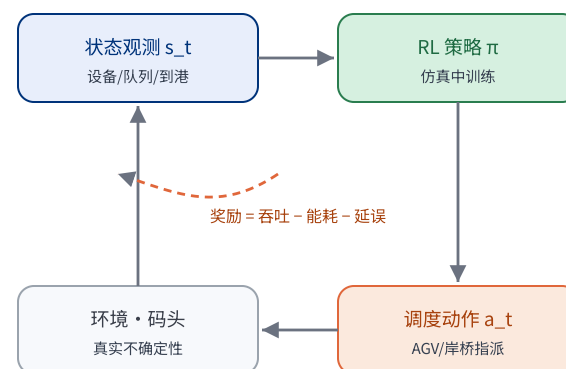
- **制造排程**：RL 控制机器人运动规划与装配顺序。
- **物流与最后一公里**：神经启发式为同城路网秒级生成每日路径。
- **电网调度**：L2O / RL 在风光波动下实时平衡发电、储能与需求。
- **预测性维护**：RL 决定何时检修，最小化停机 + 失效风险。
- **材料 / 药物发现、自动驾驶**：学习式优化加速高维、黑箱搜索。



当问题“大、结构化、时间紧迫”时——正是经典精确法失效之处——学习式方法最有用。

案例：强化学习调度与神经组合优化（码头）

- 码头**实时层**：状态 = 设备位置 / 队列 / 到港；动作 = AGV 调度与岸桥指派；奖励 = 吞吐 - 能耗 - 延误。
- RL 策略先在仿真中大量试错训练，再部署；遇扰动**即时重调度**。
- 神经组合优化（GNN + 注意力）直接为每艘船生成泊位 / 岸桥计划**草案**。
- 这正是“工程闭环”：学习在不确定性中执行并持续改进，而非一次性求得最优。



规划层（精确 MIP）给蓝图，实时层（RL + 神经组合）在不确定性中执行——两层通过共享状态耦合。

CHAPTER 5

权衡

“最优”的方法，是能同时扛住所有实际约束的那个。

方法选择受到约束



每个因素都可能否决一个本属“最优”的方法。最终存活的很少是最强的方法——而是那个契合所有限制的方法。

近似与解的质量

- **问题近似**：简化模型（由粗到精）以使其可解——代价是保真度。
- **最优 vs. 可接受**：20 辆车的车队可在数秒内求得可证明最优；2000 辆车的网络可能跑上一周仍未结束。
- **真正的权衡**：数学确定性 $\leftrightarrow$ 运营截止期。是准时交出一份不错的计划，还是坐等那份证书？

一个权衡场景：车队规模

20 辆车

精确 MILP 可在数秒内求出 *可证明最优* 调度。确定性高，适合承诺交付时间。

2000 辆车

同一模型可能跑上一周仍未结束。此时必须转向 *启发式* 或 *分解*，接受一个“够好”的计划准时交差。

权衡的本质

数学上的“最优”与运营上的“准时”经常冲突。真正被部署的方案，往往是那个在约束下最稳的，而非理论上最强的。

这正是第一篇反复出现的主线：工程问题 $\neq$ 优化问题 $\neq$ 决策问题；方法要服务于真实的约束与人。

规模放大，最优性保证让位于准时交付

20 辆车



精确 MILP · 数秒 ✓ 最优

2000 辆车



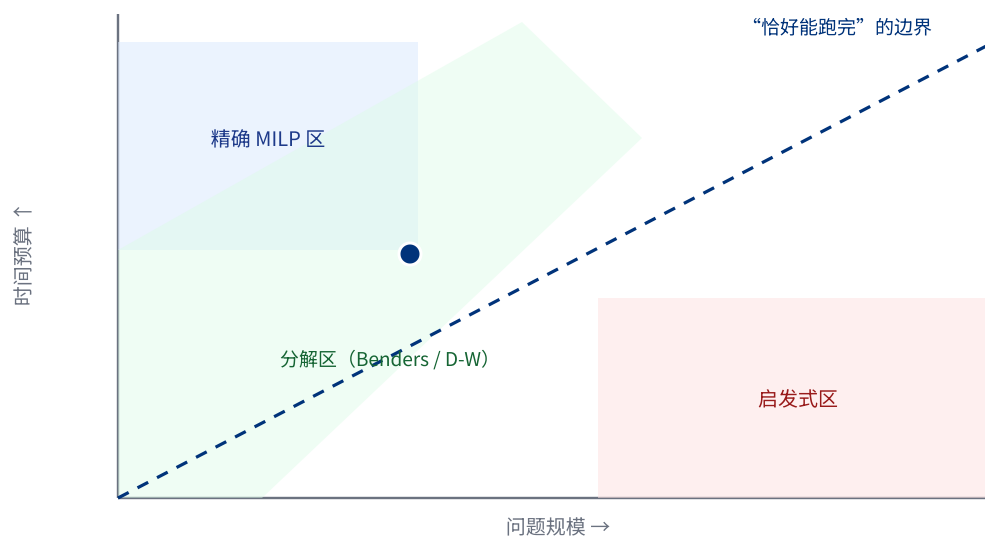
精确 MILP · 跑一周 ✗



元启发/分解 · 数分 ✓ 准时

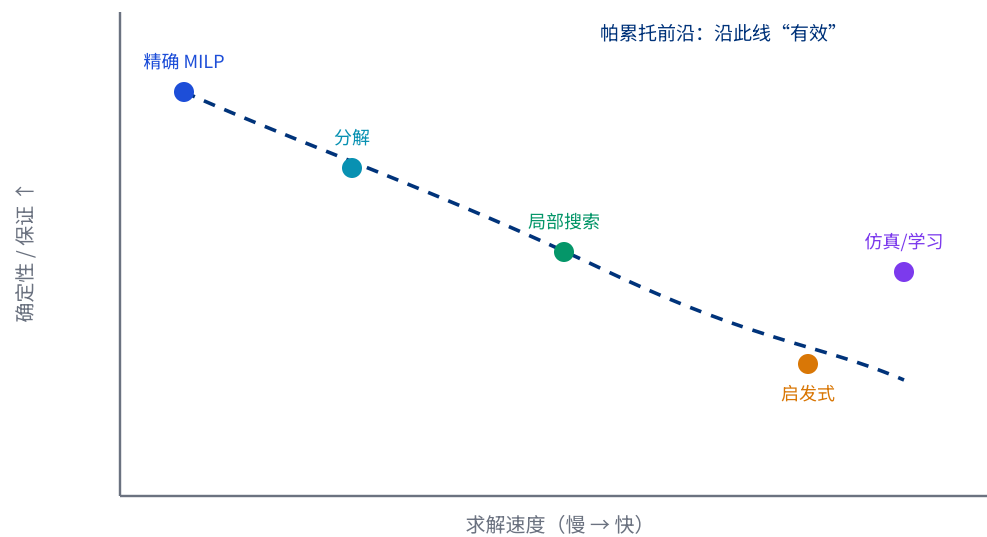
小规模车队用精确法数秒得最优；放大到上千辆车，最优性保证必须让位于“准时交差”的启发式/分解方案。

可行域：约束的交集



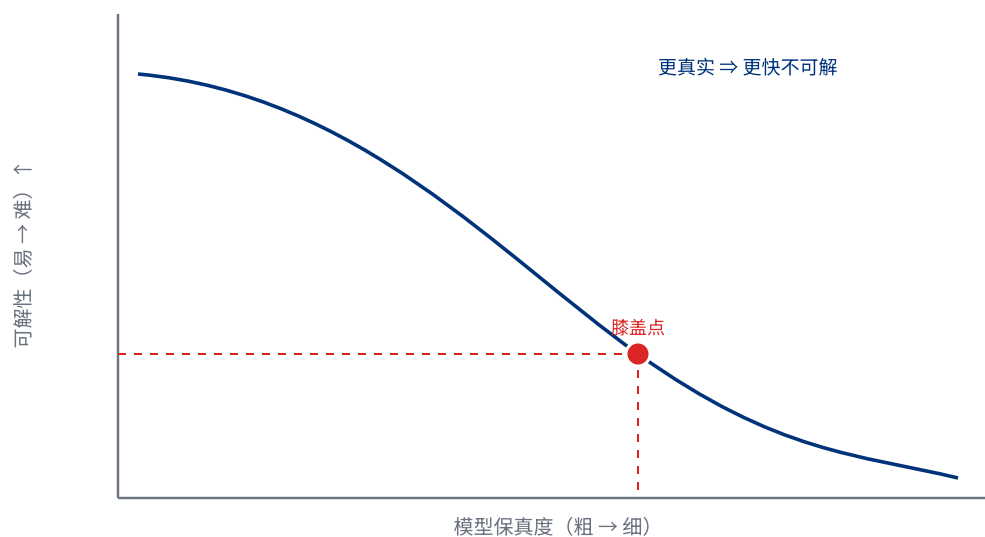
把约束画成一张图：横轴是规模、纵轴是预算，斜线是“恰好能跑完”的边界。你的（规模，预算）落在哪，就决定哪些方法还有入场资格——这正是 s-c5a 所说“约束的交集”。

求解速度：从慢到快



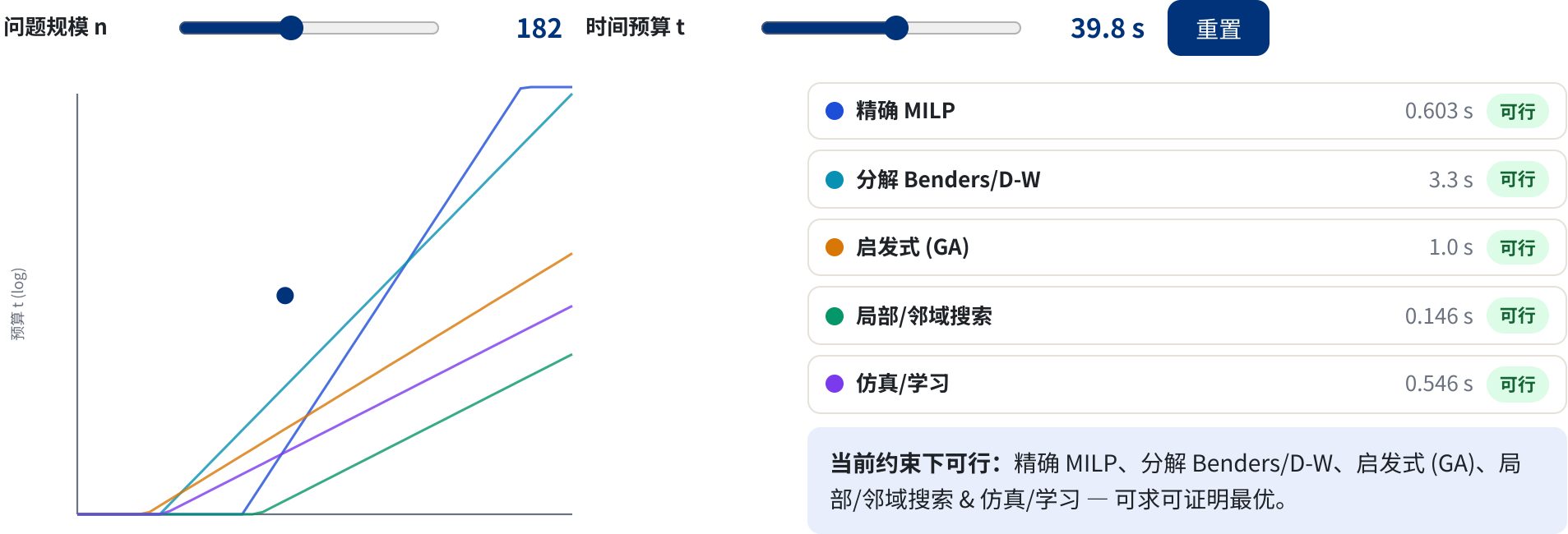
前沿上的每个点都“有效”——你无法在不牺牲一端的前提下改善另一端。取舍不是缺陷，而是边界；选点就是选你愿意放弃什么。

另一组权衡：模型保真度 vs 可解性



更精细的模型更贴近现实，却更快变得不可解。工程的艺术是在“膝盖点”附近停手——够真，且仍解得动。同类取舍还有：安全 vs 成本（更安全往往更贵）、精度 vs 鲁棒（更尖往往更脆）。

动手：拖动约束，看方法是否仍可行



● 精确 MILP	0.603 s	可行
● 分解 Benders/D-W	3.3 s	可行
● 启发式 (GA)	1.0 s	可行
● 局部/邻域搜索	0.146 s	可行
● 仿真/学习	0.546 s	可行

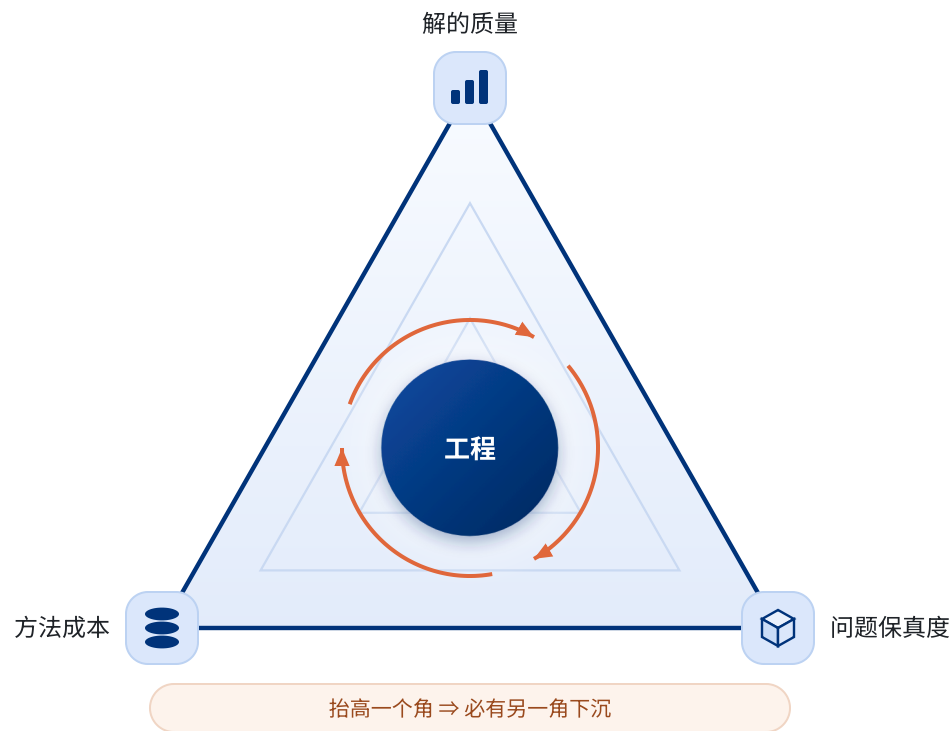
当前约束下可行：精确 MILP、分解 Benders/D-W、启发式 (GA)、局部/邻域搜索 & 仿真/学习 — 可求可证明最优。

核心要点：基础

1. 在套用任何方法之前，必须先**形式化**问题（决策 vs. 优化；可判定 vs. 不可判定）。
2. **分解**以协调代价换取求解代价——切口落在耦合最弱处（分而治之）。
3. **解**有等级之分：可行 → 近优 → 局部 → 全局。正是凸性让“局部 = 全局”。
4. **方法**分为精确 / 启发式 / 近似，各有坦诚的权衡。
5. 工程中，**方法选择**是实际约束的交集，而非对“最优”算法的自由挑选。

工程的权衡三角（不可能三角）

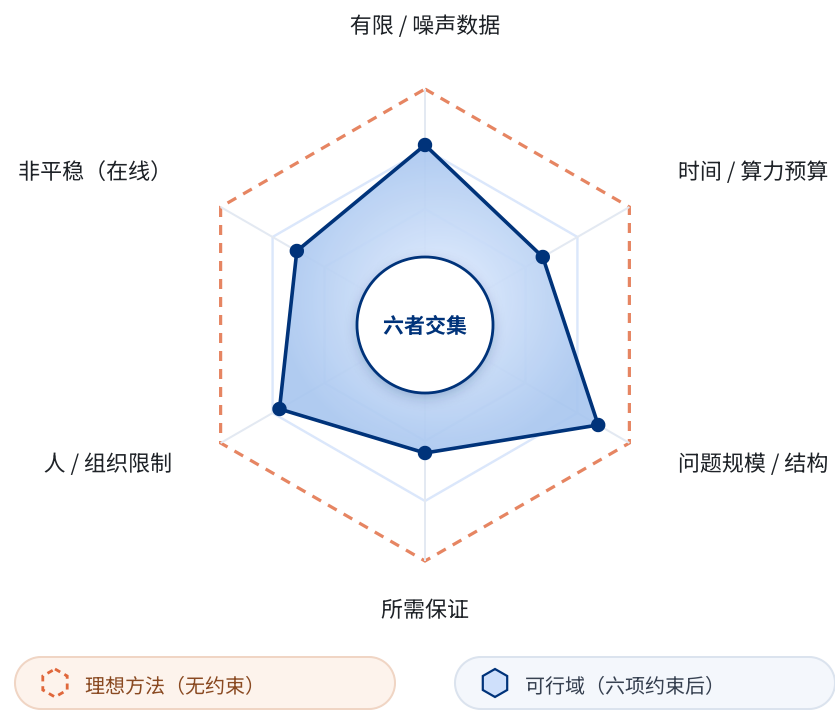
就像彭罗斯（不可能）三角：工程的三个角无法同时最大化——抬高一个角（例如解的质量），必然把另两个角（问题保真度、方法成本）压下去。



工程权衡三角——一个不可能（彭罗斯）三角。你无法同时把三个角都抬高。

约束方法选择的六个因素

纸上最优的方法，可能被任何一个现实因素否决。可行方法，是所有约束的交集。



六个现实因素会淘汰掉“理论上最好”的方法；可行选择，是它们全部施加之后剩下的那部分。

方法选择受约束

纸上最优的方法，可能被任何一个现实因素否决：数据有限/噪声、时间/算力预算、问题规模与结构、所需保证、人/组织限制、非平稳（在线）。可行方法，是这些约束的交集——往往不是最强大的那个，而是扛过所有限制的那个。



任一项约束都能把“理论上最好”的方法排除；工程选的是扛过全部限制的那个。

到达不确定下的泊位计划

数学优化器能求出“恰好贴合每艘船服务时间”的最小成本泊位计划；可一旦船期不确定、延迟几小时，这份精致计划第一天就崩。多花一点成本、主动留出缓冲时间的“次优”计划，反而更能用——用纸面上的次优，换水面上的鲁棒。

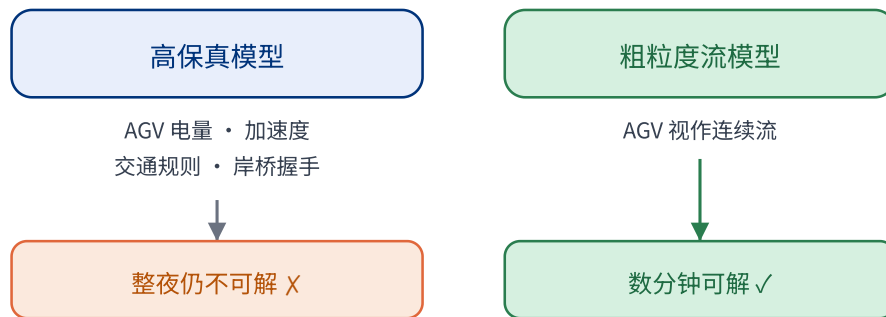


横轴=时间；上排船期背靠背无缓冲，下排每两船之间留出间隔

工程师用纸面上的“次优”，换取真实中的“可用”——质量不只看最优性，更看不确定下的适用性。

案例：洋山码头，保真度 vs 可解性

- 洋山四期若把每辆 AGV 的电量、加速度、交通规则、岸桥握手都建模进去，保真度极高。
- 但含上千步作业的日计划，精确求解整夜也算不出，失去运营价值。
- 折中：先用粗粒度“流模型”（AGV 视作连续流），仅在瓶颈——如两座岸桥争车——处再加细节。



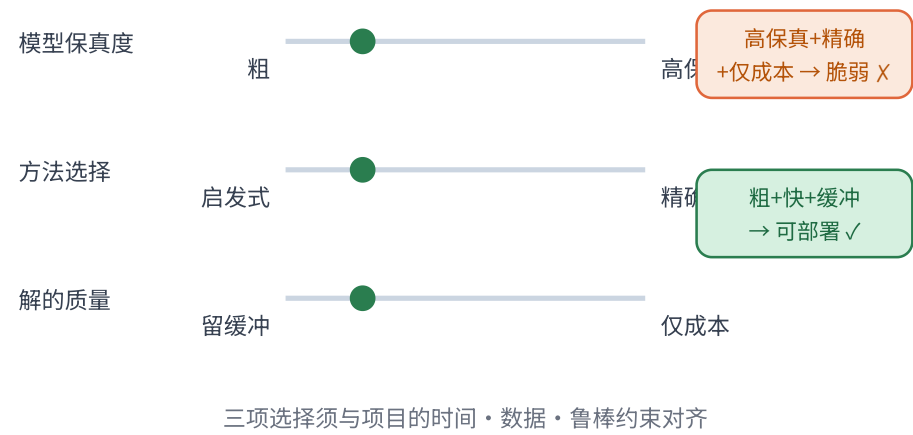
代价：可能漏掉两桥争车拥堵

工程折中：先粗后细，只在瓶颈处加细节

保真度不是越高越好——“够用且可解”才是工程目标；细节只加在真正出问题的地方。

综合：自动化码头设计的三权衡对齐

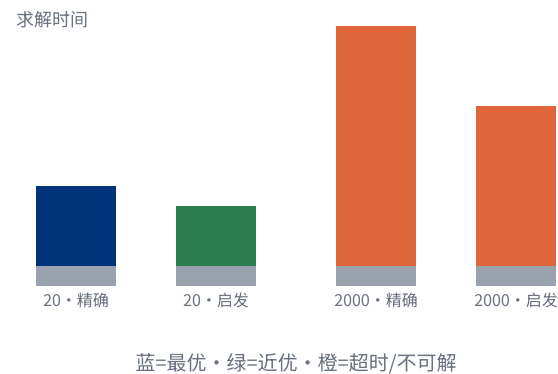
- 设计新自动化码头时，团队要同时定三件事：模型建多细、方法用多强、目标定多严。
- 高保真 + 精确求解 + 仅成本目标，理论漂亮却脆弱：一处扰动就崩。
- 粗模型 + 快速启发式 + 留缓冲，略次优却可部署、可运维、可抗扰。



工程师的本职，是把“模型 · 方法 · 质量”三项选择与项目的真实约束对齐——而非追求单点最强。

案例：车队路径——精确 20 辆 vs 启发式 2000 辆

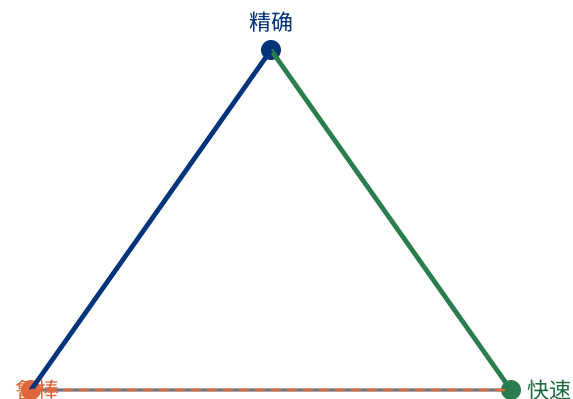
- 精确法：20 辆车可求最优，但求解时间随规模指数增长。
- 启发式：2000 辆车在分钟级得到近优解，放弃最优证书。
- 关键权衡：保真度 vs 可解性。



规模从 20 跳到 2000，精确法迅速超时；启发式以放弃证书换取可解性。

案例：不可能三角——精确·快速·鲁棒

- 三条边分别代表**最优性**、**速度**、**鲁棒性**。
- 任何方法最多牢牢抓住**其中两条**，第三条必然松动。
- 工程的艺术是**按场景选哪两条**。



实线=可同时拥有 · 虚线=必须放弃

精确与快速可兼得（蓝绿边），精确与鲁棒可兼得（蓝橙边）；但快速与鲁棒同享时往往牺牲精确（虚线）。

第二部分

工程优化

从“问题是什么”走向“如何把它解出来”：求解流程、由粗到精、以及为何有时我们要的是一堆好解而非一个最优解。

本篇逻辑框架

本篇从“问题是什么”走向“如何把它解出来”：先建立全局地图，再看求解流程、由粗到精的渐进策略，以及为何有时我们要的是一堆好解而非一个最优解。

本篇 4 章 · 按阅读顺序



逻辑主线：总览定位 → 求解流程 → 由粗到精 → 多解分析，从认知到落地的求解递进。

主线：从概览到求解流程，到由粗到精，到多解——由理解走向执行的递进。

逻辑主线：总览定位 → 求解流程 → 由粗到精 → 多解分析，从认知到落地的求解递进。

CHAPTER 6

引言

在动手求解之前，先建立全局地图：优化问题从哪里来、分成哪几类、本书如何组织。

优化在工程中的位置

优化不凭空出现：它处在“工程问题 → 数学模型 → 求解 → 决策”这条链的中段。

- 上游：模糊的工程诉求（更省、更快、更稳）必须被转成可量化的目标与约束。
- 中游：把模型交给求解器或算法，得到一组设计变量。
- 下游：解要能落地——可被制造、被调度、被信任。

一句话：优化是工程的“计算器”，但不是工程的全部。它只回答“给定模型，怎样最好”，不替你定义“好”是什么。

优化的分类总览

按不同维度切分，同一问题会落入不同格子；选方法前先定位它属于哪一类。

- 连续 vs 离散：变量是实数还是整数/组合？
- 确定 vs 随机/鲁棒：数据已知还是带不确定性？
- 单目标 vs 多目标：一个目标还是一组需要权衡的目标？
- 静态 vs 动态/多阶段：一次决策还是随时间递推？
- 合作 vs 竞争：各方同向优化还是相互博弈？

标准形：
$$\begin{aligned} \min \quad & c^T x \\ \text{s.t.} \quad & Ax = b, \quad x \geq 0 \end{aligned}$$
。这条式子在后面的每一章都会反复出现。

本书的叙事主线

第二到第五篇沿着一条主线展开：先讲“怎么解”，再讲“哪些问题长什么样”，然后“有哪些方法武器库”，最后“落到真实应用”。

- 第二篇 工程优化：求解流程与基本策略。
- 第三篇 工程优化问题：确定性、鲁棒、双层、多目标、多阶段、博弈。
- 第四篇 工程优化方法：经典方法、仿真驱动、学习驱动。
- 第五篇 应用：以集装箱码头自动化为贯穿案例。

一个贯穿全书的例子

自动化集装箱码头的“岸桥—堆场—水平运输”协同调度，几乎覆盖了本书所有主题：离散、强耦合、带不确定、多目标、且要落地。

我们不会一次讲完它；每一章都只取它身上与本章相关的一小片，最后拼成完整图景。

关键术语与记号

先统一一套记号，后面不再重复解释。

- x : 决策变量 (可以是向量)。
- z : 要最小化/最大化的目标。
- a_i, b_i : 不等式与等式约束。
- X : 可行域 (所有满足约束的 x 的集合)。
- x^* : 最优解 (全局)。

工程优化：定义与"现象 → 模型"的翻译

一门学科要先给自己要解的问题命名。这里先给出工程优化的定义，再给出它最关键的一步：把一个现场现象翻译成一个可解的数学模型。

工程优化

定义：一种系统化方法：在给定约束下设计系统、部件或流程，以获得尽可能好的性能；它融合数学建模、计算算法与工程知识。

$$\min_{x \in \mathcal{F}} f(x) \quad \text{s.t.} \quad g_j(x) \leq 0, h_i(x) = 0$$

适用范畴：工业工程、机械、土木、航空航天、化工等一切需要在多个可行方案之间做取舍的工程领域。

特点：两句话概括其立场：**优化从不从抽象开始**，它始于某个"想变得更好"的现象；也**不止于算出一个数**，而是要落地成可执行的决策。

现象 → 模型（问题发现与定义）

定义：把工业场景中实际的目标、约束与可控变量，系统地映射为一个数学优化框架。第一步不是"求解"，而是判断这一团混乱里哪一部分值得建模、哪一部分只是噪声。

$$\text{phenomenon} \mapsto (\underbrace{x \square \square \square}_{\text{决策}}, \underbrace{f \square \square \square}_{\text{目标}}, \underbrace{g, h \square \square \square}_{\text{约束}})$$

适用范畴：项目启动阶段；集装箱堆场、产线、电网等现场问题进入建模流程的第一步。

特点：这一步比看上去难：现场充满利益相关方、隐性规则与历史惯例。**先问"这个模型会改变谁的决策、那个人真正能控制什么"**，答不上来就说明现象还没被理解。

分类轴 I：数据与目标的个数

工程优化问题沿着若干条相互独立的"轴"来描述。前两条是：**数据是否确定**，以及**同时权衡几个目标**。

确定性优化 vs 随机优化

定义：确定性：所有参数已知且固定（如在已知载荷下优化梁的设计），最优解是一个可复现的决策。**随机：**部分参数不确定——需求、风况、到达时间——模型要寻求在多种实现下都"还不错"的决策，由此引出鲁棒或带补偿（recourse）的建模。

$$\min_{x \in \mathcal{X}} f(x) \text{ vs. } \min_{x \in \mathcal{X}} E_{\omega}[f(x, \omega)]$$

适用范畴：确定性：参数可精确标定的设计问题。随机：需求波动、随机到达、传感器噪声显著的运营问题。

特点：确定性模型给出一个可复现的答案；随机模型给出的是**在多种情景下都站得住**的答案，代价是要先刻画不确定集或分布。

单目标优化 vs 多目标优化

定义：单目标：只优化一个标量准则（如最小化重量），答案是唯一、无需权衡的最优点。**多目标：**同时平衡若干相互冲突的准则（如既轻又耐久）。此时不存在"全面最优"的单点，解是一组互不可比的 **Pareto 前沿**。

$$\min_{x \in \mathcal{X}} (f_1(x), f_2(x), \dots, f_m(x)) \rightsquigarrow P \text{ (Pareto 集)}$$

适用范畴：单目标：成败可用一个指标判定。多目标：成本 / 可靠性、速度 / 能耗这类必须显式权衡的情形。

特点：答案形态变了：从"一个数"变成"一条曲线"。决策工作从求解转移到**在曲线上选点**。

分类轴 II：变量类型与模型形态

后两条轴决定"该伸手拿哪一类算法"：**决策变量是连续还是离散**，以及**目标与约束是线性还是非线性**。

连续优化 vs 离散优化

定义：连续：变量可在区间内取任意值（温度、速度），可行域是一块连续区域，梯度与凸性这类工具可用。**离散**：变量只能取有限集合中的值（选材料、选规格），这就是组合优化，主导着物流与制造。

$$x \in \mathbb{R}^n \quad \text{vs.} \quad x \in \{0, 1\}^n \text{ 或 } x \in \mathbb{Z}^n$$

适用范畴：连续：配料、轨迹、控制量。离散：路径、排序、指派、装箱、选址。

特点：离散性会把"多项式时间可解"的问题推成 **NP-hard**——这是工程优化里最贵的一次跃迁。LP 松弛是否自动给出整数解，往往取决于约束矩阵是否全单模。

线性优化 vs 非线性优化

定义：线性：目标与约束都是线性的（如资源配置的线性规划），存在高效的精确求解器。**非线性**：出现非线性关系（如气动阻力下的燃油效率）。其中**凸**的非线性仍然好解，而**非凸**会引入大量局部最优。

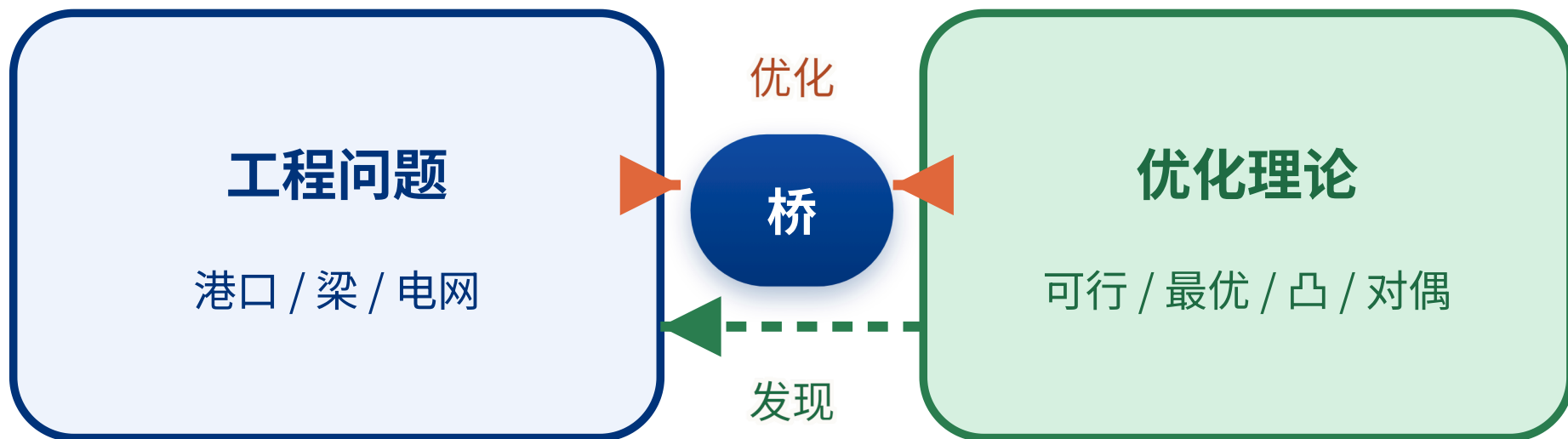
$$\min c^T x \text{ s.t. } Ax \geq b \quad \text{vs.} \quad \min f(x) \text{ s.t. } g_j(x) \leq 0 \text{ (} f, g_j \text{ 非线性)}$$

适用范畴：线性：配比、运输、产能分配。非线性：结构力学、流体、含曲率或乘积项的物理模型。

特点：**凸性是分水岭**：凸问题里局部最优即全局最优，且对偶强大；非凸问题则必须接受"局部最优 + 多起点 / 元启发式"的现实。

工程优化的两大支柱

工程优化建立在两个不可分割的支柱之上：我们先在工程情境中发现一个优化问题，再用优化理论去求解它——二者缺一不可。



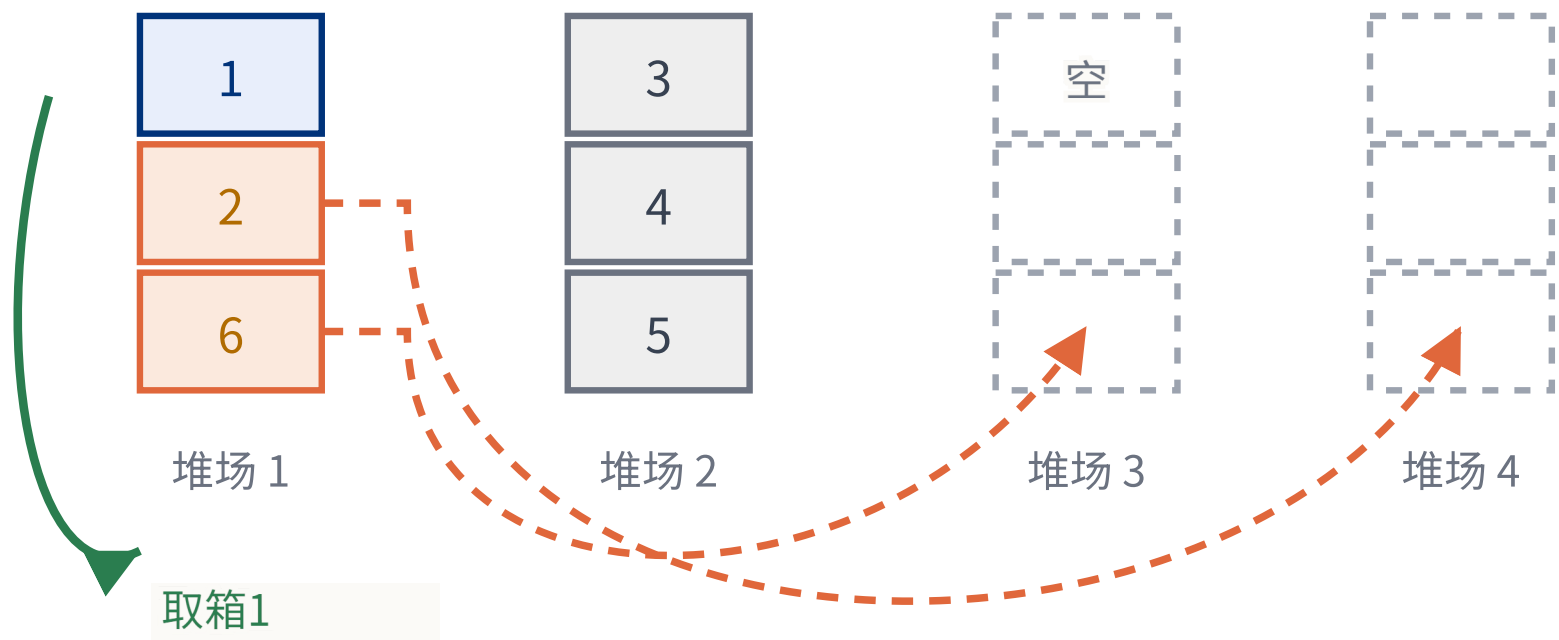
两根支柱都不孤立

没有工程问题就无事可解；没有理论就只是一句直觉。

这门学科是具体工程情境与优化数学语言之间的双向桥梁。

从现象到模型：集装箱翻箱问题

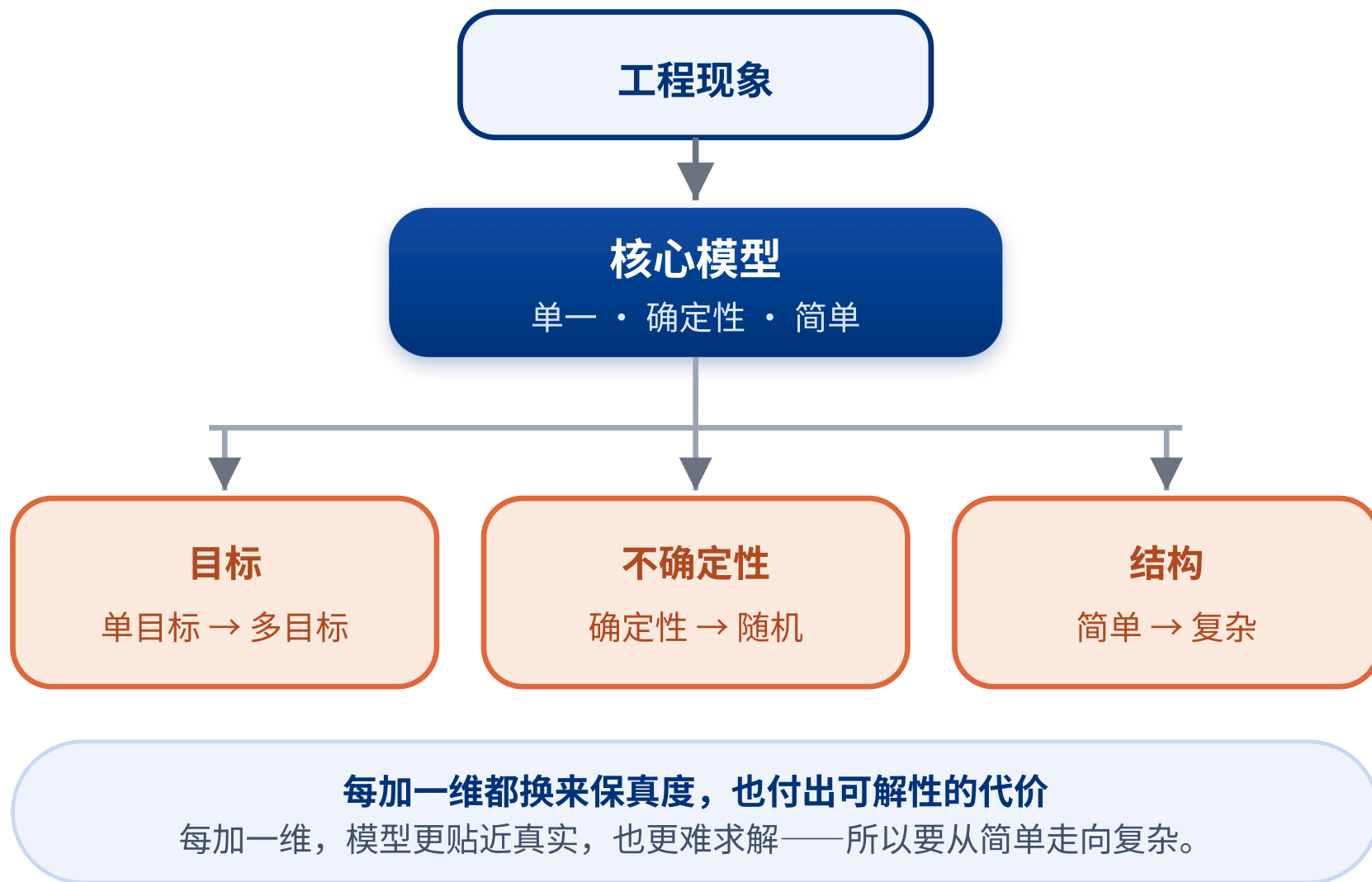
从现象到可解问题是一个渐进的过程。集装箱翻箱问题（CRP）是本书的贯穿案例：1 号箱压在堆场 1 的最底部，被 2 号和 6 号箱挡住。



要取出 1 号箱，起重机必须先把 2 号和 6 号箱翻到空堆场（红色），再把 1 号箱吊出（绿色）。把目标、约束与决策形式化，就是从现象到模型的一步。

先简后繁：拓宽三个维度

我们不会一次性建模所有东西，而是分层搭建问题，沿着三条轴逐步扩展，直到它接近真实系统。



从现象出发建立基础模型，再扩展三条维度——目标、不确定性、结构——直到模型与现实相仿。

沿四个轴分类问题

从数学曲面视角看，一个问题可同时沿着四条相互独立的轴分类；真实问题往往同时落在多条轴上。

不确定性

确定性（数据已知）vs 随机（需求、风、到港）。

变量类型

连续（任意值）vs 离散（有限集；组合）。

目标数

单目标（一个标量）vs 多目标（帕累托前沿）。

模型形态

线性（精确求解器高效）vs 非线性（凸可解；非凸多局部最优）。

注意

同一结构出现在每个工程领域——航空、土木、机械、电气、化工、环境、工业。只有物理不同。

三类方法家族与如何选择

方法可按随机性所扮演的角色分组：确定性、随机性，以及二者混合。

确定性

利用结构（导数、凸性）。基于梯度（最速下降、牛顿）与无梯度（单纯形、分支定界）。可复现，适用时给出最优证明。

随机性

用随机性（重启动、采样、种群）逃离局部最优——遗传、模拟退火、粒子群。给出强近优解；每次运行结果不同。

混合

二者结合——如确定性局部搜索精修随机探索者；数学启发式；基于学习的优化器。

如何选择

回答五个问题：问题类型、维数、光滑性、算力预算、全局 vs 局部目标。

问题全景：六类工程优化问题

自顶向下看，整个领域围绕工程师真正做出的决策来组织。六大主要类别如下：

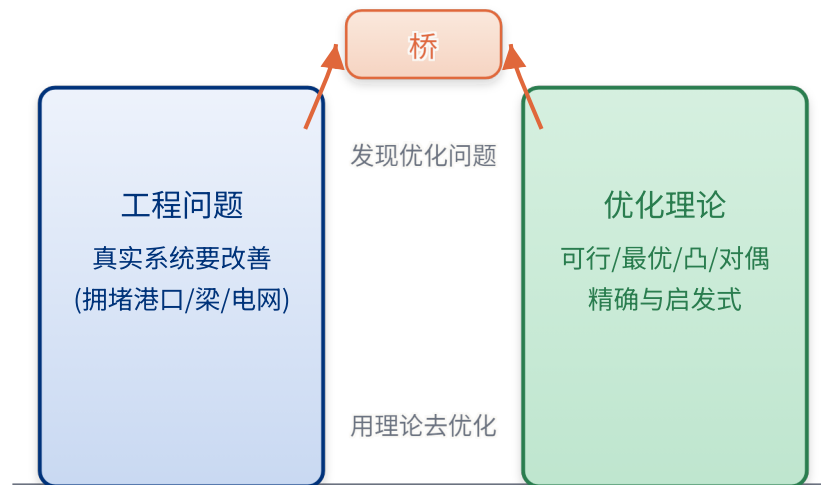
Class	Representative instances	Chapter
Combinatorial & discrete	crane/truck scheduling, routing, assignment, packing	Ch.10 (new)
Robust	design under tolerances, worst-case sizing	Ch.11
Multi-objective	weight vs strength, cost vs emission	Ch.13
Multi-stage (stochastic)	capacity expansion, recourse	Ch.15
Bilevel	toll/price setting, leader–follower design	Ch.12
Competitive (game)	Cournot/Bertrand, network sharing	Ch.16

洞见

工程优化的艺术不在于解一个方程，而在于识别系统中存在哪些类，并据此分解。

双支柱：工程问题 $\rightleftharpoons$ 优化理论

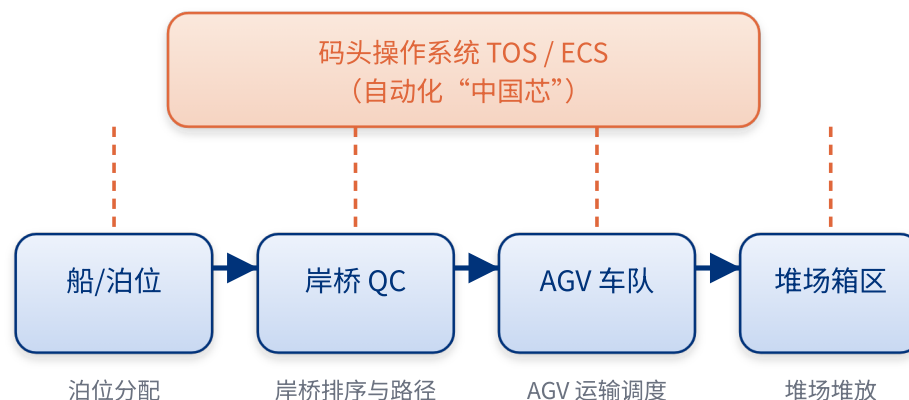
- 工程优化立足于两根不可分离的支柱。
- **工程问题**：来自物理或运营世界的具体挑战——拥堵的港口、受载的梁、风况不确定的电网。
- **优化理论**：把模糊抱负变成可解模型的语言与算法——可行、最优、凸性、对偶、精确与启发式方法。
- 学科因此是两者间的双向桥：在工程中**发现**优化问题，再用理论去**优化**它。



没有工程问题便无物可解；没有理论，问题只是直觉而非已解的决策。

例子：洋山四期自动化集装箱码头

- 洋山四期（2017 年试运营）是全球最大单体全自动化集装箱码头。
- 无人运作：自动岸桥、轨道吊、无人驾驶 AGV，由 TOS/ECS（“中国芯”）统一编排。
- **每一次自动移动本身都是优化问题**：泊位分配、岸桥排序、AGV 路径、堆场堆放。
- 这正是“组合调度/路径/指派”问题——也是本书反复使用的运行示例。

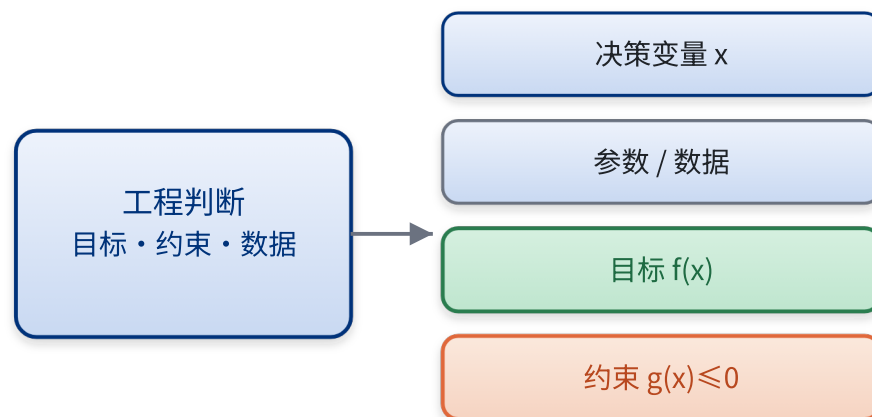


海侧 → 陆侧：每一环都是组合优化问题

码头是工程优化的缩影：链上每一环都对应书中一类组合优化问题。

案例：把一次排产会议变成优化模型

- 工程师说的是“要按期、别超预算、设备别冲突”——这是**工程判断**，还不是模型。
- 抽象化把它翻译成四类要素：**决策变量**、**参数/数据**、**目标**、**约束**。
- 同一句“别超预算”，可写成硬约束，也可写成带罚软约束——抽象方式决定后续能用的工具。



建模不是“把问题写下来”，而是选择用哪套要素来表达它——这一选择本身就有工程后果。

CHAPTER 7

求解流程

建模、求解、验证——三件事循环往复，缺一不可。

三段式流程

绝大多数工程优化都绕不开这三步，且会反复循环：

- 建模：把工程诉求写成 $\min_{\mathbf{x}} f(\mathbf{x})$ 这样的数学式子。
- 求解：把模型交给算法/求解器，迭代逼近满足约束的较优解。
- 验证：检验解是否真有意义——是否满足隐含约束、是否对数据敏感。

“建模—求解—验证”不是一次性的线性流，而是一个回环：验证发现的漏洞会逼你回到建模。

核心阶段：从现象到第一个可解的模型

求解过程的第一步不是求解，而是把一堆抱怨收敛成一个可解的核心问题。这一步做对了，后面的方法才有意义。

现象识别

定义：工程优化项目通常始于一句抱怨、一个瓶颈或一个指标：箱子堆得太高、货车等太久、产线交不了期。现象识别就是判断哪一部分值得建模、哪一部分是噪声。

选取视角

现象 → {可控变量, 目标, 硬约束}

适用范畴：项目立项与需求调研阶段；多利益相关方（船公司、司机、堆场计划员、海关）意见不一致时尤其关键。

特点：和科学研究一样，提出一个干净的问题往往比解出它更值钱。诺贝尔奖反复颁给"先把现象铸成基本问题"的人，工程优化也不例外。

核心阶段（core phase）

定义：第一个正式模型。它的目标不是完美，而是可解且有信息量：用最少的决策、约束与目标抓住主导的权衡，其余一律先放下。

core = argmin # {决策} + # {约束} + # {目标} s.t. 仍能暴露主导权衡

适用范畴：任何新问题的第一轮建模；例如 CRP 先只最小化翻箱次数，暂不考虑场桥行走时间、司机疲劳与船舶到港不确定性。

特点：简化是特性而非缺陷：它先告诉你瓶颈到底在布局还是别处。这也是第 8 章"由粗到精"的同一套态度。

从方法到解：基线、改进，与"什么时候算够好"

核心模型写下来之后，先要一个**能跑出结果的方法**，再谈改进；而"解够不够好"从来不是精度问题，而是**决策者是否比现状更信它**。

基线方法与改进方法

定义：基线方法要的不是文献最佳，而是"足够好"：能在合理时间内产出第一个可信解，用它来检验模型行为是否符合预期、目标是否真反映管理者意图。**改进方法**是在基线跑通之后的加强：更紧的建模、有效不等式、分解、元启发式或学习式初始化。

$$\text{baseline: } x_0 \rightarrow \text{improved: } f(x_{k+1}) \leq f(x_k), t_{k+1} \leq t_k$$

适用范畴：基线：新问题首轮、需要快速判断模型是否值得投入。改进：基线已通、需要在质量与耗时之间换取更好平衡。

特点：改进的目的**不是优雅，而是更好的"质量 / 时间"权衡**。精确方法给最优性证书，近似方法更快地给好解——二者常常要并用。

解的充分性 (good enough)

定义：一个解在被**解释**之前还不是终点。要问：它是否尊重那些太 messy 而没能写进模型的隐性约束？输入数据轻微漂移后是否仍然合理？改进幅度是否值得实施成本？

$$\text{deploy} \Leftrightarrow \text{trust}(x) > \text{trust}(\text{status quo})$$

适用范畴：每一次交付前的评审；尤其是模型忽略的因素（实施难度、人员负荷、维护性）可能主导成败时。

特点：理论上的最优性 $\neq$ 工程上的可接受性。这也解释了为什么多个（近）最优解往往比一张最优性证书对决策者更有用。

反馈回路与工程引擎

解会反过来告诉我们下一个模型该含什么——这个回路很优雅，但跑起来很贵。让它变便宜的，是一层**工程引擎**。

反馈回路

定义：一个解可能暴露出目标函数写错了、某条约束过紧，或某个被忽略的因素其实才是主导。此时建模者回到更早的环节——有时是方法，有时是问题定义，偶尔一路退回到现象本身。

$$\text{模型}_n \rightarrow \text{解}_n \rightarrow \text{模型}_{n+1} \quad (\text{复用, 而非推倒重来})$$

适用范畴：每一轮扩展之后；鲁棒优化取阶段 n 的确定最优作为标称情景，双层规划则把阶段 n 显式当作投资（上层）去挖掘它在阶段 $n+1$ 的价值。

特点：复用而不是重建：阶段 n 的解作为下一轮的标称点、热启动与验证脚手架，研发成本才不会随轮数爆炸。这个回路不是失败的标志，而是工程认识前进的正常机制。

工程引擎

定义：一层薄薄的基础设施：把功能模块与运行环境（车载或离线）隔离，使模块本身在不同场景下永不改变，从而保证每次迭代可复现、且重复执行时**足够便宜**。

$$\begin{array}{ccccc} & \text{calls} & & \text{isolates} & \\ \text{modules} & \longrightarrow & \text{engine} & \longrightarrow & \text{environment} \end{array} \quad \text{同一输入} \Rightarrow \text{同一输出}$$

适用范畴：需要反复求解—仿真—再求解的场景；典型如美团无人配送的仿真系统（同样的模块在同一输入下，因机器负载不同竟会给出不同输出）。

特点：它解决两件事：**行为一致性**（固定调度器与输入后执行顺序可复现，否则无法分辨变化来自模型还是运行时）与**重复执行的效率**（Node 声明依赖、Module 负责计算，上游结果缓存成数据服务供下游复用）。

解的表示

同一个解可以有多种表示；表示方式直接决定算法能否高效搜索。

离散编码：0/1 数组、排列、树——适合组合问题，但邻域定义要小心。

连续编码：实数向量——梯度、牛顿类方法得以施展。

表示即偏置：你选择怎么描述解，就已经替算法假设了“相邻的解长什么样”。

收敛判据

算法何时算“到了”？工程上常用三类判据，常组合使用。

- 目标变化量： $\|f(x_{k+1}) - f(x_k)\|$ 。
- 步长/变量变化量： $\|x_{k+1} - x_k\|$ 。
- 梯度幅值： $\|g_k\|$ （光滑问题）。

注意：“看起来不动了”不等于“到最优了”——可能停在局部谷，也可能只是步长太小。

数值稳定性

漂亮的数学公式落到计算机上会“生病”：舍入、溢出、病态条件都会让结果失真。

- 病态条件：输入微小扰动被目标放大成巨大误差。
- 溢出/下溢：指数、阶乘、概率连乘容易爆量程。
- 累计误差：成千上万次迭代里舍入悄然累积。

对策：缩放变量到相近量级、用对数域、用带容差的比较，而非严格相等。

终止条件与最优性证书

停手容易，证明“这就是最好”难。两类终止要分清：

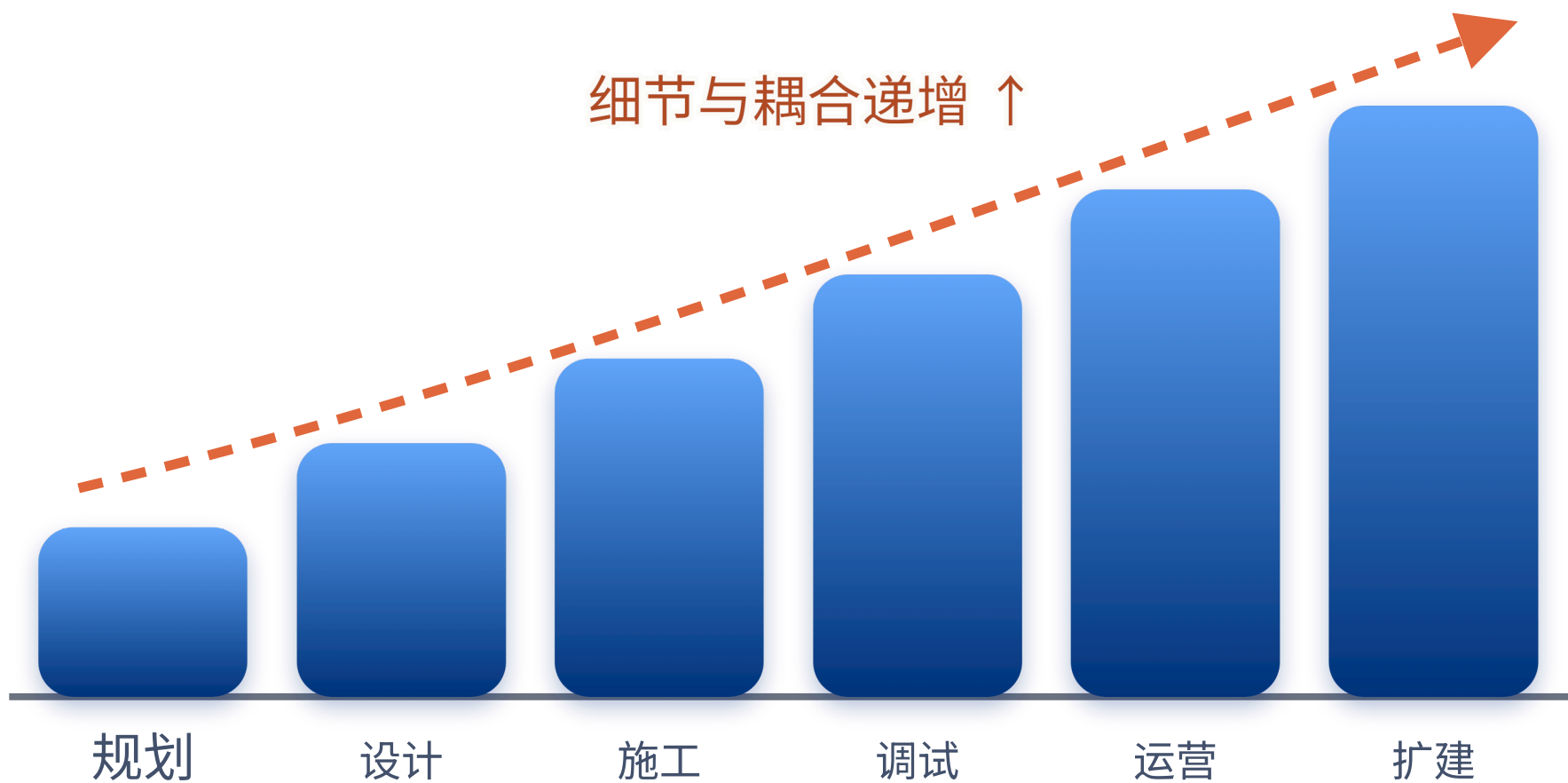
启发式终止：达到迭代上限或收敛判据即停，给的是可行/近优解，无证书。

精确终止：分支定界等收敛时给出上下界差 $<$ 容差，可证明全局最优。

工程里绝大多数时候你拿不到证书；清楚自己停在哪一种，比追求证书本身更重要。

工程是分阶段建成的

真实系统通过多个阶段实现（规划 → 设计 → 建设 → 运行 → 扩展）。优化问题也不例外：应把它当作分阶段建造的过程，而非一次性规格。复杂度越高，保真度越好，但可解性越差。



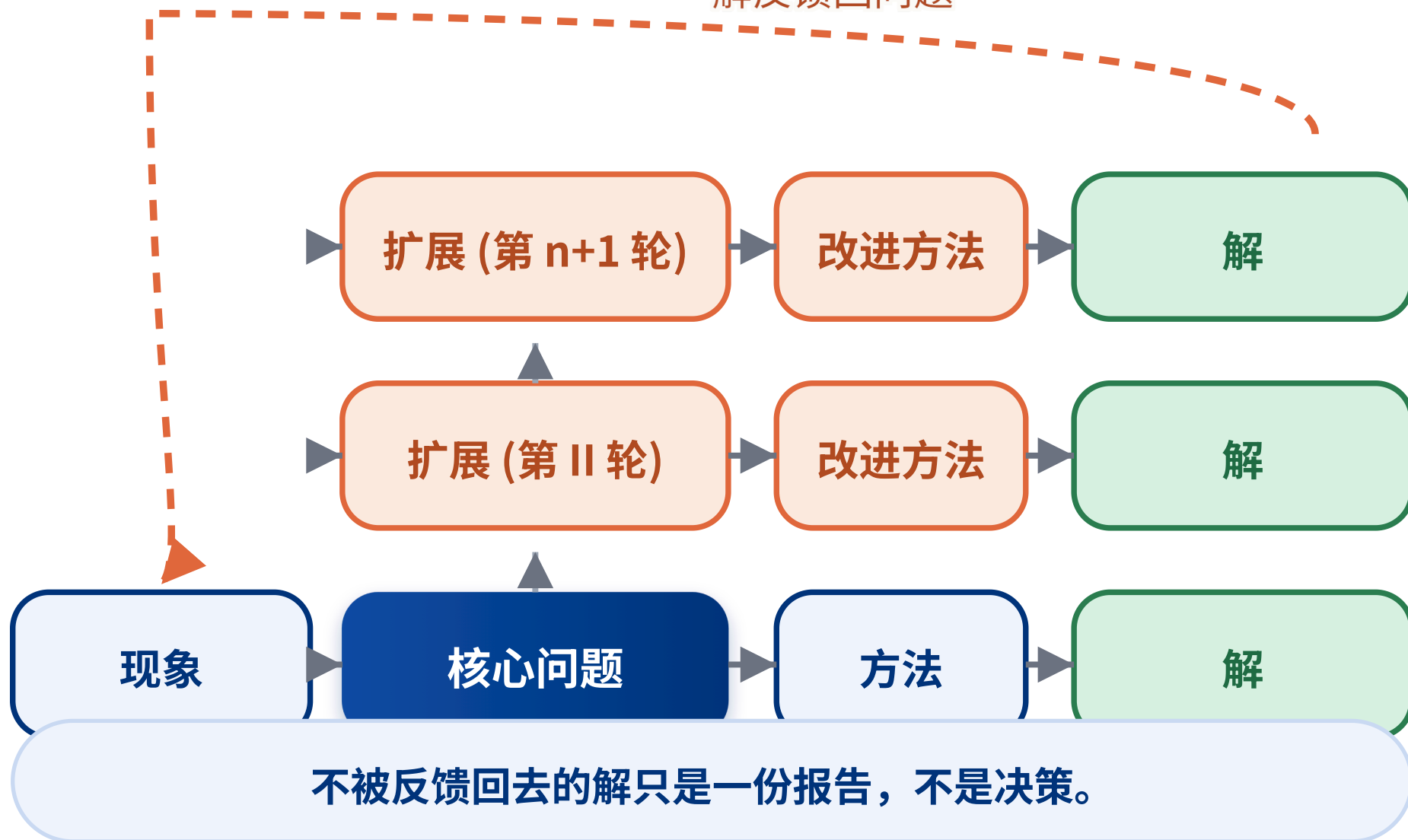
系统分阶段建成，优化模型也照同样的方式逐层长起来。

难度大致随复杂度指数增长。工程实践从核心模型出发，向复杂度上限推进，但不会越过由进度和成本设定的实际边界。

求解流程一览 (EOP)

整个研究是一连串近似，每一次都要用前一次忽略的约束来检验。箭头（反馈）与方框本身同等重要。

解反馈回问题



能反馈回问题陈述的解才是工程决策；否则只是一份报告。

第一阶段：从现象到核心问题

第一阶段是第一个形式化模型。它的目标不是完美，而是可解且能提供信息。

识别现象

判断这团混乱中哪部分值得建模。这个模型会改变谁的决定？那个人能控制什么？

核心阶段

用最少的决策、约束、目标抓住主导权衡。其余一概省去。

例子

翻箱问题的首个模型可以只最小化搬运次数——忽略吊机行走、工人疲劳、到港不确定。这种简化是优点：它告诉我们瓶颈是否在布局本身。

复用

第 n 阶段的解成为第 $n+1$ 阶段的标称点与热启动；只需对新加的维度建模。

模型为何增长：三个常见扩展

核心问题只是一幅草图；只有经过扩展才真正有用。以下三类扩展反复出现：

- **更多目标**——单一成本目标扩展为多目标模型（时间、能耗、排放、风险）。
- **更多不确定性**——确定性数据被场景、分布或不确定集取代（鲁棒/多阶段）。
- **更多结构**——连续松弛变为离散；单阶段变为多阶段；孤立决策变为竞争性博弈。

为何费这劲？

扩展模型也放大了已有工作的回报：鲁棒对应保护第 n 阶段的最优；双层把它的价值挖掘为上层决策。

反馈闭环与迭代模板

弯曲的“解”箭头闭合了循环。一个解可能暴露错误的目标、过紧的约束或被忽视的主导因素——这时我们就要回到前面的某个方框。把它当作检查点来管理：

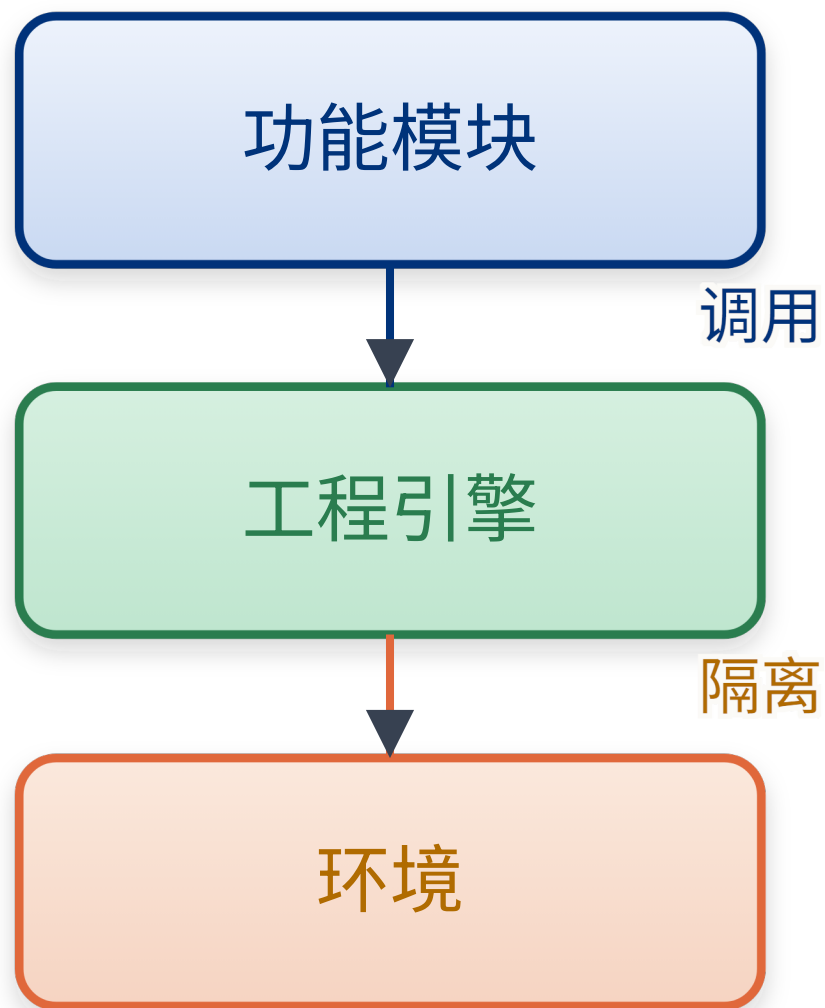
- 1. 范围——决策是什么，什么是可控的。
- 2. 核心模型——暴露主要权衡的最简形式化。
- 3. 基线解——可信方法能否及时给出解？
- 4. 验证——领域专家是否认可？
- 5. 扩展——遗漏哪个因素最能改进决策？
- 6. 再求解——扩展是否改变了方法需求？
- 7. 回到第 4 步——直到足够可用再部署。

洞见

得到好解的最快方式往往是先要一个会出错的小模型——它精确告诉你忘了什么。复用，别重建。

工程引擎让闭环变廉价

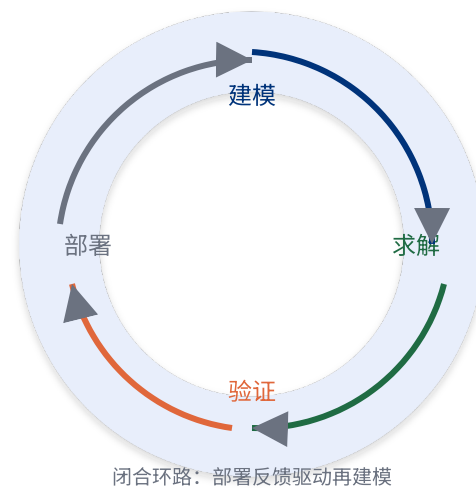
反馈循环在理论上很优雅，实践中却代价高昂。每一次迭代都必须可复现，因此一个薄薄的“工程引擎”基础设施层把功能模块与（不断变化的）环境隔离开来。



通过固定调度器和计算模型，引擎让每次迭代都可复现，从而使反馈循环低成本、可靠地运转。

案例：工程优化流程（EOP）

- 建模 → 求解 → 验证 → 部署，再回到反馈。
- 任何一步失败都回到上一步，而非推倒重来。
- 模型会**随迭代增长**：先核心，后细节。



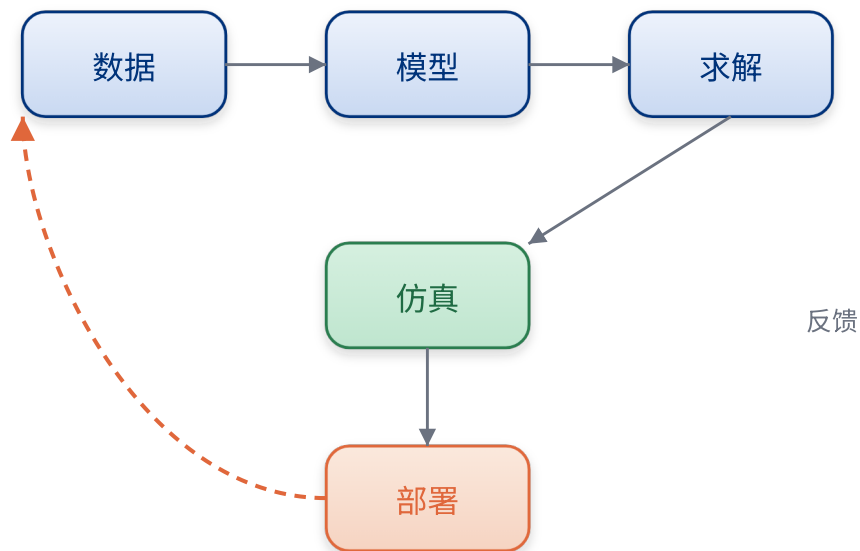
求解不是一条直线，而是一个会自我修正的环路；每一步都可回退。

案例：美团式工程引擎

- 美团配送系统可粗分为**履约系统**、**运营系统**与**主数据平台**：履约负责发单/调度/派单，运营负责经营规划/骑手运营/结算，主数据平台统一组织、商家、用户、运力等核心实体。
- 进入精细化阶段后，美团建设了**仿真平台**（发单回放+骑手行为模拟，离线评估算法）、**算法数据平台**（特征→模型→预测→效果评估闭环）与**LBS 平台**（地图、路径、ETA、热力图），把试错成本降到线下。
- 其本质是一个持续回灌的**工程引擎**：数据—模型—算法—仿真—部署—真实反馈，驱动配送系统在高速增长中持续降低试错成本、提升履约效率。

参考

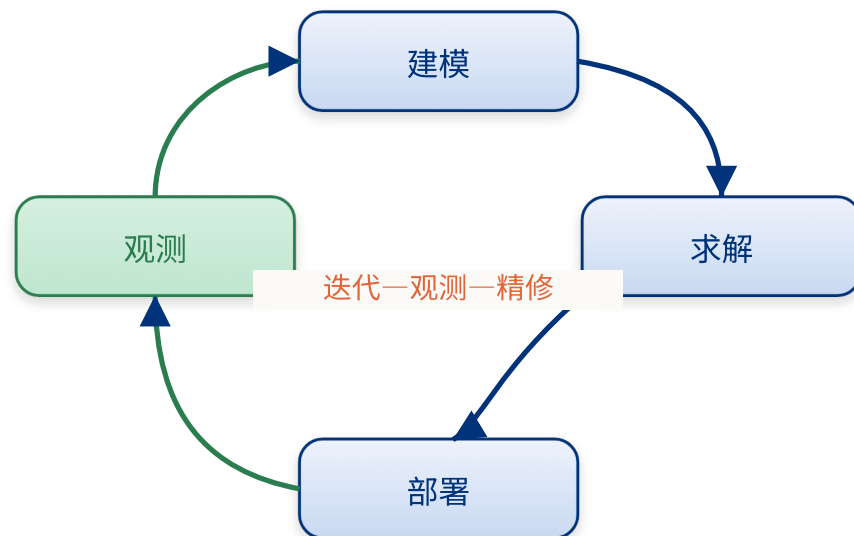
- 美团技术团队. 美团配送系统架构演进实践. 2018.



这不是一次性流水线，而是数据—模型—求解—仿真—部署不断回灌的引擎。

工程优化流程：建模→求解→部署→观测→精修

- 工程优化不是一次性计算，而是一条纪律化的迭代循环。
- **建模**：把现象译成变量、目标、约束。
- **求解**：选方法得到候选解；**部署**：集成进系统并运行。
- **观测→精修**：看真实表现，回到建模端改进——而非直奔求解器。



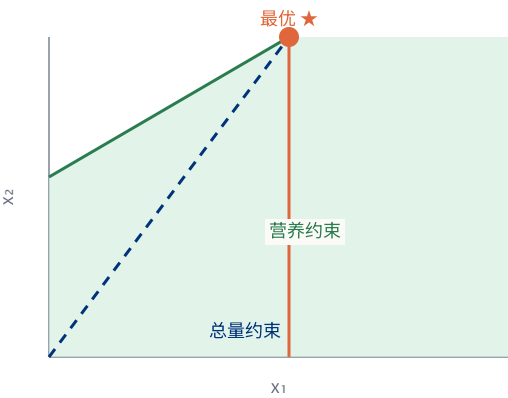
自顶向下 ≠ 只从顶开始：从“识别问题类”逐级下到模型、算法、验证。

案例：一个可手算的小 LP（配方问题）

问题描述与模型

两种原料 x_1, x_2 混合，单价 3、5；至少需营养 6 单位（每单位含 1、2），总量至少 4。求最低成本。

1	2	1	2	1	2
验证：最优 $(x_1, x_2)=(4,1)$ ，成本 17；敏感度显示营养约束最关键。					



建模→求解→验证：画出可行域、沿目标梯度找到顶点 (4,1)、再用敏感度确认哪个约束最关键。

CHAPTER 8

由粗到精方法

先用粗糙模型快速定位，再在局部精细求解——用廉价的近似换昂贵的精确。

工程项目的两类成本：显式与隐式

工程项目要在工期前完工，同时把材料、人工、能源、环境等总成本压到最低。这些成本对应优化模型的目标，分为两类：**显式成本**容易写成数学公式，**隐式成本**难以形式化。



露在水面上的只是少数。绝大多数真实成本——劳动力可达性、社区反对、等待与拥堵、设备磨损——都没能写进最初的目标函数。

简化目标 vs 复杂目标：五项权衡

选用简化目标还是复杂目标，是求解工程优化问题时最关键的一次取舍。

维度	简化的目标函数	复杂的目标函数
计算效率	快：时间与资源成本低	慢：时间与资源成本高
数据需求	只需有限的领域知识	需要大量领域知识
灵活性	表达能力受限	高度灵活
鲁棒性	结构简单，对噪声更鲁棒	对噪声更敏感
部署可行性	易于实时系统部署	需专用硬件

洞见

化简不是偷懒，而是在信息不足时先拿到一个解；复杂化不是目标，而是在项目周期允许时不断逼近真实。

一鸟在手，胜于百鸟在林

当一个工程优化问题对领域而言是全新的、没有现成方法可借鉴时，决策者必须从零开始探索。此时**拿到一个可接受的解**，比"存在一个（近）最优解但求不出来"更有价值。

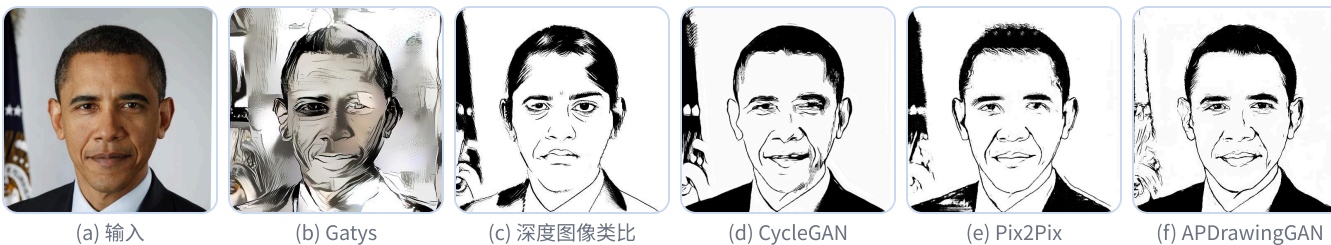
- **第一步**：设计一个简单模型，在可接受时间内得到可行解，问题由此变得可解。
- **第二步**：把更多因素加进模型，让它更接近真实问题，解也随之变好。
- **重复**：直到项目截止，或再无细节可加。

核心

由粗到精的出发点很朴素：**真正拿到的可行解，胜过拿不到的最优解。**

模型只是近似：一张由粗到精的画像

工程优化问题的数学模型，只是对底层复杂系统的一种近似——就像再逼真的画像也只是照片的近似。



奥巴马画像（六种方法）：(a) 输入 (b) Gatys (c) 深度图像类比 (d) CycleGAN (e) Pix2Pix (f) APDrawingGAN。六种“画像”都是对同一个人的不同近似——正如工程模型是对真实系统的不同近似（Yi et al., 2019）。

参考文献：Ran Yi, Yong-Jin Liu, Yu-Kun Lai, Paul L. Rosin. "APDrawingGAN: Generating Artistic Portrait Drawings from Face Photos with Hierarchical GANs." IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR 2019), 10743–10752.

含义

不要把“模型更细”当成“模型更真”的充分条件——细化只减少了某一类误差，不会消除模型与系统之间的鸿沟。

设施选址：一个典型的粗模型

设施选址是工业工程与物流管理中的经典问题：为工厂、仓库或配送中心选址，以最小化成本、提升效率或服务水平。

设施选址：一个粗模型

设 J 为候选设施点、 I 为客户。 x_j 表示是否建设设施 j ， y_{ij} 表示客户 i 是否由设施 j 服务。

$$\begin{aligned} \min \quad & \sum_{j \in J} c_j x_j + \sum_{i \in I} \sum_{j \in J} h_{ij} y_{ij} \\ \text{s.t.} \quad & \sum_{j \in J} y_{ij} = 1, \quad \forall i \in I \\ & y_{ij} \leq x_j, \quad \forall i \in I, j \in J \\ & x_j, y_{ij} \in \{0, 1\} \end{aligned}$$

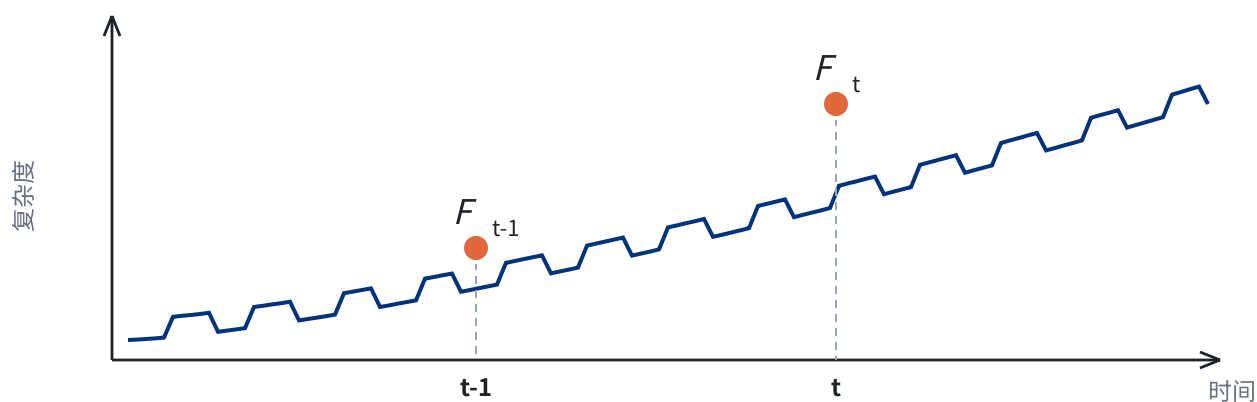
显式成本：设施建设成本 c_j 与配送成本 h_{ij} ，两者都写进了目标。

隐式成本：劳动力可达性、社会关系成本（例如“别建在我家后院”的邻避反应会引发抗议、负面报道、许可延迟，进而拖长工期并推高公关支出）——一个都没进目标。

特点：把隐式成本补进目标后得到的模型，称为“精模型”；粗模型是精模型的简化版本。

目标的近似：从 $F(t-1)$ 到 $F(t)$

设第 $t-1$ 期用简化目标 F_{t-1} 评估，第 t 期用 F_t 评估，且 F_t 比 F_{t-1} 更复杂。 P_{t-1} 通常是**单目标、线性、确定性**的；到了 P_t 则可能变成**非凸、非线性或含不确定性**的问题。



目标复杂度沿时间阶梯式上升：每一次“精化”都把问题推到更难的一类。

目标近似的定理：两条必然成立的不等式

记 X_t^* 为用 F_t 求得的最优解， X_{t-1}^* 为用 F_{t-1} 求得的最优解。对极小化问题，下面两条同时成立：

$$F_t(X_t^*) \leq F_t(X_{t-1}^*) \leq F_{t-1}(X_{t-1}^*) \leq F_{t-1}(X_t^*)$$

洞见 1

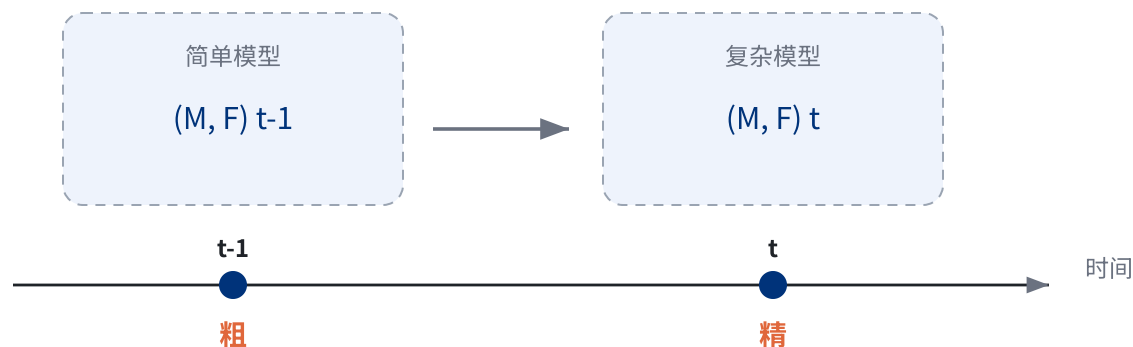
当 $F_{t-1}(X_{t-1}^*) = F_{t-1}(X_t^*)$ 时，第 $t-1$ 阶段的问题存在**多重最优解**。先枚举它们，再用更丰富的 F_t 去评估——这比在第 t 阶段直接硬解 F_t 省得多。

洞见 2

当 $F_t(X_t^*) = F_t(X_{t-1}^*)$ 时， X_{t-1}^* 是 F_{t-1} 意义下的一个**近优解**。因此第 t 阶段探索近优解也是有价值的；Gurobi、CPLEX 等求解器都提供多重最优解与解池的取用接口。

粗阶段与精阶段

M_t 是第 t 期以 F_t 为目标建立的数学模型。因为 F_t 比 F_{t-1} 更接近真实问题， M_t 也比 M_{t-1} 更复杂。



M_{t-1} 是粗模型， M_t 是精模型；两个阶段合起来构成一轮由粗到精。

- **粗阶段：**尽量简化目标函数（只保留一部分简单目标），并删去一部分复杂约束，使问题可解。
- **精阶段：**复用为 $t-1$ 开发的求解方法，去构造更贴切的 t ；把 $t-1$ 的多重最优解与近优解当作 t 的热启动（warm start）或初始解。

由粗到精的框架

粗阶段与精阶段合起来就是下面的框架。停止条件由决策者掌握：开发周期长时可能走十轮，周期短时可能只有一两轮。

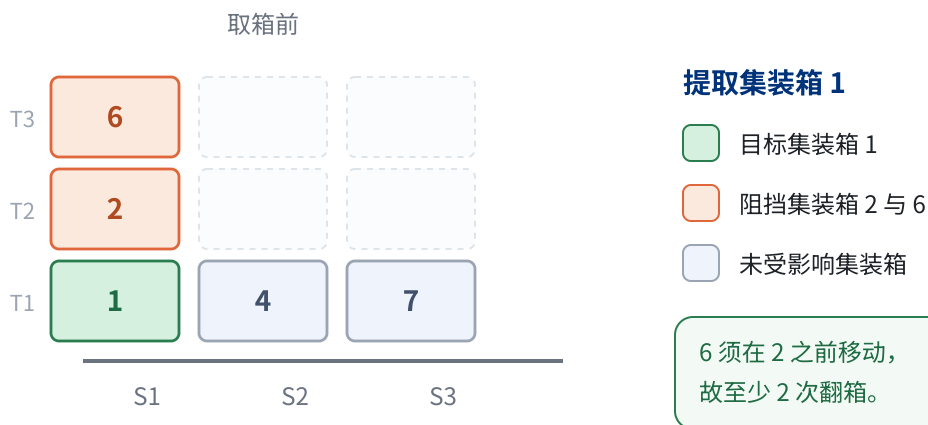
```
t = 0
while stopping conditions not met:           # coarse phase
    define a simplified objective F(t-1)
    build the coarse model M(t-1)
    develop a solution method for M(t-1)
    find all optimal solutions
    find near-optimal solutions
    # fine phase
    define a more complex objective F(t)
    evaluate F(t) on the optimal and near-optimal solutions
    if a better solution is found:
        build the fine model M(t)
        design a specialised method for M(t)
        t = t + 1
return the best solution
```

注意

"站在巨人的肩膀上"在工程上是有成本的：为一个简单模型开发求解方法也要投入大量时间与资金，所以精阶段必须复用它，而不是推倒重来。

案例：集装箱翻倒问题（CRP）

翻倒问题由 Kim 与 Hong 于 2004 年在线发表、2006 年正式刊出，至今已被研究二十余年。当目标箱被压在其他箱下面时，必须先把上方的阻挡箱临时搬到别处，才能取出目标箱——代价是移动次数、作业时间或设备磨损。

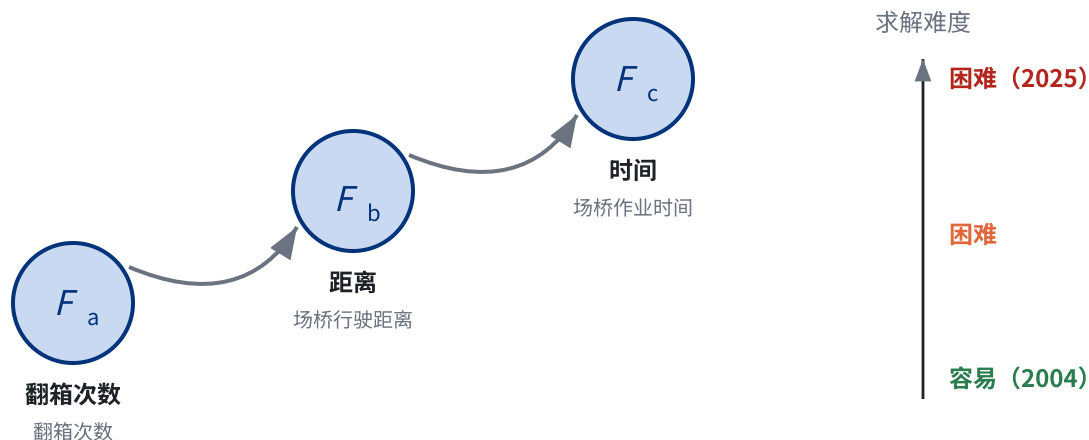


一个 3 栈 × 3 层的贝位。目标箱 1 位于 S1 底层，被 2 号与 6 号箱压住；由于 6 在 2 之上，必须**先搬 6 再搬 2**，因此至少要 2 次翻箱。

- **阻挡箱**：因上方压有其他箱而无法直接取到的箱。
- **翻箱（relocation）**：把阻挡箱临时移到其他栈或空位，腾出目标箱。

CRP 的三个目标：由粗到精的目标阶梯

翻倒问题最初的目标是**最小化场桥作业时间** F_c 。但场桥的作业时间难以评估，于是 Kim 与 Hong 用**翻箱总次数** F_a 去近似它；用**场桥移动距离** F_b 去近似，也比直接用 F_c 容易。



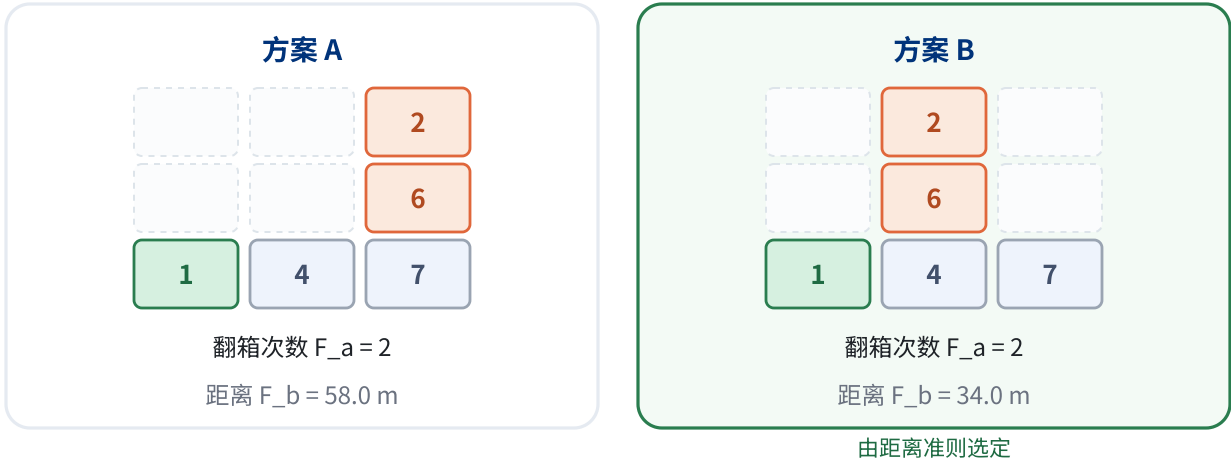
三个目标都能近似"翻箱代价", 但得到的 CRP 难度截然不同: F_a (2004 年, 易) $\rightarrow F_b$ (难) $\rightarrow F_c$ (2025 年, 很难)。

关键假设

用 F_a 近似 F_c , 隐含"翻箱次数越少, 场桥移动距离越短"这一假设。该假设**从未被数学证明**。

F(a) 下的多重最优解与距离准则

即使 F_a 极其简单，CRP 在 F_a 下仍然存在**多重最优解**：下面两个方案的翻箱次数都是 2， F_a 无法区分它们。



两种方案都需 2 次翻箱： F_a 无法区分， F_b 可以。

按"栈间距 6.0 m、层高 2.6 m、越障高度 9.0 m"的几何模型计算：方案 A 的移动距离为 58.0 m，方案 B 为 34.0 m。在 F_a 的最优解集里再按 F_b 排序，就能挑出更好的那个。

做法

这正是第 9 章的二次优化：

*

，这里

b_0 成本极低——只需对已有一批最优解重新算一次距离。

反例：同样的翻箱操作，距离却不同

两个堆场的初始布局不同，但执行**完全相同**的两次翻箱操作（把 6 号箱移到 S2，再把 2 号箱移到 S3）。落箱层位由各栈当前高度决定，因此移动距离并不相同。



布局 1 的落箱层位更低，总距离 48.6 m；布局 2 的落箱层位更高，总距离 43.4 m。翻箱次数相同，移动距离不同。

结论

因此"翻箱次数越少 $\Rightarrow$ 移动距离越短"并不总成立。用 a 近似 b （进而近似 c ）是有代价的近似——这正是需要由粗到精、而不是一步到位的原因。

三条洞见

以翻倒问题为线索，本章给出三条对实践最有用的结论。

- **简单目标 $\neq$ 简单答案。** 带简单目标的复杂工程优化问题容易求解，但这个简单目标**可能容纳多重最优解**——解不唯一本身就是可利用的资源。
- **更贴切的目标 $\neq$ 更好的解。** 换用更复杂的 t 并不保证得到的解在真实意义上更优；它只保证在 t 这个（仍然近似的）尺度上更优。
- **目标必须和项目周期一起权衡。** 面对一个全新的复杂问题，决策者必须在"目标的精细程度"与"项目的时间尺度"之间取平衡。

一句话

先拿到一个解，再让目标变复杂——**但永远记得目标只是系统的画像。**

CHAPTER 9

多解

工程里往往不止一个好解；给出一堆好解，比赌一个“最优”更稳妥。

为什么单个最优解不是终点

大多数教科书在求解器返回一个最优解 x^* 时就停下了。而工程实践恰恰从这里开始——**模型只是真实决策的简化**。下面三个事实，使得"好的解的集合"比其中任何一个解都更值得研究。

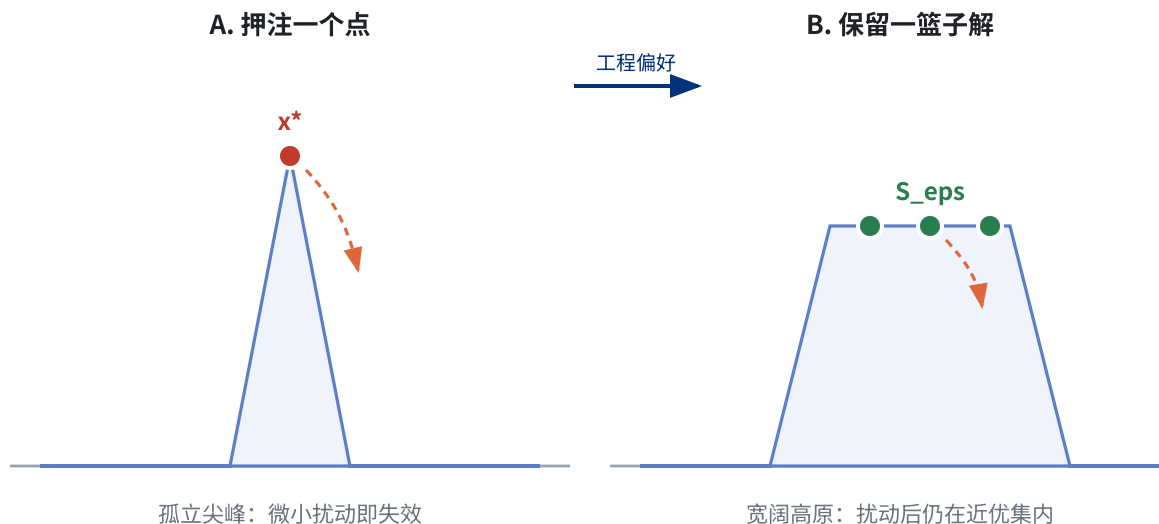
- **模型是不完整的。** 实施成本、操作员负荷、可维护性、安全裕度、合同弹性……这些维度模型几乎从不明确定价。两个被模型判为"等价"的解，在这些被省略的维度上可能天差地别。
- **模型是对称的。** 相同机器、可互换设施、可置换节点，都会让许多不同决策产生同一个目标值。求解器只是**任意**挑了一个。
- **数据是近似的。** 参数都是估计值。一个仅以毫厘之差胜出的解是脆弱的；一个略差一点、却坐在宽阔可行盆地里的解更安全。

要点

求解器给出的是一个**点**，决策者需要的是一次**知情的选择**。与最优解"实质上一样好"的那批解，才是工程价值真正所在之处。

工程为何要一篮子解，而非一个最优

数学告诉我们，最优集与近优集可以很大（9.3–9.7 节）。工程师之所以"要"这一整组解、而非求解器返回的孤点，根源在决策，而非代数：



左：单一最优解孤立、狭窄，现实稍有偏差就离开最优甚至不可行。右：近优集宽阔、平缓，扰动后仍处近优且可行——这正是工程想要的"弹性"。

- **不确定性与鲁棒性。** 名义最优 x^* 一旦现实偏离模型就可能违背约束。一篮子解让我们保留"近优邻域在任何扰动下都保持可行"的那个解。
- **风险不是一个数。** 我们在"一组解"上最小化期望后悔 ξ x^* ，单一坏实现就不会拖垮整个方案。
- **模型省略了人。** 可维护性、操作员负荷、安全文化、合同弹性几乎从不明确定价。在模型判为等价的解里，工程师挑被这些省略维度偏好的那一个。
- **多方目标冲突。** 成本、工期、安全被不同部门赋予不同权重。一组解支撑协商，而不是强加唯一赢家。

- **预案与期权价值。** 现场方案失败时，预先审核过的备选解即刻可用；为明天保留灵活性本身就有价值（灵活性价值）。

结论

单一最优解只是**数学产物**；一篮子好解才是**工程资产**。我们研究 ϵ 与 ϵ ，是为了让求解器成为推荐引擎，而非神谕。

定义：最优解集与替代最优解

要讨论"多重最优解"，先要把"所有最优解的集合"这个概念定下来。

最优解集

考虑标准形式的优化问题 $\max f(x)$ ，其中 F 是可行集。

$$S^* = \{x \in F : f(x) = f^*\}$$

替代最优解：任意满足 $x' \in S^*$ 且 $x' \neq x^*$ 的点。 S^* 的基数可以是 1（唯一最优）、有限（若干孤立最优点），或无穷（一个连续统，例如可行域的整条边或整个面）。

适用范畴：线性/凸规划、组合优化、混合整数规划——凡是目标对若干不同决策“无差别”的问题。

特点：求解器返回哪一个最优解，取决于实现细节（分支顺序、退化转轴、单纯形法的 pivoting 规则），与问题的工程含义无关。

替代最优解从何而来：四个常见来源

工程模型里遇到的替代最优解，几乎总能归入以下四类之一。

- **线性规划的退化。** 若干个不同的基可行解对应同一个顶点，而该顶点恰好最优；单纯形法也可能经过多次退化转轴才到达最优。
- **冗余约束。** 一条在最优处从不起作用的约束留下了松弛，求解器可以在这些松弛之间自由交换，于是产生一族最优解。
- **目标与某个面平行。** 当目标梯度与可行域的某个面正交时，整个面都取到同一个极值。
- **结构对称性。** 交换相同的资源——并行产线、对称的仓库选址、可置换的工件族——得到只是重贴标签的等价最优解。

最优解集的几何：凸性定理

对于工程优化中占主导地位的凸问题，最优解集的结构是被严格限定的。

定理：凸规划的最优解集

若可行集 是凸集、目标 是凸函数，则最优解集 S^* 也是凸的（可以是单点，也可以为空）。对线性规划

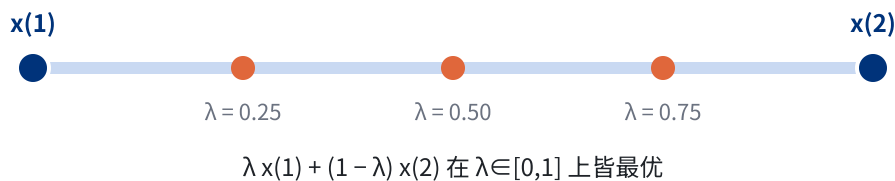
$$\min \{ c^T x : Ax \geq b, x \geq 0 \}$$

S^* 是可行多面体的一个面；只要它的维数大于 0，就包含不可数多个最优解。

适用范畴：凸规划、线性规划、以及一大类可凸化的工程模型。

特点：这正是 LP 求解器"返回任意一个最优解"都算正确的原因——集合是凸的，所以两个最优解的任意凸组合仍然最优。

若 $x(1)$ 与 $x(2)$ 皆最优，则其任意凸组合亦最优



推论：只要有二个最优解，就能批量制造更多——这也是后续"后最优分析"步骤的基础。

二次优化：用次级准则在最优解中挑选

"模型说这几个一样好"不是终点，而是**提问的起点**：既然模型分不出来，就交给模型之外的准则来分。

二次优化 (secondary optimization)

既然有许多个 都能达到 $*$ ，工程师就可以用一个**次级准则** 在它们之间挑选：

$$\min_x g(x) \quad \text{s.t.} \quad x \in S^*$$

等价地，先最小化 f 、再在所有最优的 f 之中最小化 g 的**字典序**问题。

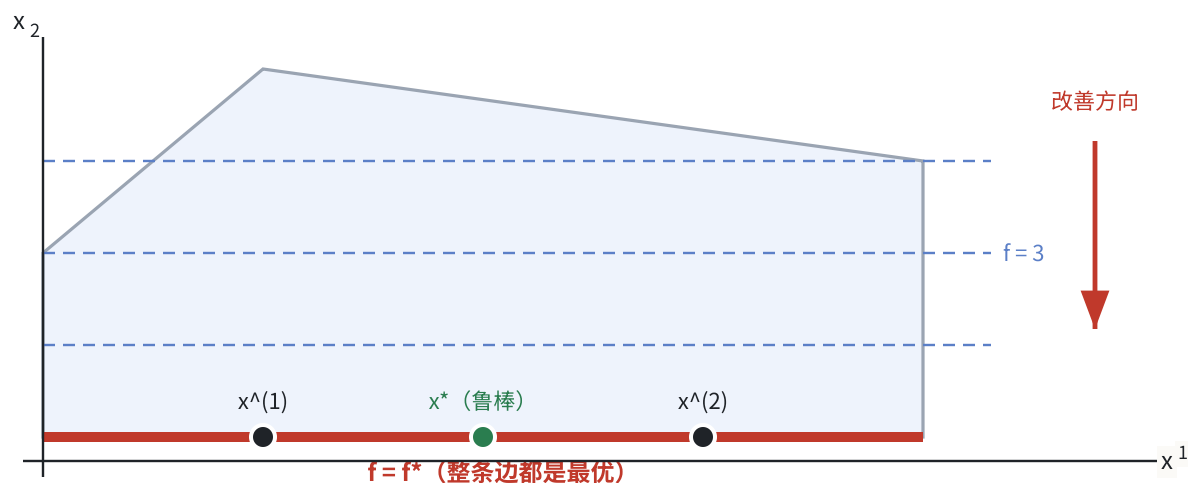
典型工程用法：在最优生产计划中更换线总时间最少的；在最优网络设计中开放最少的设施（更易运营与审计）；在最优调度中让关键路径的松弛最大（最抗设备故障）。

枚举办法

反复"砍掉当前解再重解"即可枚举替代最优解。对 0-1 变量用 **Hamming 割**： $j: x_j^* = 1 \rightarrow j$ $j: x_j^* = 0 \rightarrow j$ ；对连续问题则给目标平面加一个小扰动。

图示：一整条最优边

可行域为 $P = \{x \geq 1, x \leq 5, y \geq 1, x + 3y \leq 17, 2x - y \geq -1\}$ ，目标为最小化 $f = y$ 。目标等值线与下边界平行，于是**整条下边都是最优的**——不只是两个端点。



模型对 $x^{(1)}$ 、 $x^{(2)}$ 以及它们连线上的每一点都一视同仁，但工程师不能：把"最小松弛最大化"作为次级准则，最优点落在 $x = (3.00, 1)$ （最小松弛 2.00），而端点 $(1, 1)$ 与 $(5, 1)$ 只有 0.00。

提醒

* ，最优解有无穷多个。求解器返回端点是**实现细节**，不是工程结论。

定义：近优解集与 ε --最优

在实践中，模型误差本身就是 ε 的量级。死守唯一最优解、无视整个 ε --邻域，在统计上站不住脚。

--最优 (near-optimal)

最优解与"稍差一点"的解之间的差别，相对于模型误差而言往往毫无意义。因此把目标集放大：

$$S_\varepsilon = \{x \in F : f(x) \leq f^* + \varepsilon\}, \quad \varepsilon \geq 0$$

$\varepsilon = 0$ 时退化为 S^* 。 ε 的取值应反映模型的**保真度**：若数据只有百分之几的可信度，却要求解的精度高于此，那是**虚假精度**。

特点：若 F 与 f 都凸，则 S_ε 对任意 $\varepsilon \geq 0$ 都是凸的（两个凸集之交）。

例子：设最优值 $f^* = 100$ ，模型数据误差约 ± 2 。取 $\varepsilon = 3$ ，则 $S_3 = \{x : f(x) \leq 103\}$ 都是“足够好”的解；若强行要求 $f(x) = 100$ 才是解，是在追求超出数据精度的虚假精度。

为什么近优解更有价值：三个性质

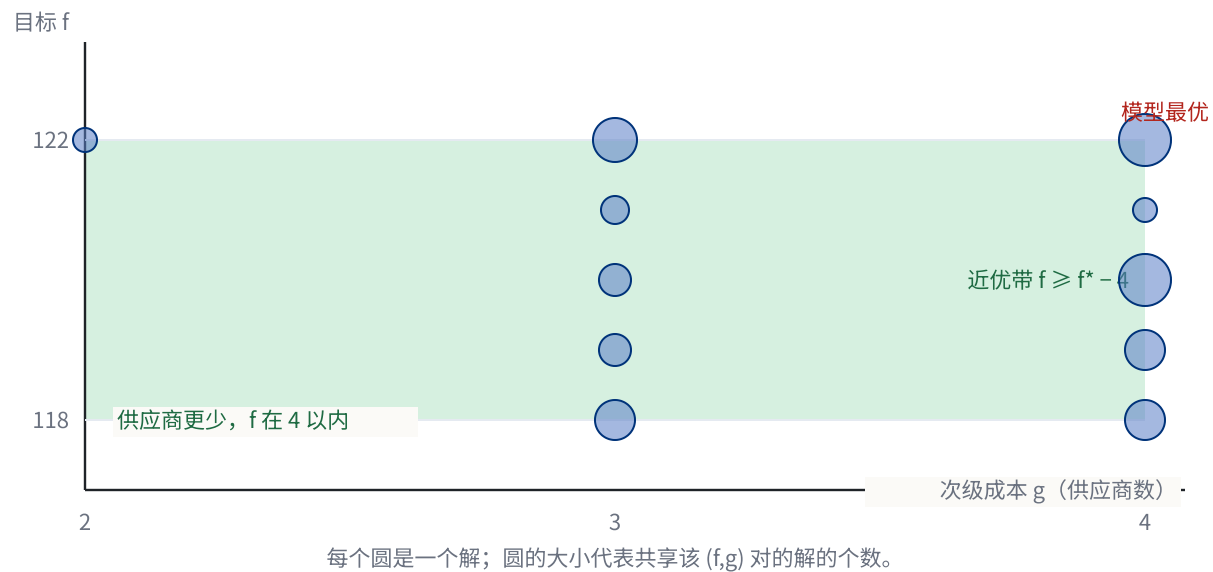
近优解集之所以是"真正该拿的战利品"，来自下面三个性质。

- **结构多样性。** 近优解往往不是彼此的微扰，而可能激活不同的约束、开放不同的设施、选择不同的网络拓扑。正是这种多样性，让一个次级目标**几乎免费**地得到满足。
- **鲁棒性与弹性。** 被宽阔的可行 --邻域包围的解，比孤零零立在尖峰上的解更稳定；数据的小扰动不容易摧毁它。
- **通向多目标思维的桥。** 把单目标放宽 并探索所得集合，正是把主目标与其他目标做交换的起点——也就是多目标优化里要系统化的那件事。

生成多样化解池

现代求解器都维护**解池** (solution pool)：一批按目标排序的互不相同的可行解。CPLEX 的 populate、Gurobi 的 PoolSearchMode / PoolSolutions、SCIP 都能在最优值的一个间隙内枚举多样解。

```
solve P → f*
add the epsilon-constraint f(x) ≥ f* - epsilon
pool = {x*}
while |pool| < K and feasible solutions remain:
    add a diversity cut excluding anything within r of a member of pool
    re-solve the extended problem
    if a new feasible x' is found: pool = pool + {x'}
    else: break
return pool
```



对容量为 58 的 0-1 背包穷举得到：最优值 $f^* = 122$ ，最优解 16 个；取 $\varepsilon = 4$ （不超过 3.3% 的损失）后，解池有 54 个解，其中"供应商数"从 2 到 4 —— **主目标几乎不动，次级成本却差了 2 家供应商。**

与其他章节的联系

"多重解"不是一个孤立技巧，它是贯穿全书的一条线索。

- **由粗到精（第 8 章）**：把粗模型的一批近优候选，当作精化阶段的输入——正是本章的 ϵ 。
- **鲁棒优化（第 11 章）**：偏好那个"近优邻域在任意不确定实现下都保持可行"的解。
- **多目标优化（第 13 章）**：把思路一般化——不再只是一条带，而是刻画完整的权衡（Pareto）曲面。
- **数学启发式与元启发式（第 18–20 章）**：维护并改进一个近优解池，就是它们的中心数据结构。

案例：同一个背包的解池

容量 58，10 件物品，穷举全部 $2^{10} = 1024$ 个子集。最优值 $f^* = 122$ ，最优解共 16 个。放宽 $\varepsilon = 4$ 后解池有 54 个解。

- **最优值：** f^* （16 个最优解），求解器只会给你其中一个。
- **近优池：** $f^* + \varepsilon$ 时共 54 个解，主目标最多损失 4（3.3%）。
- **次级准则的收益：** 把"供应商数"作为 g ，池内从 2 到 4 不等；选 $g = 2$ 的解，供应商数减少 2 家，主目标只损失不超过 4。

解读

模型误差本来就有 这么大。在 带里再优化次级目标，相当于**用模型不在乎的精度，换模型没算的收益。**

案例：Hamming 割枚举替代最优解

在同一背包上反复"加割—重解"，可以逐个取出不同的最优解： $(p=122, w=58, \text{items } 2,3,6,8,9,10) \rightarrow (p=122, w=58, \text{items } 1,2,4,5,6,9) \rightarrow (p=122, w=58, \text{items } 1,2,4,5,7,9)$ 。

$$j: x_j^* = 1 \quad j \quad j: x_j^* = 0 \quad j$$

- 这条割**只排除**当前解 x^* ，而把 x^* 完整保留下来。
- 重解后若目标值仍等于 x^* ，就找到了一个替代最优解；重复即可。
- 对连续问题改用**目标平面的小扰动**；Gurobi 与 CPLEX 都把这套流程做成了开箱可用的接口。

练习：最优边上的次级选择

回到第 s-c9f 页的多边形。 $f^* = 1$ ，最优集是整条线段 $\{(x, 1) : 1 \leq x \leq 5\}$ 。现在把"最小松弛最大化"当作次级准则 g 。

- 四个约束的松弛分别为 、 、 、 ；在最优边上 ，取四者最小值。
- 端点 与 的最小松弛都是 0.00 —— 它们**贴着约束边界**，数据一抖就可能变不可行。
- 中点附近的 最小松弛为 2.00，是这条边上最"抗抖"的点。

结论

同一个 x^* ，不同的工程后果。求解器挑 还是 纯属偶然，但"最抗抖的解"是 —— 这个结论只能靠次级准则得出。

本章小结

单个最优解是工程决策的**起点**，不是终点。

- 最优集 $\mathcal{X}^*$ 、尤其是近优集 $\mathcal{X}_\epsilon$ ，包含许多**模型分不出、工程师分得出**的解。
- 对凸问题与线性规划，这两个集合都是凸的（LP 情形下是多面体的一个面），因此容易刻画、也容易采样。
- 用排除割枚举替代最优解，或让现代求解器 populate 一个多样化解池，能把一个不透明的求解器变成**透明的推荐引擎**。
- 在"实质等价"的一批解里，挑那个最好实施、最鲁棒、或最公平的。

一句话

当模型说"这几个都一样好"，工程师应当追问："哪一个对模型没算到的那个世界最好？"

第三部分

工程优化问题

问题长什么样，往往决定了你能用什么方法。本篇按结构把问题分类：确定性/组合、鲁棒、双层、多目标、多阶段、博弈。

本篇逻辑框架

问题长什么样，往往决定了你能用什么方法。本篇按结构把问题分类：从确定性 / 组合，到鲁棒、双层、多目标、多阶段与竞争博弈。

本篇 7 章 · 按阅读顺序

10

确定性 / 组合

确定性 / 组合

组合优化



11

鲁棒优化

鲁棒优化

抗扰动



12

双层规划

双层规划

主从博弈



13

多目标

多目标

权衡前沿



14

目标规划

目标规划

柔性目标



15

多阶段

多阶段

时序决策



16

竞争博弈

竞争博弈

对抗均衡

逻辑主线：沿“确定 → 不确定”“单主体 → 多主体”“单阶段 → 多阶段”三轴展开，七章覆盖主要问题结构谱系。

主线：三条轴——确定性、参与者、时间——贯穿七章。

逻辑主线：沿“确定 → 不确定”“单主体 → 多主体”“单阶段 → 多阶段”三轴展开，七章覆盖主要问题结构谱系。

CHAPTER 10

确定性 / 组合

变量是连续还是离散、结构是可分解还是强耦合，直接决定问题落在 P 还是 NP-hard。

连续优化：LP 与 NLP

连续优化的变量取实数。线性规划（LP）目标与约束都是线性的，存在多项式时间内的单纯形/内点法；非线性规划（NLP）允许非线性项，难度陡增。

LP 标准形： $\min c^T x$ s.t. $Ax \leq b, x \geq 0$ 。凸 NLP 仍可能全局可解，非凸则只剩局部保证。

具体形式（运输 LP）

$$\min \sum_{i,j} c_{ij} x_{ij} \quad \text{s.t.} \quad \sum_j x_{ij} = a_i \ (\forall i), \sum_i x_{ij} = b_j \ (\forall j), x_{ij} \geq 0$$

例：2 产地运力 (20,30)，2 销地需求 (25,25)，运费矩阵 $\begin{pmatrix} 3 & 5 \\ 4 & 2 \end{pmatrix}$ 。这是 s-c10a 抽象形 $\min c^T x, Ax \leq b$ 的具体展开：变量连续、约束线性，故多项式可解。

应用：饲料配比是一个 LP

连续优化的典型应用：在比例性与可加性约束下最小化成本。下面就是一个可手算的 LP——它恰好对应 s-c10a 的连续问题。

具体数学模型（饲料配比）

$$\min 3x_1 + 5x_2 \quad \text{s.t.} \quad 2x_1 + x_2 \geq 8, \quad x_1 + 3x_2 \geq 9, \quad x_1, x_2 \geq 0$$

两种原料 x_1, x_2 （吨），成本 3/5（元/吨）；约束为蛋白质 8、纤维 9 的下限。最优解落在多面体顶点：单纯形法一步到位，最优 $x_1 = 3, x_2 = 2$ ，成本 19。放宽任一下限，可行域平移，顶点随之移动——这正是"连续、线性、多项式可解"的直观体现。

要点

连续问题一旦写成 LP，就有多项式保证；难点在"建模"（比例性是否成立），而非求解。

整数与组合优化

当变量必须是整数或取离散选择（选址、排班、路由），问题往往落入 NP-hard。这是工程优化里最常见的“难”。

- 0-1 变量刻画“是否”类决策（建不建这个仓库）。
- 组合爆炸： n 个二元变量有 2^n 种组合。
- 精确法（分支定界）能证最优但规模受限；启发式换速度。

具体形式（0-1 背包）

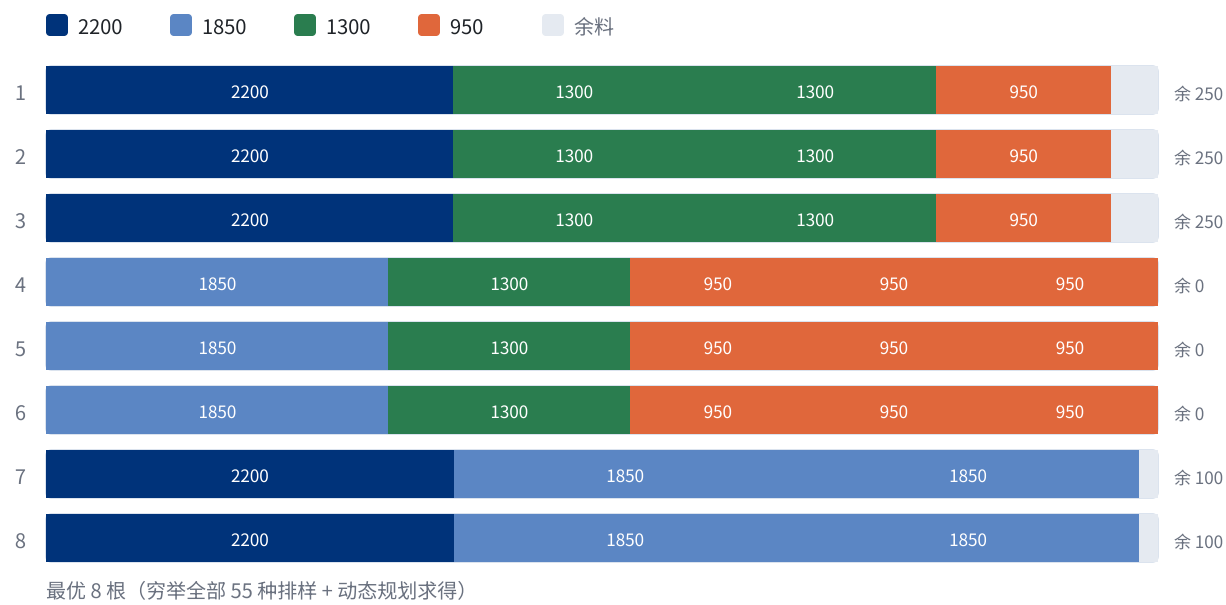
$$\max \sum_j p_j y_j \quad \text{s.t.} \quad \sum_j w_j y_j \leq W, \quad y_j \in \{0, 1\}$$

例：容量 $W = 8$ ，三件物品 $(p, w) = (6, 2), (10, 4), (12, 5)$ 。枚举 $2^3 = 8$ 种；最优 $y = (1, 0, 1)$ ，价值 18、重量 7。这正是“ n 个 0-1 变量 $\rightarrow 2^n$ 组合”的具体展开。

案例：一维下料（装箱与切割）

6000 mm 型材，四种长度共 33 件。逐根塞满（First-Fit Decreasing）是最自然的贪心，但它看不见

- 下界：总长 47,050 / 6,000 → 至少 8 根。
- 贪心 FFD 用 9 根，比最优多 12%。
- 最优 8 根（废料 950 mm，仅 2.0%）：3 根 (2200+1300×2+950)，3 根 (1850+1300+950×3)，2 根 (2200+1850×2)。



排样方式（pattern）才是决策变量，不是

网络流问题

许多工程问题本质是图上的流：最短路径、最大流、最小费用流。它们结构特殊，常有高效甚至多项式算法。

启示：先把问题识别成网络流，往往能绕过通用 MIP 的昂贵求解。

具体形式（最小费用流）

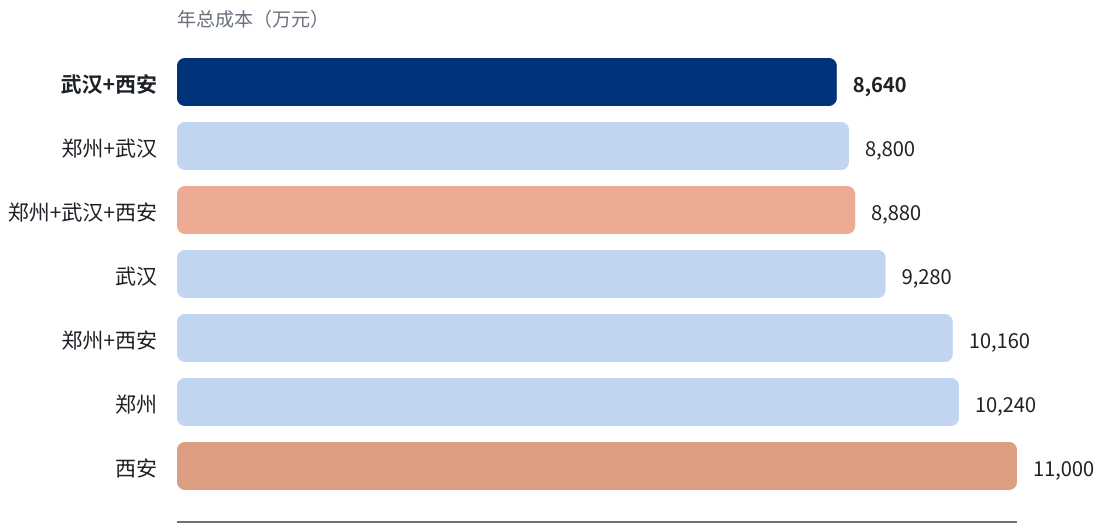
$$\min 2x_{SA} + 5x_{ST} + 1x_{AT} \quad \text{s.t.} \quad x_{SA} + x_{ST} = 12, x_{SA} = x_{AT}, 0 \leq x \leq 10$$

例：源 S 供 12、汇 T 需 12；弧 S→A (cap 10, cost 2)、S→T (cap 5, cost 5)、A→T (cap 10, cost 1)。最优走 S→A→T 用满 10、S→T 用 2，费用 30。这是 s-c10c 抽象守恒式的具体展开。

案例：冷链仓网（固定费用网络设计）

三个候选中心仓、四个需求城市。开仓要付固定费用，运费随距离变。这类问题里两个直觉都是错的。

- 直觉一
- 直觉二
- 最优是武汉 + 西安：8,640 万元。固定费用让目标函数不再单调，必须真的比一遍。



固定费用 + 线性运费 = 混合整数规划。它的可行域是若干离散点的并集，任何

调度问题

调度把“谁在何时做什么”排成序列，是组合优化的大本营：车间调度、车辆路径、码头作业都属于此。

- 单机/并行机：相对易解。
- 流水/作业车间：强 NP-hard。
- 带时间窗与资源约束：现实但更难。

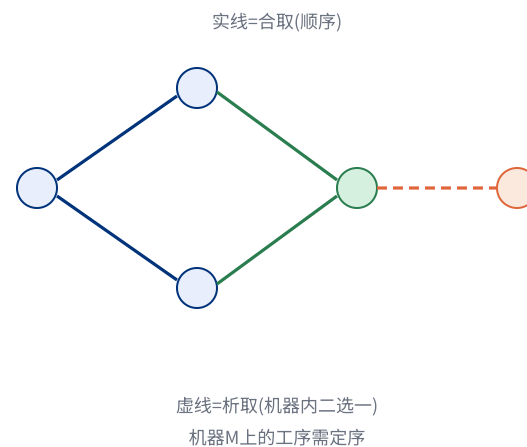
具体形式（2 机流水车间）

例：3 工件工时 $P = \begin{pmatrix} 3 & 2 \\ 4 & 1 \\ 2 & 3 \end{pmatrix}$ （行=工件，列=机器 1/2）。Johnson 规则得序 (1,3,2)，最大完工 11；随意序可达 13。这是 s-c10d 调度式的具体展开。

$$\min C_{\max} \quad \text{s.t.} \quad C_{\max} \geq p_{1,\pi(k)} + p_{2,\pi(k)} + \sum_{\ell} \dots$$

案例：Job-shop 调度（析取图）

- 每个工件是一道**工序序列**，必须按机器顺序加工。
- **合取弧**固定工序先后；**析取弧**在同机工序间二选一。
- 选好所有析取弧，就得到一个无环调度——这正是组合难点。



析取图把调度问题画成“待定序”的机器冲突；每化解一处析取弧，就离可行调度更近一步。

典型结构如何决定可解性

同一个“最小化”目标，因结构不同，可解性天差地别。识别结构是第一要务。

好结构：凸、网络、树、子模——常能高效或近似。

坏结构：非凸、强耦合、对称破缺难——常需启发式或精确法兜底。

分类的三条正交轴

确定性问题按其决定使用何种工具的维度进行分类。

By model shape (linear vs nonlinear)

- Linear → LP: simplex / interior-point, polynomial time.
- Nonlinear → NLP; convexity decides local = global.

By constraints (constrained vs unconstrained)

- Unconstrained → calculus / gradient search.
- Constrained → capacity, safety, regulation make it real.

By variable type (continuous vs combinatorial)

- Real-valued → smooth feasible region.
- Integer / binary → combinatorial explosion.

组合问题的统一模型：MIP

$$x,y \quad \begin{matrix} T & T \\ j & + & k \end{matrix}$$

这里 y 编码离散选择（分配、排序、挑选），而 x 是与它们耦合的连续量。这一统一形式支撑着泊位分配、岸桥调度、路径选择与存储优化。

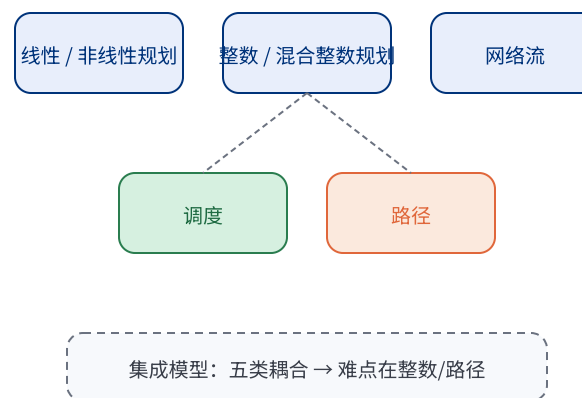
组合问题的五类原型

Archetype	Decision	Terminal instance
Assignment	one-to-one matching under cost	Quay crane ↔ berth section
Scheduling	order tasks on resources over time	Quay-crane load/unload sequence
Routing	visit nodes at minimum travel cost	AGV / yard-truck paths
Packing / cutting	fit items into capacitated bins	Container stowage, yard-block filling
Network design	choose cheapest arcs / nodes	Terminal layout, transshipment links

Each archetype hides an explosion: jobs → permutations; nodes → tours.

案例：码头的五类组合原型

- **LP/NLP**（连续）、**IP/MIP**（整数）、**网络流**、**调度**、**路径**。
- 一个码头同时包含这五类——它们彼此耦合。
- 结构决定**可解性**：哪类最难，往往就是瓶颈。



把码头拆成五类原型后一目了然：连续部分易解，整数与路径部分才是 NP-hard 的根源。

为何难：整数间隙与 NP-hard

- **Integrality gap.** The LP relaxation bounds the integer optimum; for the assignment problem the matrix is totally unimodular, so the relaxation is already integer-optimal.
- **NP-hardness.** TSP, bin-packing and most sequencing problems are NP-hard: no polynomial algorithm unless $P = NP$.
- **Engineering reading.** NP-hardness forbids a *guarantee* of optimality in polynomial time — blend exact (small) with heuristic (large).

连续与整数问题的标准型

这一章的分水岭是"决策变量能不能取分数"。能不能，决定了问题是多项式可解还是 NP-hard。

线性规划 LP

定义：目标函数与全部约束都是决策变量的线性函数，决策变量取连续值。

$$\min_x c^T x \quad \text{s.t.} \quad Ax \geq b, x \geq 0$$

适用范畴：配比、配料、运输、产能分配——只要比例性（加倍投入则加倍产出）与可加性（无交互项）成立。

特点：可行域是多面体，最优解必在顶点；多项式时间可解，强对偶成立，对偶变量就是影子价格。

例子：最小化 $2x_1 + 3x_2$ ，满足 $x_1 + x_2 \geq 4$, $x_1, x_2 \geq 0$ 。可行域是第一象限的半平面交集，最优解落在某顶点（如 (4, 0) 处目标值 8）。

非线性规划 NLP

定义：目标或约束中出现非线性函数，决策变量仍连续。

$$\min_x f(x) \quad \text{s.t.} \quad g_j(x) \leq 0, h_i(x) = 0, x \in \mathbb{R}^n$$

适用范畴：物理与化学定律（阻力、势能）、几何设计、风险与效用度量等含乘积或幂律的关系。

特点：凸 NLP（ f 与 g 凸、 h 仿射）下 KKT 点即全局最优；非凸时只能保证局部最优，需要多起点或全局优化。

例子： $\min x^2 + y^2$ s.t. $x + y = 1$ 。目标为二次（非线性），用拉格朗日乘子得最优解 (0.5, 0.5)，目标值 0.5。虽简单，但一般 NLP 没有闭式解，需梯度类迭代。

混合整数规划 MIP

定义：部分或全部变量被限制为整数（常见 0-1），用来表达离散选择、逻辑关系与固定费用。

$$\min_{x,y} c^T x + d^T y \quad \text{s.t.} \quad Ax + By \geq b, x \in \mathbb{Z}_+^n, y \geq 0$$

适用范畴：开/关、选与不选、批量、排班、路径——任何"是或否"的决策。

特点：NP-hard；LP 松弛给出下界，分支定界与割平面收紧；整数间隙衡量松弛有多松，而模型写法（对称性、big-M 的松紧）直接决定求解速度。

例子： $\min 4x + 3y$ s.t. $x + y \geq 2, x \in \{0, 1\}, y \geq 0$ 。变量 x 表示“是否建仓”，必须为 0 或 1；正是这个 0-1 选择让问题成为 NP-hard。

展开为具体数值（LP / NLP / MIP）

$$\text{LP: } \min 4x_1 + 3x_2 \text{ s.t. } x_1 + 2x_2 \geq 8, 3x_1 + x_2 \geq 9, x \geq 0;$$

$$\text{NLP: } \min x^2 + y^2 \text{ s.t. } x + y = 1 \Rightarrow (0.5, 0.5);$$

$$\text{MIP: } \min 5y_1 + 8y_2 \text{ s.t. } 3y_1 + 5y_2 \geq 7, y \in \{0, 1\}^2 \Rightarrow y = (1, 1).$$

抽象的 $c^T x$ 、 $f(x)$ 、 $(c^T x + d^T y)$ 在此化为具体系数；LP 顶点解、NLP 的 KKT 点、MIP 的 0-1 组合一目了然。

网络、调度与结构

同样是组合问题，结构不同则命运不同：网络流因为全单模而好解，调度因为排列而难解。

网络流问题

定义：在有向图上确定各弧流量，满足节点流量守恒与容量限制，并使总费用最小。

$$\min \sum_{(i,j) \in A} c_{ij} x_{ij} \quad \text{s.t.} \quad \sum_j x_{ij} - \sum_j x_{ji} = b_i \quad \forall i, \quad 0 \leq x_{ij} \leq u_{ij}$$

适用范畴：运输、转运、分配、最短路、最大流与最小割、多层供应链网络。

特点：约束矩阵全单模，LP 松弛的解自动是整数——“看起来是整数问题，其实多项式可解”；并有强多项式算法。

例子：三点 $S \rightarrow A \rightarrow T$ ，弧 $S \rightarrow A$ 容量 10、费用 2， $A \rightarrow T$ 容量 10、费用 1，需从 S 送 10 单位到 T 。最小费用流为各弧送 10 单位，总费用 $10 \times 2 + 10 \times 1 = 30$ ，且处处满足容量与流量守恒。

调度问题

定义：把一批作业分配到资源上并排序，优化完工时间、延误或吞吐量；常用 Graham 三元组 分类。

$$\min_{\pi} \sum_{j=1}^n w_j C_j, \quad C_{\pi(k)} = \sum_{\ell=1}^k p_{\pi(\ell)}$$

适用范畴：车间作业、港口岸桥、手术排程、计算集群、项目排期。

特点：除少数结构（SPT、WSPT、EDD 等规则最优）外基本是 NP-hard；常用析取图建模 + 优先规则或元启发式；换模时间、交期、机器可用性一变，可解性就跟着变。

例子：3 工件在 2 台机器上，工时矩阵 $P = \begin{pmatrix} 3 & 2 \\ 4 & 1 \\ 2 & 3 \end{pmatrix}$ （行=工件，列=机器 1/2）。用 Graham 三元组记为 $F 2||C_{\max}$ ；一台最优排序（如 $1 \rightarrow 3 \rightarrow 2$ ）使最大完工时间最小，体现“排序即决策”。

展开为具体数值（网络流 / 调度）

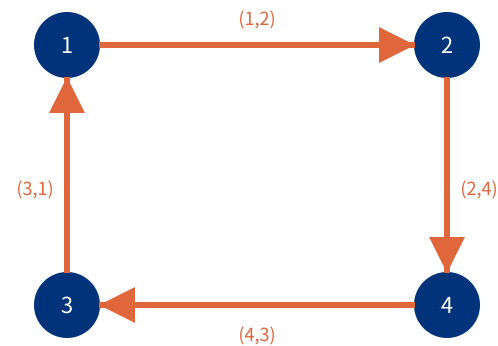
流: $\min 2x_{SA} + x_{AT} \text{ s.t. } x_{SA} = x_{AT} = 12, 0 \leq x \leq 10$ (费用 30);

调度: 3 工件 $P = \begin{pmatrix} 3 & 2 \\ 4 & 1 \\ 2 & 3 \end{pmatrix}$, Johnson 序 $(1, 3, 2)$, $C_{\max} = 11$.

抽象守恒式与 Graham 三元组在此化为具体弧流量与具体工时矩阵；"全单模→多项式"与"排列→NP-hard"有了可计算的对照。

小例子：4 点旅行商（TSP）

- 组合问题：决策变量取自有限集合（选哪台机器、哪条路线、哪个 bay）。
- 4 城市的 TSP：候选环只有 $(4-1)!/2 = 3$ 条，可穷举比较。
- 规模稍大即爆炸：n 个二值决策有至多 2^n 种配置。
- 这正是物流与制造的“工作母机”问题——分支定界证最优，元启发式扩到工业规模。



4 点 TSP：仅 3 条不同环 — 小实例可枚举

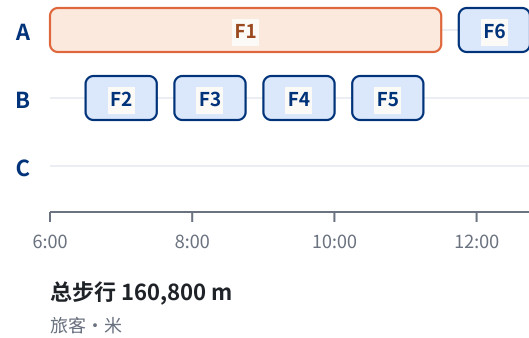
组合问题的难点不在建模，而在“搜索空间随规模指数膨胀”。

案例：机场登机口指派（贪心的陷阱）

六架航班、三个登机口，目标是让旅客走得最少。按

- 长停航班 F1 停 5.5 小时，把它放在近机位，就把后面 5 个短停航班全挤到远机位。
- 最优解相反：长停航班去 B（260 m），五个短停全部留在 A（120 m）。
- 差 39,200 旅客 · 米，贪心比最优差 32.2%。

贪心：先到先得占住近机位



最优：长停航班让出近机位



同一个指派问题，换一种

CHAPTER 11

鲁棒优化

数据从不精确。鲁棒优化把“最坏情况仍能接受”作为目标，用保守度换取可靠性。

先有模型，再谈不确定：从 MIP 到不确定性模型

优化问题通常先写成混合整数规划（MIP）。一旦某些系数不确定，同一个模型就要换成“在不确定集上做决策”。下面给出抽象形式、两个例子，再由“不确定到什么程度”决定后面用哪种方法。

抽象 MIP 模型

x, y

T

T

n

p

连续决策 与 0-1 决策 共同决定可行与成本。

例 1 · 产能规划：产量，需求 不确定。若，
须保证 对每一个可能需求都成立——一次决策扛住整族。

对应的不确定性模型

须

可行

不再只有一个，而是一族可能取值；决策要扛住整族。

例 2 · 投资组合：配置，收益 不确定。若只知道 的范围，须保证最坏情形下组合仍达标——这正是本章鲁棒对等要处理的。

“不确定性到底已知多少” 决定选哪种方法——这正是全书后段的分工：

- 只知道一个点估计（如需求=100）→ **确定性优化**：简单但脆弱（前几章已讲）。
- 已知概率分布（如需求 80/100/120 各 1/3）→ **随机规划 SP**：优化期望值，见第 15 章情景树 / 两阶段。

- 只知道取值范围 / 集合、无分布（如需求 $\in[80,120]$ ） $\rightarrow$ **鲁棒优化 RO**：防最坏情形，本章 s-c11c 预算集。

鲁棒对等问题

普通优化只盯着名义值（例如需求=100）求最优；可一旦真实需求是 120，方案可能直接崩溃。更糟的是，约束要“对 U 里每一个取值都成立”——这等于有无穷多条约束，列都列不完。

鲁棒对等就是这套要求的“可处理写法”：把“对每种情形都可行”翻译成一个带 $\min_{u \in U}$ 的优化——先让对手挑最坏情形，你再在主问题里防住它。就像签合同要写“无论油价涨跌都履约”，而不是赌油价不变。

形式： $\min_{x \in X} \max_{u \in U} f(x, u)$ 。外层的 $\min_x$ 就是“最坏情形”，内层约束要对所有 $u \in U$ 成立。

例子：约束 $a \leq x$ ，其中 $a \in \{8, 9, 10, 11, 12\}$ 。逐条写要求 $a=8, 9, 10, 11, 12$ 都成立，但因线性，等价于只防最坏 $a=12$ ，即 $x \geq 12$ 。鲁棒对等自动把无穷多条约束压成这一条。

预算不确定集

不确定集 U 的形状决定保守程度。预算集限制“同时偏离”的总量，是最常用的折中。

为什么不用盒式集？盒式假设“所有参数同时取最坏”（三个需求都冲到 120），这几乎从不发生，于是方案被无谓地过度设防。预算集（Bertsimas-Sim）更讲理：它只允许至多 k 个参数偏离名义值，其余保持 nominal。

预算集：
$$U = \{x \mid \sum_{i \in I} |x_i - \bar{x}_i| \leq k \sigma_i\}$$
。 k 越大越保守， $k=0$ 退化为确定性优化。

例子：3 个需求，名义 $\bar{x}_i$ ，波动 ± 20 。要备货 Q 满足 $Q \geq x_1 + x_2 + x_3$ 对所有情形。盒式集 $\rightarrow$ 最坏情形 $x_i = \bar{x}_i + 20$ ， $Q \geq 120$ ；预算集 $k=1 \rightarrow$ 至多 1 个偏离，最坏情形 $x_i = \bar{x}_i + 20, x_j = \bar{x}_j$ ，只需 $Q \geq 100$ 。预算集省下 40 单位库存，却仍防住“只有一个需求暴涨”的现实风险。

把 k 想成“容错旋钮”：调到 0 等于赌一切正常（确定性），调到 n 等于为“全员最坏”买单（盒式）。工程师按历史波动把它拧到合适位置。

保守度权衡

鲁棒性不是免费午餐：越保守，平均性能越差、资源越浪费。「」是工程上要拍板的旋钮。

- 低 ：贴着 nominal，省钱但怕波动。
- 高 ：抗冲击，但常年“过度设防”。

一个鲁棒示例

以集装箱配载为例：船舶吃水与稳性约束受货物密度估计误差影响。用预算集包住误差后，得到的解在真实密度波动下仍满足稳性。

代价：相比 nominal 配载，多用约 3% 舱容换来了“任何 $\pm 5\%$ 密度误差下都安全”的保证。

两种不确定性哲学

面对不确定性，两条主流路线押不同的注：随机规划赌“分布已知”，鲁棒优化赌“范围已知”。

随机规划 SP

- 假设随机向量 ξ 的分布已知（或可由数据估计）。
- 优化期望意义下的总成本。
- 弱点：分布一旦错配，解会被系统性带偏。
- **适用条件：**历史数据充足、分布可信、决策可重复（库存、发电计划、机组组合）。
- **例子：**应急库存。先订 x ，需求 ξ 有分布（如 80/100/120 各 1/3）；再按缺货/过剩补救。SP 最小化 $c x + \mathbb{E}[\text{补救成本}]$ ，而非只盯最坏。

鲁棒优化 RO

- 只要求参数落在不确定集 内，无需分布。
- 优化 上的最坏情形。
- 特点：对 中每个 u 都可行，无分布也成立。
- **适用条件：**数据稀缺、极端情形必须守住（安全约束、服务承诺、灾害应急）。
- **例子：**需求 $d \in [80, 120]$ ，进货 x ；要求对所有 $d \in U$ 满足 $x \geq d$ ，则须 $x=120$ ——最坏情形成本最高，却永不缺货。

鲁棒对应与不确定集

$$x \in X \quad u \in U$$

不确定性集合 U 越大，保护越强，但解也越保守（成本越高）。三种标准形状如下：

盒式 Box

$$x_j \in [l_j, u_j]$$

各参数独立区间；常常过于悲观、过度设防。

椭球 Ellipsoidal

$$\|x - \bar{x}\|_T \leq 1$$

用协方差耦合各参数，避开最坏极端。

预算集 Budget (Bertsimas-Sim)

$$\sum_{i \in I} |x_i - \bar{x}_i| \leq \Gamma$$

在名义（ $\bar{x}$ ）到完整盒式之间平滑插值，是常用折中。

Bertsimas–Sim 线性化

预算型不确定线性约束可转化为一小组线性约束，因此整个鲁棒线性规划仍是线性规划。

$$j \quad j \quad j \quad j \quad j \quad j \quad j$$

保护一条约束，只需多引入一个变量 与 个变量 j 。

可调鲁棒与工程选择

$$x \in X \quad u \in U \quad y \in Y(x,u)$$

- **可调鲁棒 ARO**：部分决策 在观测到 后才做出；用仿射决策规则 z_k^0 z_k^k 保持可解。
- **工程要点**：预算 依据历史数据标定——选对它往往比选求解器更重要。

处理不确定性的三种范式

差别在于"你对分布知道多少"：知道分布、只知范围、还是只知道样本。

随机规划 SP

定义：假设随机向量 的分布已知（或由数据估计），优化期望意义下的成本。

$$\min_x c^T x + E_{\xi}[Q(x, \xi)], \quad Q(x, \xi) = \min_y \{ q^T y : W y \geq h(\xi) - T(\xi)x \}$$

适用范畴：历史数据充足、分布可信、决策可重复执行：库存、产能、发电与机组计划。

特点：结果可解释（期望成本），但情景数随期数指数爆炸，需要采样或分解（L-shaped / Benders）；分布一旦错配，解就会系统性偏。

例子：应急库存。第一阶段决定备货 x ，需求 ξ 有分布（如 80/100/120 各 1/3）。第二阶段按缺货/过剩成本补救；SP 最小化 $cx + E_{\xi}[\text{补救成本}]$ ，而非只盯最坏情形。

鲁棒优化 RO

定义：不假设分布，只要求对所有属于不确定集 的情形都可行，并优化最坏情形下的目标。

$$\min_x \max_{\xi \in U} f(x, \xi) \quad \text{s.t.} \quad g(x, \xi) \leq 0 \quad \forall \xi \in U$$

适用范畴：数据稀缺、极端情形必须守住：安全约束、服务承诺、灾害与应急。

特点：计算上常可处理（盒式对偶后仍是线性，预算集可线性化）；代价是保守。不确定集的选取，本质上就是风险偏好的建模。

例子：需求 $d \in [80, 120]$ ，决策进货 x 。取盒式不确定集 $U = [80, 120]$ ，要求对所有 $d \in U$ 都满足 $x \geq d$ ，则须取 $x = 120$ ——最坏情形成本最高，但永不缺货。

分布鲁棒优化 DRO

定义：介于两者之间：真实分布未知，但被限定在一个模糊集（给定矩、或以经验分布为中心的 Wasserstein 球），优化最坏分布下的期望。

$$\min_x \sup_{P \in \mathcal{D}} E_P[f(x, \xi)]$$

适用范畴：有样本但不想赌上某个具体分布；小样本下的库存、机器学习与金融。

特点：比 RO 少保守（用上了分布信息），比 SP 稳健（不赌单一分布）；矩约束或 Wasserstein 球可把它转化为可解形式。

例子：同上库存，但只知需求均值 100、方差 400，不确定分布。取以经验分布为中心的 Wasserstein 球 D ，DRO 优化“最坏分布下”的期望成本——比 SP 更稳，比 RO 更省。

机会约束与不确定集的形状

不确定集的"形状"决定了对偶之后得到什么类型的约束，也就决定了问题好不好解。

机会约束规划 CCP

定义：允许小概率违约，把约束写成概率形式：满足约束的概率不低于 $1 - \alpha$ 。

$$\min c^T x \quad \text{s.t.} \quad P(g(x, \xi) \leq 0) \geq 1 - \alpha$$

适用范畴：服务水平、可靠性、缺货率、电网 N-1 校验等"允许偶发违约"的场合。

特点：语义直观 ("95% 不缺货")，但可行域通常非凸；只有特殊分布或离散情景 + big-M 才可精确处理，否则用采样近似 (SAA) 或 CVaR 等保守凸近似。

例子：电网备用容量约束写为 $P(\text{负荷} \leq \text{可用容量}) \geq 0.99$ 。允许 1% 违约概率，从而比“绝对不违约”的鲁棒约束大幅放宽，显著降低备用投资。

三种典型不确定集

定义：盒式（全部同时取最坏）、椭球（刻画相关性）、预算集（用 Γ 限制同时偏离的个数）。

$$U_\infty = \{\xi: \|\xi - \hat{\xi}\|_\infty \leq \rho\}, \quad U_2 = \{\xi: \Sigma^{-1/2}(\xi - \hat{\xi})^T \Sigma^{-1/2} \leq \Omega\}, \quad U_\Gamma = \{\xi: \sum_j \frac{|\xi_j - \hat{\xi}_j|}{\hat{\xi}_j} \leq \Gamma\}$$

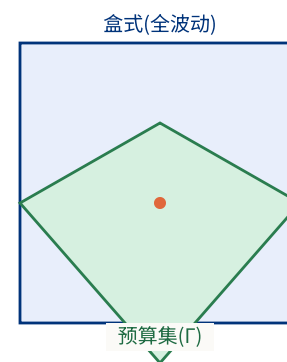
适用范畴：盒式用于最坏情况分析；椭球用于有协方差信息时；预算集用于"不可能所有参数同时最坏"的现实判断。

特点： Γ 从 0 调到 n ，就得到一条从名义解到盒式解的保守度曲线；椭球对偶后给出二阶锥约束。三者都在"可解"的范围内。

例子：需求估计 $\hat{\xi} = 100$ ，误差 ± 20 。盒式 $U_\infty = [80, 120]$ 要求全部最坏；预算集 $\Gamma = 1$ 只允许一个分量取最坏、其余取 nominal，得到的解更不保守。

案例：预算不确定集 (Γ)

- **盒式集**：每个参数独立地在 $\pm\%$ 内波动，过于保守。
- **预算集**：至多 Γ 个参数同时偏离，其余取 nominal。
- Γ 越大越保守，越小越接近确定性——**保守度可调**。

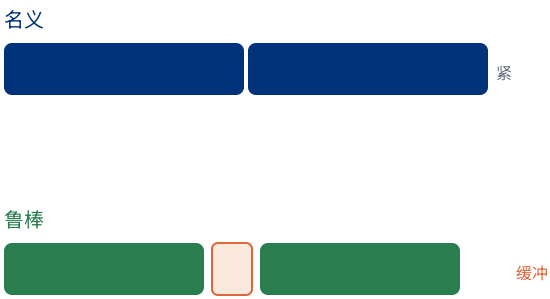


绿菱形 $\subset$ 蓝方框：仅 Γ 个偏离

盒式集允许所有参数同时最坏；预算集只允许 Γ 个同时最坏，从而大幅收紧保守度。

案例：不确定泊位分配

- 船舶到港时间有波动——**名义计划**排得很满，一延误就连锁崩盘。
- **鲁棒计划**在船期之间留缓冲，牺牲一点效率换稳定。
- 这就是**确定性 vs 速度**权衡在鲁棒视角下的体现。

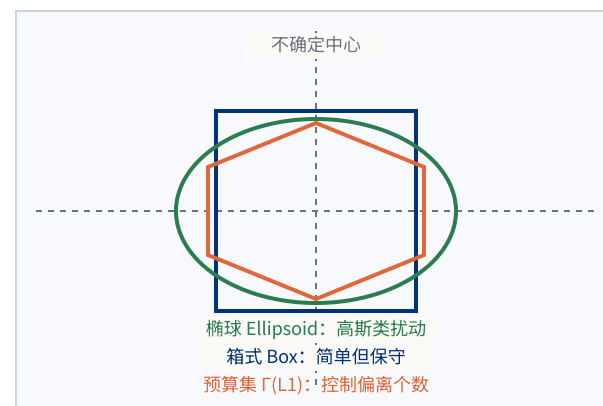


橙条=缓冲时间，吸收到港波动

名义计划把船排满以求吞吐；鲁棒计划插入缓冲以吸收延误，体现效率与稳定的交换。

不确定集：箱 / 椭球 / 预算 Γ

- 鲁棒优化在“不确定集”上对抗最坏情形，而非假设分布。
- 箱式**：各参数独立有界——简单，但往往过于保守。
- 椭球**：刻画高斯类扰动，适中保守度。
- 预算集 Γ (L1)**：至多 Γ 个参数可同时偏离——既可控又不太保守，是工程常用选择。

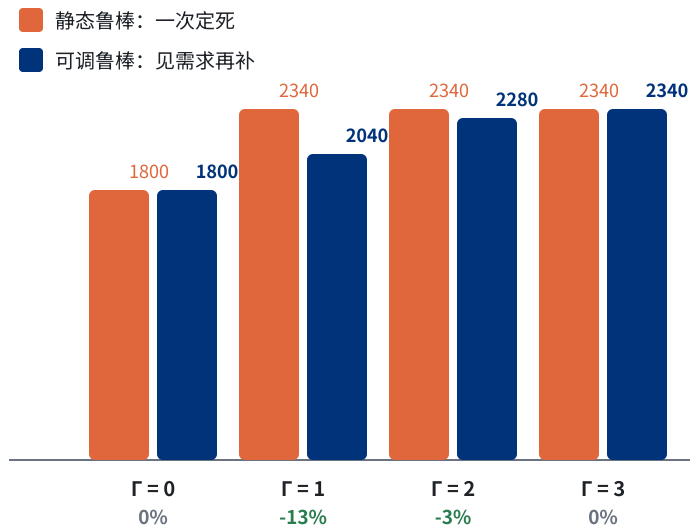
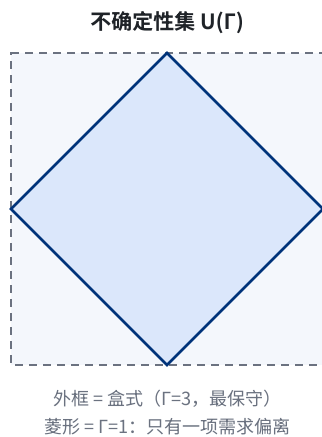


不确定集形状决定“保守度 vs 可解性”的权衡：越贴合现实，越难解。

案例：应急物资预置（静态鲁棒 vs 可调鲁棒）

三个区域，需求 100 ± 30 。现在预置 y （6/件），见到实际需求后再应急采购 z （8/件）。 Γ 控制

- $\Gamma = 0$ （名义值）：两者都是 1,800，可调性没有价值。
- $\Gamma = 1$ ：静态鲁棒 2,340，可调鲁棒 2,040 —— 省 12.8%，这时
- $\Gamma = 3$ （盒式，最保守）：两者又重合于 2,340，因为最坏情形总会出现。



可调性的价值不是常数：它在中等保守度下最大。把 Γ 当成免费的安全垫一路调大，反而把第二阶段能省的钱也一起锁死了。

CHAPTER 12

双层规划

当一方的决策卡在另一方的“最优反应”里，问题变成一层套一层——天然难解，却频繁出现在监管与市场中。

领导者—跟随者结构

有些问题天然分层：监管者定规则、企业据此最优反应；或总部分配、子公司执行。一方的“变量”卡在另一方的“最优反应”里。

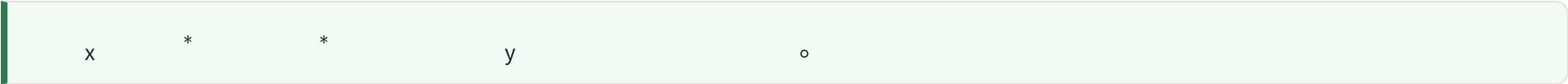


双层结构：上层先决策，下层在其约束下最优反应，再把结果反馈上层——一层套一层。

数学形式

上层选 x ，下层在给定 x 下选使自己目标最小的 y 。上层的目标同时依赖 x 与下层的反应 $y^*(x)$ 。

怎么读这个公式？当成一句话：外层“我选 x ”；但 x 一旦给定，内层立刻跑一个“在你给的 x 下，我选对自己最有利的 y^* ”的子问题；我（领导者）的目标 F 用的不是随便的 y ，而是这个最优反应 y^* 。就像家长定规矩、孩子按规矩选最舒服的做法——家长的满意度取决于孩子的“最优回应”。



例子：领导者设收费（即 x ），跟随者选路径使自身时间最小（即 y^* ）。领导者的目标 F 是总拥堵，依赖 x 与跟随者选出的 y^* 。改一下 x ，跟随者的最优路径可能整个变掉——这正是“嵌套”难解的根源。

为何难解

下层的最优反应可能是多值、不连续、甚至 NP-hard 的子问题。把一层塞进另一层的约束里，整体通常极难。

即使两层都是线性（线性双层），问题也已 NP-hard，且可能出现“缝隙解”需要特殊刻画。

求解思路

没有万能算法，按结构挑武器：

- KKT 改写：把下层最优条件写进上层，化为带互补约束的单层（MPCC）。
- 枚举/分支：下层反应有限时枚举候选，再分支定界。
- 启发式：遗传/粒子群在 x 上搜索，每个 x 内层单独求解。

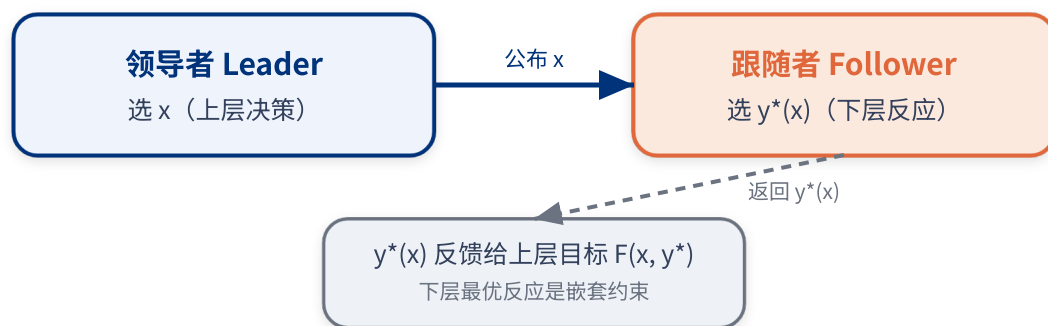
工程应用

凡是“一方定参数、另一方在其下优化”的场景都适用：

- 网络防御：攻击方选目标，防御方选布防。
- 供应链：品牌定批发价，零售商定订货量。
- 交通：路网管理者设收费，出行者选路径。

领导者-跟随者结构

领导者-跟随者：嵌套优化结构



领导者预判跟随者的最优反应；下层问题被“嵌套”在上层问题内部，这就是双层规划的本质。

双层规划模型



下层最优性是双层规划的本质特征： y 是对 x 的最优反应，而非自由变量。若存在多个最优解，领导者可持乐观或悲观态度。

具体数学例子（产能-反应）

上层（投资方）选产能 $x \geq 0$ ，目标 $F=x+2y$ ，约束 $x+y \geq 3$ ；下层（运营方）给定 x ，选 $y \geq 0$ 最小化 $f=y$ ，约束 $y \geq x$ 。

下层反应： $y^*(x)=x$ ($x \geq 0$ 时)。代回上层： $F=x+2x=3x$ ，且 $x+x \geq 3 \rightarrow x \geq 1.5$ 。

最优解： $x^*=1.5, y^*=1.5, F^*=4.5$ 。可见 y 不是自由变量，而是对 x 的最优反应。

为何难，如何解

本质难点

- 诱导出的可行域**非凸**。
- 即便上下层都线性，也**NP-hard**。
- * 可能跳变，导致 **不连续**。

求解思路

- **KKT 重构** → 单层 + 互补约束（MPEC）。
- **罚函数 / 正则化**：逼出唯一连续反应。
- **嵌套元启发式**：编码 ，对每个候选解求解下层。

双层结构与其博弈解释

双层规划的核心不是"两个问题", 而是"一个问题的目标里嵌着另一个问题的最优解"。

双层规划 BLP

定义：上层（领导者）先决策，下层（跟随者）看到上层决策后再求解自己的问题；上层目标依赖下层的解。

$$\min_{x \in X, y} F(x, y) \quad \text{s.t.} \quad G(x, y) \leq 0, \quad y \in \arg \min_{y' \in Y(x)} f(x, y')$$

适用范畴：定价与收费、补贴与规制、网络设计、安全资源分配、总部一分部的资源配置。

特点：即使全部线性也是 NP-hard；下层最优解集可能不唯一，需明确乐观解还是悲观解；可行域非凸，甚至可能不连通。

例子：领导者设收费 τ ，跟随者在给定 τ 下选路径使自身通行时间最小；领导者目标是系统总拥堵最小。均衡即「领导者给定 τ 、跟随者最优反应 $y^*(\tau)$ 」的 Stackelberg 解。

Stackelberg 博弈

定义：博弈论视角的双层：领导者承诺一个策略，跟随者给出最优反应，均衡由逆向归纳得到。

$$y^* \in BR(x) = \arg \max_y u_2(x, y'), \quad x^* \in \arg \max_x u_1(x, BR(x))$$

适用范畴：市场进入、价格领导、关税与补贴设计、平台抽成比例。

特点：先动者通常优于同时行动的纳什均衡（先动优势）；前提是承诺必须可信；当最优反应不连续时，均衡可能根本不存在。

例子：政府设碳税 τ （领导者），企业选减排投入 y （跟随者）最小化自身成本。企业最优反应 $y^*(\tau)$ ；政府选 τ 使社会总减排成本最小，均衡即“先承诺、后反应”的 Stackelberg 解。

把双层变成单层：KKT 重构与 MPEC

工程上求解双层的标准做法是把下层换成它的最优性条件——但这一步会引入互补约束，带来新的麻烦。

KKT 单层重构

定义：当下层问题是凸的且满足约束规范时，把它替换为其 KKT 条件，双层问题化为一个单层问题。

$$\nabla_y f(x, y) + \lambda^\top \nabla_y g(x, y) = 0, \quad 0 \leq \lambda \perp g(x, y) \leq 0$$

适用范畴：下层为凸、可微，且满足 LICQ / MFCQ 的情形。

特点：换来单层结构，但互补约束让 MFCQ 处处不成立，普通 NLP 求解器会失败；必须用正则化、SOS1 或大 M 等专门技巧。

例子：下层 $\min_y y^2$ s.t. $y \geq x$ 。凸且可微，其 KKT 为 $2y - \lambda = 0, \lambda \geq 0, y \geq x, \lambda(y - x) = 0$ 。连同这些约束把双层化为单层，可用现成求解器处理。

MPEC 与 EPEC

定义：MPEC：一个领导者 + 下层均衡约束；EPEC：多个领导者各自带下层均衡，再在领导者之间求均衡。

$$\text{find } x^* : \quad x_i^* \in \arg \min_{x_i} F_i(x_i, x_{-i}^*, y_i(x^*)) \quad \forall i$$

适用范畴：电力市场竞价（多家发电商报价 + 调度中心出清）、多主体网络设计与容量投资。

特点：难度显著高于 MPEC；实践中常用对角化（循环求解单个 MPEC）迭代，但收敛性没有保证，可能振荡或不收敛。

例子：电力市场——一个寡头（领导者）决定装机，其下层是“各机组按成本竞标达到均衡”（均衡约束）；多个这样领导者之间再求 Nash 均衡，就是 EPEC。均衡约束使问题比普通 MIP 更难。

案例：收费设定（领导者—跟随者）

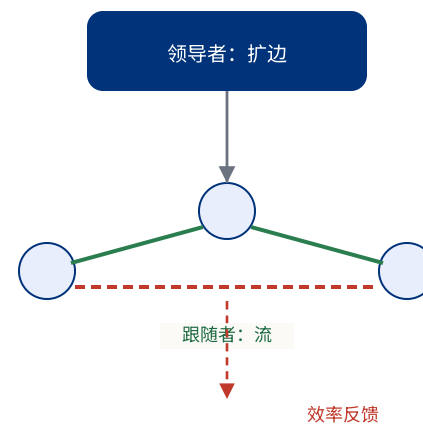
- **领导者**（交管）设定各路段收费 τ 。
- **跟随者**（承运商）在给定 τ 下选最便宜路径。
- 领导者的目标（收入/拥堵）取决于跟随者如何反应——**嵌套优化**。



领导者不能无视跟随者：它必须把“收费→承运商反应→自身目标”作为一个整体来优化。

案例：网络设计双层

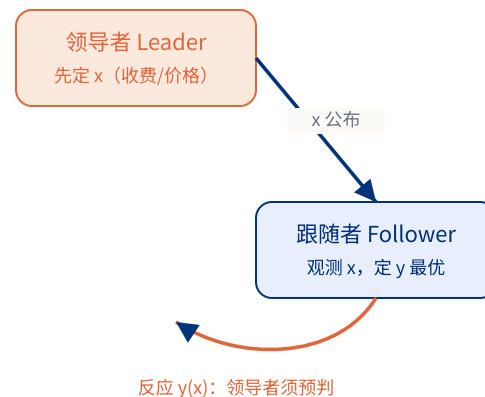
- **领导者**决定在哪些边扩建容量（花钱）。
- **跟随者**在扩建后的网络上做最短路/最小费用流。
- 领导者权衡**投资**与**系统效率提升**。



扩建哪条边，取决于扩建后跟随者会怎样重新分配流量——领导者必须先“模拟”跟随者。

Stackelberg: 领导者—跟随者

- 双层问题含两个相互作用的决策层：上层（领导者）与下层（跟随者）。
- 领导者先定 x （如收费、价格）；跟随者观测 x 后，选对自己最优的 y 。
- 关键点：领导者在优化时必须**预判**跟随者的反应 $y(x)$ 。
- 常见实例：收费设定、领导者—跟随者产品设计——均属机制设计而非单纯求最优。

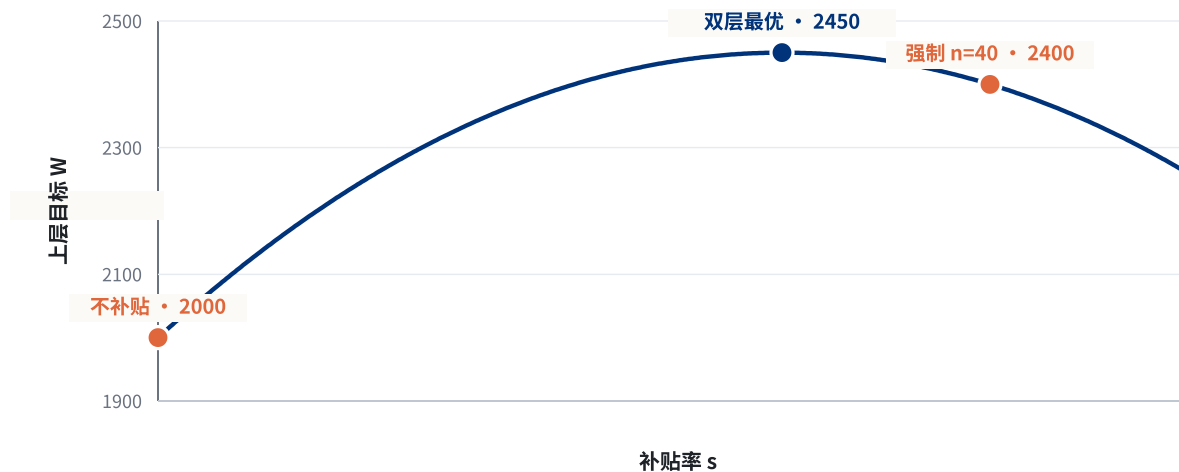


工程师常是“规则制定者”而非玩家：设计让均衡变好的机制。

案例：充电网络补贴（政府是领导者）

政府先定补贴率 s ，运营商再按利润最大化决定建多少桩 $n(s) = 20 + 50s$ 。上层若忽略这条反应曲线，就会算错最优补贴。

- 不补贴： $n = 20$ ，社会净收益 2,000。
- 强制 $n = 40$ （需要 $s = 0.4$ ）：净收益 2,400 —— 覆盖面更大，反而更差。
- 双层最优 $s^* = 0.30$ ， $n^* = 35$ ，净收益 2,450，两者都优于极端方案。



双层规划的关键不是

CHAPTER 13

多目标

当目标互相冲突，没有“全最好”，只有一条 Pareto 前沿——算法给选项，人来做决断。

多目标优化与 Pareto 最优

当目标互相冲突（成本 vs 时间、安全 vs 效率），没有“全最好”的解，只有一组“互不支配”的解——Pareto 前沿。

数学形式（向量极小化）

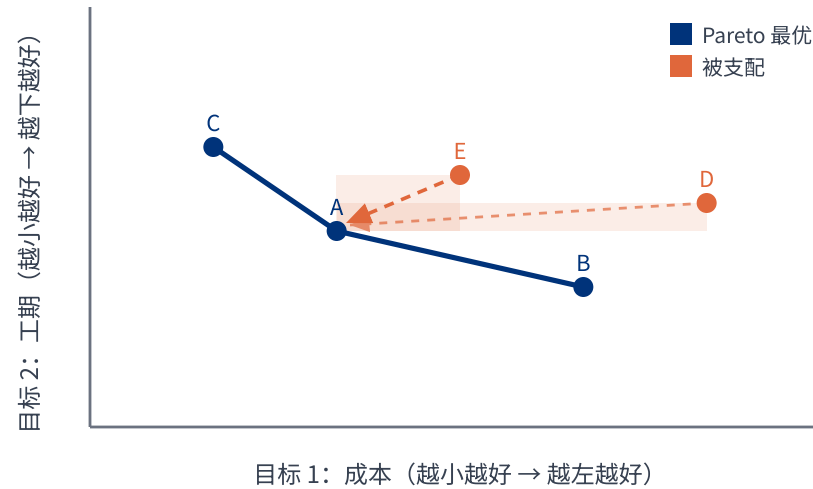
$$x \in X \quad \begin{matrix} 1 & 2 & & k & & j \end{matrix}$$

目标是向量，没有自然的全序：无法说 (3,5) 与 (4,4) 谁“更大”。

Pareto 定义

支配：若解 y 满足 $f_i(y) \leq f_i(x)$ （所有 i ）且至少一个严格更小，则 y **支配** x 。

Pareto 最优：若无任何解支配 x ，则 x 为 Pareto 最优；全体构成 **Pareto 前沿**。



前沿（蓝 A/B/C）上的点互不支配：想改善一个目标，必牺牲另一个。橙色点（如 E）被 A 在“两项都更好”的意义下支配；D 同理。

数值例子：5 个设计方案，目标均为越小越好

成本 / 工期：(A)100/12、(B)120/10、(C)90/15、(D)130/13、(E)110/14。

Pareto 最优 = {A, B, C}（互有取舍）；D、E 被 A 在两项上都超越，故**被支配**。

标量化方法

把多目标合成单目标，就能复用单目标求解器。权重法是常见起点。

为什么要标量化？因为单目标求解器（线性规划、梯度法等）已经非常成熟，而多目标之间没有天然的“谁大谁小”可比。标量化就是给各目标配权重、合成一个分数，把多目标“降级”成单目标来算。代价是：权重就是你的偏好，换一组权重就得到前沿上的另一个点。

- 加权求和： $\sum_i w_i f_i$ ，权重反映偏好。
- 切比雪夫： $\max_i w_i |f_i - f_i^*|$ ，更易铺满前沿。
- 缺陷：非凸前沿上加权求和会漏解。

例子：两个目标——成本 f_1 与安全 f_2 （越大越好，记 f_2 ）。权重 (0.7,0.3) 偏向省钱，得到前沿“便宜但一般安全”那一端；权重 (0.3,0.7) 偏向安全，解滑向“贵但更安全”那一端。把权重从 0.7 扫到 0.3，就描出整条 Pareto 前沿。

ϵ -约束法

保留一个目标最小化，其余目标转成约束 ($\leq \epsilon$)。扫描 ϵ 即可逐点描出前沿，对非凸前沿也有效。

形式：
$$\min z_1$$
$$s.t. \quad z_j \leq \epsilon_j, \quad j = 2, \dots, m$$
。每个 ϵ 组合给出一个前沿点。

前沿的近似

现代多目标算法（NSGA-II、MOEA/D）用种群一次给出一组多样的前沿近似，而非逐点求解。

- 多样性：解要尽量铺满前沿，不挤在一处。
- 收敛性：整体要贴近真实前沿。
- 指标：HV（超体积）、IGD 用以量化“好前沿”。

决策者的角色

算法只给“一组互不支配的解”，最终选哪一个仍是人的判断——这是多目标优化与单目标最本质的区别。

做法：把前沿交给决策者，用第 9 章的多解展示法让其交互筛选，而非替他拍板。

一个多目标小例

码头配载想同时最小化“翻倒次数”（安全）与“岸桥移动距离”（效率）。两目标冲突：少翻倒常要多移动。

前沿上每一点是一种权衡：决策者按当天船期紧不紧，在“更稳”与“更快”之间挑一点。

支配与帕累托最优

$x \preceq x' \iff f_i(x) \leq f_i(x') \text{ for all } i; f_j(x)$

- A solution is **Pareto optimal** if no other dominates it.
- The set of all such points is the **Pareto set**; its image is the **Pareto front**.
- The front is the engineer's map of the trade-off landscape — exploring it beats reporting one point.

标量化：加权和与 ϵ -约束

Weighted sum

$$\min \sum_i w_i f_i(x)$$

Every positive weight gives a Pareto point; misses non-convex (concave) parts.

ϵ -constraint

$$\min f_1(x)$$
$$\text{s.t. } f_i(x) \leq \epsilon_i, \quad i = 2, \dots, n$$

Sweep ϵ_i to trace the front, including non-convex regions.

进化多目标：NSGA-II 与 MOEA/D

NSGA-II

- Rank by non-domination layers.
- Crowding distance keeps spread along the front.

MOEA/D

- Decompose into scalar sub-problems.
- Shares neighborhood information; strong on many objectives.

EMOAs approximate — they do not prove — the front; the shape of the trade-off is the real deliverable.

帕累托最优与两种标量化

多目标问题不产出"一个最优解", 而是产出一组不可互相替代的解; 标量化只是把选择权交还给决策者。

帕累托最优

定义: 若不存在另一可行解在所有目标上都不更差、且至少一个严格更好, 则该解非支配 (帕累托最优)。

$$x^* \in X \text{ 非支配} \Leftrightarrow \nexists x \in X : f_k(x) \leq f_k(x^*) \ \forall k, \exists k : f_k(x) < f_k(x^*)$$

适用范畴: 目标不可公度 (成本 vs 排放、收益 vs 风险), 且不应由建模者预先设定换算率。

特点: 解是一个集合 (前沿) 而非单点; 需区分弱帕累托与严格帕累托; 加权和只能给出其中的支撑部分。

例子: 两目标 $\min(f_1, f_2)$, 候选解 $A(2, 5), B(3, 3), C(5, 2)$ 。A 不被 B 支配 (f_1 更小但 f_2 更大), B, C 互不支配, 三者都在 Pareto 前沿上。

加权和标量化

定义: 按非负权重把多个目标合成为一个目标, 再交给任意单目标求解器。

$$\min_{x \in X} \sum_{k=1}^p w_k f_k(x), \quad w_k \geq 0, \sum_k w_k = 1$$

适用范畴: 前沿是凸的、或决策者愿意并能够给出权重。

特点: 最简单、可复用任何求解器; 但会漏掉非凸部分的解, 且权重均匀变化并不对应前沿上的均匀分布。

例子: 两目标 $\min(f_1 = \text{成本}, f_2 = \text{排放})$ 。取 $w = (0.6, 0.4)$, 最小化 $0.6f_1 + 0.4f_2$, 交给任意 LP/MILP 求解器即得一个折中解; 换不同 w 可描出 Pareto 前沿的支撑部分。

ϵ -约束法

定义：保留一个目标作为优化对象，其余目标转为带阈值 的约束。

$$\min_x f_\ell(x) \quad \text{s.t.} \quad f_k(x) \leq \varepsilon_k \quad (k \neq \ell), \quad x \in X$$

适用范畴：需要完整（含非凸部分）的前沿，或存在明确的"必须达到"的水平（如排放上限、预算红线）。

特点：能取到加权和取不到的解；但需要先选 ε ，取值不可行会导致无解，因此通常要系统扫描 (payoff table)。

例子：仍以成本、排放为例，固定排放上限 $\varepsilon_2 = 50$ ，最小化成本 $\min f_1 \text{ s.t. } f_2 \leq 50$ 。逐次放松/收紧 ε_2 即可沿前沿取点，且能刻画非凸（非支撑）部分，这是加权和做不到的。

进化算法与决策者的接口

当前沿太大、太扭曲、或根本写不出导数时，就只能让一群解去"逼近"它。

进化多目标 (NSGA-II / MOEA/D)

定义：用种群同时逼近整个前沿：NSGA-II 靠非支配排序 + 拥挤度保持分布，MOEA/D 把问题分解为一组标量子问题协同求解。

$$x < y \Leftrightarrow f_k(x) \leq f_k(y) \ \forall k, \exists k: f_k(x) < f_k(y)$$

适用范畴：大规模、非凸、不可微、黑箱（仿真或实验评估）的多目标问题。

特点：一次运行给出一批解，天然处理非凸与离散决策；但不保证最优，需要调参，且每次评估昂贵时成本很高。

例子：NSGA-II 维护一群候选解，用“非支配排序”保留优秀者、用“拥挤度”保持分布均匀；迭代若干代后，种群收敛到一条分布良好的 Pareto 前沿，无需预先给权重。

先验 / 后验 / 交互式

定义：先验：先问偏好再求解；后验：先求整个前沿再让决策者挑选；交互式：求解与偏好表达交替进行。

循环：给出候选解 → 决策者指出偏好 → 缩小区域重新求解

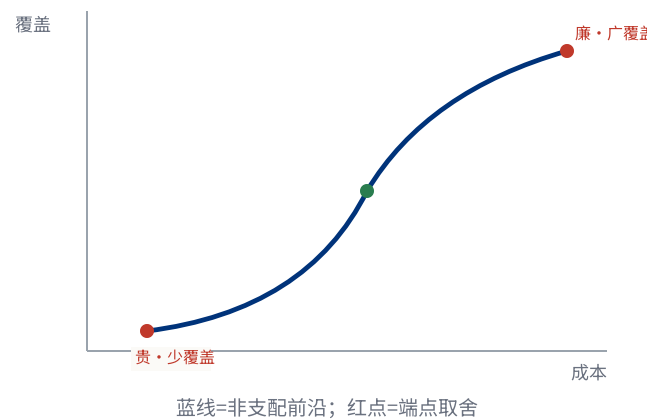
适用范畴：先验适合时间紧、目标少；后验适合需要理解权衡结构；交互式适合前沿大得无法一次展示。

特点：多目标建模的价值在于把权衡显式化，而不是替决策者做决定——方法的选择取决于决策者愿意投入多少时间。

例子：后验法——先让 NSGA-II 吐出 20 个互不支配的解，决策者对照“成本-排放”散点图挑出满意的那个；交互式则在每轮后让决策者说“更看重排放”，算法据此缩小搜索区再算。

案例：设施选址的 Pareto 前沿

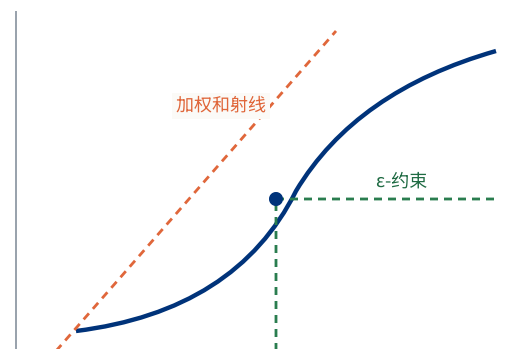
- 两个冲突目标：**建设成本** ↓ 与 **覆盖人口** ↑。
- 没有单一最优，而是一条**非支配前沿**：沿前沿改进一个必牺牲另一个。
- 决策者沿前沿**挑一点**，而非算法替他定。



想全覆盖就得多花钱（右端），想省钱就少覆盖（左端）；前沿上的每一点都是一种合理的妥协。

案例：标量化——加权 vs ε -约束

- **加权法**：把多目标合成 $w_1 x_1 + w_2 x_2$ ，沿不同权重扫出前沿。
- **ε -约束**：固定一个目标 $\leq \varepsilon$ ，再优化另一个，逐次移动 ε 。
- 加权法可能漏掉**非凸**段； ε -约束能覆盖全部。



橙=加权扫 · 绿= ε -约束逐点

加权法像从原点射出射线切线（可能漏非凸段）； ε -约束则像水平线逐点采集前沿。

权重扫描：沿 Pareto 前沿取点

- 多目标无单一最优，解是一组不可比的**Pareto 前沿**。
- 加权法：把目标合成 $w_1f_1 + w_2f_2$ ，扫不同权重 w 即可沿前沿取不同点。
- 图中三枚蓝点： $w=(0.8,0.2)$ 、 $(0.5,0.5)$ 、 $(0.2,0.8)$ 分别落在前沿不同位置。
- 工程师事后在 Pareto 解集上按需挑选——这也是多解价值的体现。

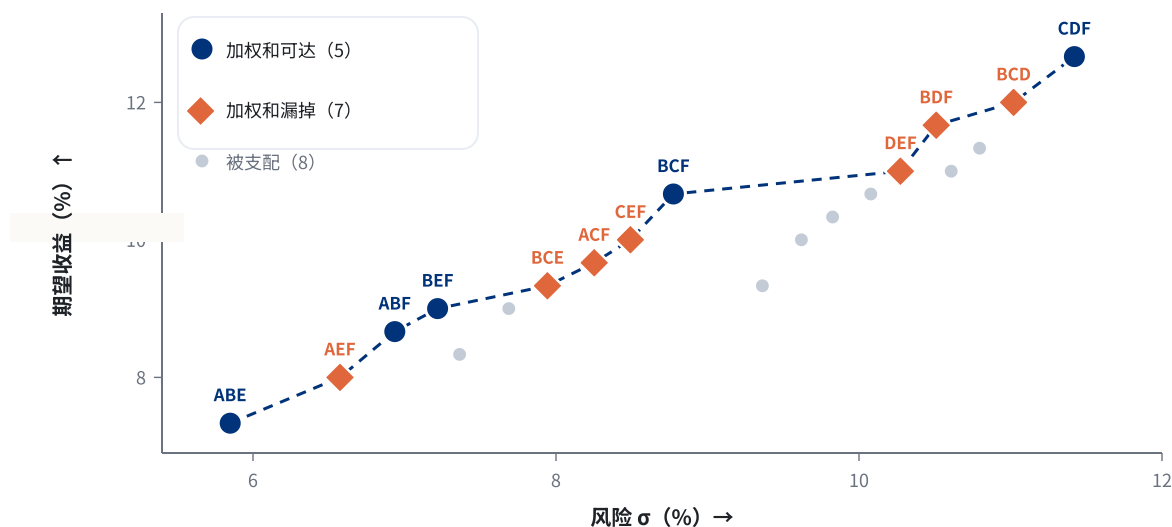


权重向量决定“折中点在哪”；极端权重偏向单一目标，均衡权重取中间。

案例：基数约束下的组合前沿（加权和会漏解）

六只资产里恰好选三只，同时最大化收益、最小化风险。枚举全部 20 个组合后，有 12 个是非支配的。

- 加权和只能找到其中 5 个（蓝色圆点）——它们落在前沿的凸包上。
- 另外 7 个（琥珀菱形）是非支配的，却没有任何权重能取到：基数约束让可行域不再连续。
- 这正是要用 ε -约束法或 NSGA-II 的原因：它们不要求前沿是凸的。



前沿一旦因为整数/基数约束而变得离散、非凸，

CHAPTER 14

目标规划

当多个“都希望达到”的软目标彼此冲突，目标规划用偏差量把它们量化成可求解的账本。

目标与约束之别

普通优化里“必须”的是约束、“想最优”的是目标。目标规划把许多“希望达到”的软目标也写成可 violated 的目标，用偏差量衡量不满。

适合“几个目标都重要、谁也不愿彻底牺牲”的管理型问题，而非纯物理最优问题。

偏差变量

对每个目标 $f_i(x) \diamond g_i$ ，引入非负偏差 d_i^+, d_i^- 表示“超过/不足”，目标变为让这些偏差尽量小。

把每个“希望达到的目标”想成尺子上的目标刻度 g_i 。实际做到的是 f_i ，可能比刻度高（超出）也可能低（不足）。偏差变量就是两把“读数表”： d_i^+ 记超出多少， d_i^- 记不足多少。目标规划不再强求“绝对达标”，而是把所有目标的“不满程度”加起来尽量压小。

$d_i^+, d_i^- \geq 0$ ，二者不能同时为正。

例子：目标 1 利润 ≥ 100 ($g_1 = 100$)，实际 115，则 $d_1^- = 0, d_1^+ = 15$ (超 15)；目标 2 排放 ≤ 50 ($g_2 = 50$)，实际 58，则 $d_2^- = 8, d_2^+ = 0$ (超排 8)。目标规划让这两类偏差尽量小——赚钱与环保谁更重要，由权重或优先级决定。

优先级与权重

目标间有轻重。两类处理：优先级（高优先级未达标前不考虑低优先级）或权重（同层级加权求和）。

- 优先级法：Lexicographic，先最小化第一优先的总偏差。
- 权重法： $\sum_i w_i d_i^+ + \sum_i w_i d_i^-$ 。

一个目标规划示例

码头排班：目标① 总成本 $\leq$ 预算、目标② 工人加班 $\leq 10\text{h}$ 、目标③ 设备利用率 $\geq 80\%$ 。三者冲突时，按优先级让最不可妥协者先满足。

结果不是“唯一最优排班”，而是一份“各目标偏离多少”的清晰账，便于管理者拍板。

目标规划的局限

它给出“相对目标达成度好”的解，但不保证帕累托最优；目标与优先级的选择本身很主观。

建议：用目标规划做“可解释的方案生成”，再用多目标前沿（第 13 章）做最终权衡。

从优化到满意：偏差变量

每个目标 g_i 都转化为用欠达标/超达标衡量的软约束：

$$f_i \leq \bar{f}_i + d_i^- \quad f_i \geq \bar{f}_i - d_i^+$$

d_i^- means $f_i \leq \bar{f}_i$; d_i^+ means $f_i \geq \bar{f}_i$. GP minimizes a function of the deviations, not the objectives directly.

三种聚合方式

Weighted

$$w_i^- d_i^- + w_i^+ d_i^+$$

simple, but weights hard to set.

Lexicographic

$$d_1, d_2$$

priority classes: safety before cost.

Chebyshev

$$t_i \frac{w_i^- d_i^- + w_i^+ d_i^+}{t_i}$$

fair: pulls the worst goal up.

目标规划 vs 多目标

Aspect	Multi-objective	Goal programming
Returns	the whole front	one closest-to-goals point
Preference	<i>a posteriori</i> (after the front)	<i>a priori</i> (targets first)
Constraints	hard only	hard + soft goals
Fit	explore trade-offs	meet thresholds (budget, schedule)

从"最优"到"满意": 偏差与目标分层

目标规划不追求把某个指标做到极致，而是把"离目标有多远"最小化。

偏差变量与目标约束

定义：每个目标写成软约束，用正、负偏差变量分别度量"超出"与"未达"。

$$f_i(x) + d_i^- - d_i^+ = b_i, \quad d_i^-, d_i^+ \geq 0, \quad d_i^- d_i^+ = 0$$

适用范畴：目标更像"期望达到的水平"而不是硬性约束时。

特点：把原本可能无解的目标变成可解；偏差量本身就是可以直接汇报给管理层的信息。

例子：目标「产量 ≥ 1000 」，实际生产 900，则负偏差 $d^- = 100$ （未达标）、正偏差 $d^+ = 0$ ；目标规划即最小化 d^- 。

加权目标规划

定义：把各目标的偏差按权重加总后最小化，允许不同目标之间相互折算。

$$\min \sum_i (w_i^- d_i^- + w_i^+ d_i^+)$$

适用范畴：目标之间可以互相折算（同量纲，或已做过归一化）。

特点：形式上与多目标的加权和一致，因此继承同样的缺点：未归一化的量纲会让数值大的目标自动主导。

例子：目标①成本 ≤ 100 、目标②加班 $\leq 10h$ 。偏差 d_1^-, d_2^- 分别度量未达标量；最小化 $5d_1^- + 2d_2^-$ 表示“成本超标更严重”。权重允许两目标折算成一个数。

优先级（字典序）目标规划

定义：目标分层：先最小化最高优先级的偏差，再在不劣化高层的前提下依次优化低层。

$$\text{lex}\backslash, \min [h_1(d), h_2(d), \dots, h_L(d)], \quad h_\ell(d) = \sum_{i \in \mathcal{P}_\ell} (w_i^- d_i^- + w_i^+ d_i^+)$$

适用范畴：存在明确的刚性次序：先保交付，再控加班，最后压库存。

特点：不需要在目标之间定换算率；但低优先级几乎不起作用，且求解必须逐层进行（或用足够大的权重近似）。

例子：优先级 P1=安全达标（不可妥协）、P2=成本。先最小化 $d_{\text{安全}}^-$ ，再在不恶化它的前提下最小化 $d_{\text{成本}}^-$ 。这样安全永远优先，不像加权那样可能被权重“平均掉”。

均衡偏差，以及与多目标的分工

加权和追求"总偏差最小"，Chebyshev 追求"最差的那个不要太差"——两者给出的方案常常不同。

Chebyshev (min-max) 目标规划

定义：最小化最大的归一化偏差，追求"最均衡"而非"总和最小"。

$$\min \lambda \quad \text{s.t.} \quad w_i^- d_i^- + w_i^+ d_i^+ \leq \lambda \quad \forall i$$

适用范畴：公平性、负载均衡、服务水平一致性——不希望某一个环节特别差。

特点：结果更"平均"，但可能牺牲总效率；常与加权和组合成增广 Chebyshev，以免得到弱帕累托解。

例子：三个归一化偏差为 0.1、0.4、0.2。Chebyshev 最小化最大值，得 $\lambda = 0.4$ 并压低最差那一项；若把 0.4 那项改善到 0.25，则 λ 降到 0.25——追求全局最均衡。

目标规划 vs 多目标优化

定义：多目标保持向量形式、给出整个前沿（描述性）；目标规划把目标退为约束、直接给出一个可执行的解（规定性）。

$$\text{MO: } \min_{x \in X} (f_1(x), \dots, f_p(x)) \quad \text{GP: } \min h(d) \quad \text{s.t.} \quad f_i(x) + d_i^- - d_i^+ = b_i$$

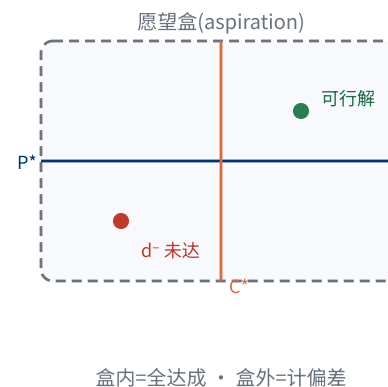
适用范畴：需要向管理层交付"一个方案"时用 GP；需要理解权衡结构、或还要继续谈判时用 MO。

特点：GP 的解落在目标的邻域内，天然用最优性换取可行性；代价是目标值本身不再被推到极致——这正是"满意化"的含义。

例子：同是“成本 vs 排放”。MO 给出整条前沿（多个解供选）；GP 把排放设为约束 ≤ 50 后只交回一个可执行解。MO 是“描述”，GP 是“规定”。

案例：容量扩张的目标规划

- 目标不是硬约束，而是带期望值的 aspiration：成本 $\leq C^*$ 、产能 $\geq P^*$ 。
- 引入偏差变量 d^-/d^+ ：未达/超过各自计分。
- 优先级：先满足产能（高优），再压成本（次优）。



把硬约束软化成“愿望”，用偏差变量量化与愿望的距离，再按优先级逐层满足。

案例：码头日计划（多目标 → GP）

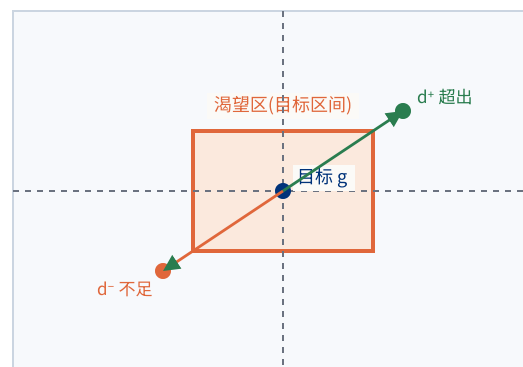
- 日计划同时想要：**高吞吐、低延误、少移箱**。
- **多目标**给出一串 Pareto 点，难直接拍板。
- **目标规划**把它们变成带优先级的愿望，输出一个可执行的折中计划。



多目标给出“所有可能”，目标规划从中挑出“一个能落地”的计划，并说清每处偏差的代价。

渴望盒与偏差变量 d^+ / d^-

- 目标规划不追求“最优”，而让各目标尽量贴近**渴望值**。
- 引入偏差变量： d^+ （超出目标）、 d^- （未达目标），并最小化其加权和。
- 图中落在渴望区外的解产生偏差，优化即把偏差压到最小。
- 适合多目标难以归一、且各目标有“可接受区间”的工程场景。



目标规划：最小化偏差 d^+/d^-

目标规划把“多目标”转成“达标偏差最小化”，更接近决策者的真实诉求。

案例：优先级目标规划（先保交付，再谈成本）

正常班最多 800 件（10 元/件），加班最多 200 件（16 元/件），需求 700 件。三个目标按优先级排列，逐层满足。

- P1 产量 ≥ 1000 ：达成（偏差 0），为此必须开满加班。
- P2 加班 ≤ 10 小时：实为 20 小时，偏差 +10 —— 优先级低一级，只能认。
- P3 库存 ≤ 200 ：实为 300，偏差 +100。



若只求成本最小，解是产量 700、成本 7,000 —— 交付目标完全没进模型。目标规划用 11,200 元换来 1,000 件的产量，多付的 4,200 元就是

CHAPTER 15

多阶段

决策随时间递推、未来不确定。动态规划给出优雅原理，却常撞上维度灾难——于是有了近似动态规划。

时序决策的本质

多阶段问题里，今天的决策影响明天的可行集与收益。不能只顾眼前，要“向后看约束、向前看收益”。

典型：库存补货、设备维护、码头逐船作业——决策随时间递推，且未来不确定。

动态规划原理

若问题有“无后效性”（未来只取决于当前状态，不取决于如何到达），可用贝尔曼最优性原理把大问题拆成阶段子问题。

贝尔曼原理一句话：“从今天往后的最优决策，只取决于你‘现在在哪儿’（状态 s ），而不取决于你‘怎么走过来的’。”所以你不必背下整条历史，只要为每个状态 s 存一个“从这儿往后最好的总代价” J_t 。这把指数级的全局搜索，压成每个状态一次的小计算。

贝尔曼方程： $J_t = \min_a \{ C(a, s_t) + J_{t+1}(s_{t+1}) \}$ 。每个状态只需记“从这儿往后最优值”。

例子：从 A 到 D 的最短路。站在 B 时，“B 之后怎么走最省”只取决于你在 B，不取决于你是从 A 直奔 B 还是绕了远路。若 $B \rightarrow C \rightarrow D$ 代价 5，则 $J_B = 5 + J_C$ ，与来路无关。每个点只算一次自己的 J ，全程最短路就出来了。

状态与转移

把世界压缩成“状态” s ，决策 a 触发“转移” $s \rightarrow s'$ ，并付出代价 c 。状态选得好坏决定能否套用 DP。

- 状态太少：遗漏关键记忆，决策短视。
- 状态太多：维度灾难，算不动。

维度灾难

状态有 d 个维度、每维 k 个取值，状态数就是 k^d ，指数爆炸。这是 DP 落地的最大障碍。

对策：要么简化状态（聚合、截断），要么放弃精确 DP，改用近似/启发式。

近似动态规划

当精确 DP 不可行，用函数近似（线性/神经网络）来估计价值函数 $V(s)$ ，在采样上迭代学习。

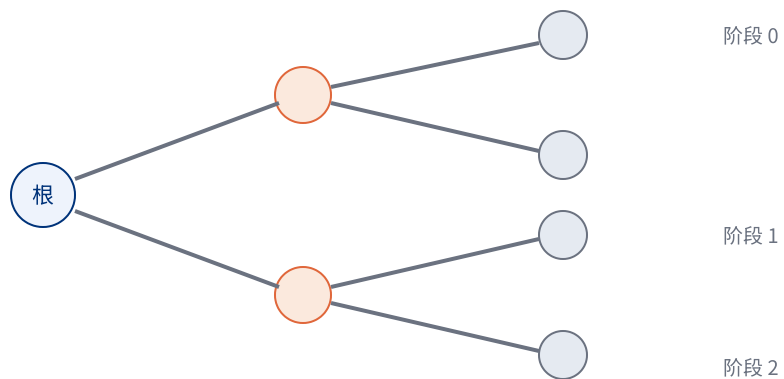
- 值函数近似：用参数化函数替查表。
- 时序差分 / 策略梯度：与强化学习同源。
- 代价：只剩近似最优，需验证泛化。

两阶段随机规划

$$x^T \quad \xi \quad y^T$$

第一阶段决策 x 是此时此地 (here-and-now)；第二阶段决策 $y(\xi)$ 是观望后的追索 (wait-and-see)。鲁棒优化问“能否扛住最坏情况”；两阶段随机优化问“平均修复成本是否可承受”。

情景树与非预期约束



Decisions at nodes sharing a history must coincide (non-anticipativity). Size grows exponentially with stages.

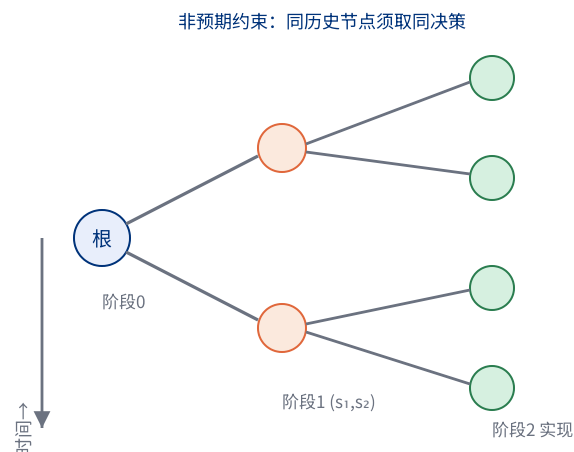
与动态规划的联系：贝尔曼

$$V_t^*(x_t) = \max_{a_t} \{ R_t + \gamma V_{t+1}^*(S_{t+1}|S_t, a_t) \}$$

- DP avoids enumerating the whole tree but needs (or approximates) the value function.
- Approximate DP / RL handle the **curse of dimensionality** on large instances.

情景树与多阶段随机规划

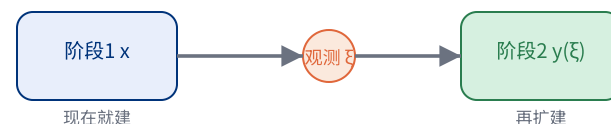
- 不确定随时间逐步揭示，单次的“现在就定”很浪费——可以等、看、再动。
- 多阶段把决策分成阶段，引入 **recourse**（**recourse**）：先看结果，再修补。
- 信息结构最自然画成**情景树**：根是当下，节点是各阶段世界状态，边是可能实现。
- **非预期约束**：相同历史下的节点必须取相同决策。



阶段与情景都是“设计选择”：何时承诺，往往比承诺什么更重要。

两阶段 recourse: 现在建, 事后补

- 两阶段模型: 先选第一阶段的 x (现在就定), 观测随机结果 ξ , 再选第二阶段 recourse $y(\xi)$ 修补。
- 目标 $\min c^T x + E[\min q^T y(\xi)]$: 对 all 情景取 recourse 代价的期望。
- 与鲁棒不同: 用分布/情景集而非最坏集, 且允许 “事后花代价补救”。
- 多阶段 $\Leftrightarrow$ 动态规划: 用 Bellman 递归向后归纳求解 (大规模时用近似 DP/RL)。



$$\min c^T x + E[\min q^T y(\xi)]$$

例: 电厂/仓库容量——现在建多少, 需求揭晓后再补

鲁棒问 “最坏能否扛住”; 两阶段问 “平均代价+事后能否补救”

多阶段 $\Leftrightarrow$ 动态规划 (Bellman 递归)

最佳策略往往不是一张固定时间表, 而是一条 “对将来实际情形反应” 的规则。

序贯决策：动态规划与两阶段随机

多阶段问题的共同结构是"今天决定一点，明天看到新信息再补一点"。

动态规划与贝尔曼方程

定义：把多阶段决策拆成"每阶段只看当前状态"，用值函数递归，避免一次性枚举所有策略。

$$V_t(s) = \min_{a \in A(s)} \{ c_t(s, a) + E[V_{t+1}(s') | s, a] \}, \quad V_T(s) = \text{terminal cost}$$

适用范畴：状态可紧凑描述且满足无后效性：库存、设备更换、水库与储能调度、路径选择。

特点：把指数级枚举压成"状态 × 阶段"的查表；代价是维度灾难——每多一个状态维度，状态数就翻一倍。

例子：最短路中，从节点 i 到终点的最小代价 $V(i) = \min_{j \in \text{邻居}(i)} \{ c_{ij} + V(j) \}$ ，把「全程最短」拆成「当前一步 + 余下最短」，即 Bellman 最优性原理。

两阶段随机规划

定义：第一阶段 here-and-now 决策在看到实现前做出，第二阶段 recourse 决策在观测之后做出，优化期望总成本。

$$\min c^T x + \sum_{\omega} p_{\omega} q^T y_{\omega} \quad \text{s.t.} \quad Ax = b, \quad T_{\omega}x + W y_{\omega} = h_{\omega}, \quad x, y_{\omega} \geq 0$$

适用范畴：产能扩张、选址、采购、机组组合——"先定大方向、后补细节"的决策。

特点：非预期性由" x 不随 ω 变化"表达，这是建模中最关键的一条约束；大规模实例用 L-shaped / Benders 分解。

例子：第一阶段在需求揭晓前决定建厂规模 x ，第二阶段按实际 ξ 调整生产 y_{ω} 。目标最小化 $4x + E_{\omega}[2y_{\omega}]$ ，约束 $x \geq \xi_{\omega}$ （产能须覆盖需求）。here-and-now 与 wait-and-see 的权衡是核心。

情景树与近似动态规划

当阶段变多、状态变连续，精确方法就走不动了——要么离散化成情景树，要么近似值函数。

多阶段随机规划与情景树

定义：把随机过程离散成一棵情景树，树的每个节点就是“信息揭示的时刻”。

$$\min \sum_{n \in \mathcal{N}} p_n c_n^T x_n \quad \text{s.t.} \quad x_n = x_{a(n)} \quad (\text{同一信息集内决策必须相同})$$

适用范畴：需要随信息逐步揭示而反复调整的多期决策：资产配置、能源调度、产能爬坡。

特点：树规模随期数指数增长，必须做情景缩减；“同一父节点下的决策必须相同”就是非预期性——决策不能偷看未来。

例子：12 个月用电规划。情景树根=当前，第 1 层节点=“夏季偏热/正常”，第 2 层再分“电价高/低”；同一节点下的决策必须一致（非预期约束），不同分支可不同——把“随时间揭示的信息”显式建模。

近似动态规划 ADP

定义：用参数化函数（线性基、神经网络、查表加插值）替代精确值函数，并借助仿真向前推进。

$$\hat{V}(s; \theta) \approx V(s), \quad \theta \leftarrow \theta - \alpha \nabla_{\theta} (\hat{V}(s; \theta) - \text{target})^2$$

适用范畴：状态空间过大（连续、多维），或转移模型只能仿真、写不出解析式。

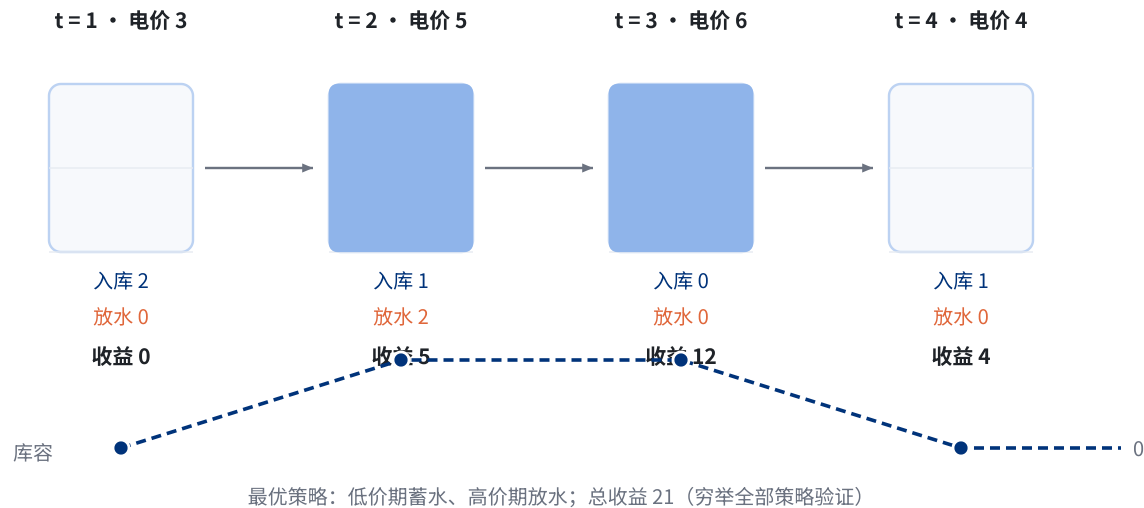
特点：突破维度诅咒，但失去最优性保证；效果强烈依赖基函数选取与探索策略，调参经验成分大。

例子：大型水库调度，状态=水位×来水×需求，维度爆炸无法穷举。用线性回归 $\hat{V}(s; \theta) = \theta_0 + \theta_1 \text{水位} + \theta_2 \text{来水}$ 近似值函数，借仿真滚动更新 θ ，得到“够用”的近优策略。

案例：水库调度的动态规划

四个时段，库容 2 单位，入库径流已知，电价随时段变化。状态是

- 最优策略：t=1 电价最低（3）时一滴不放，把水存起来。
- t=3 电价最高（6）时把两单位全部放掉，收益 12。
- 四期总收益 21（穷举全部策略校验）；维数一旦上升，就要靠近似动态规划。



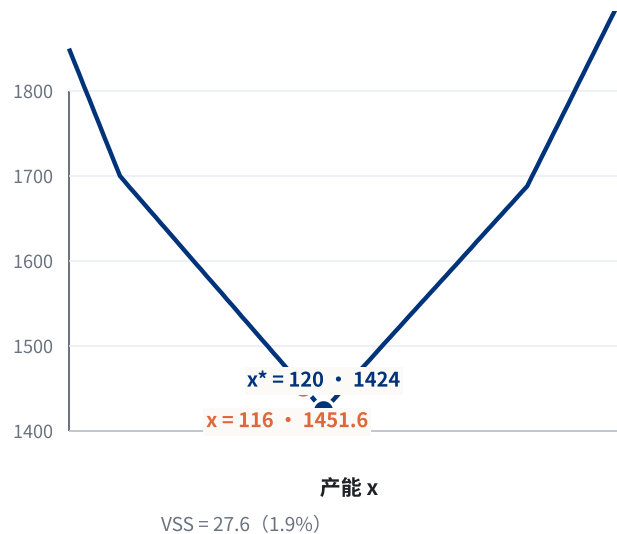
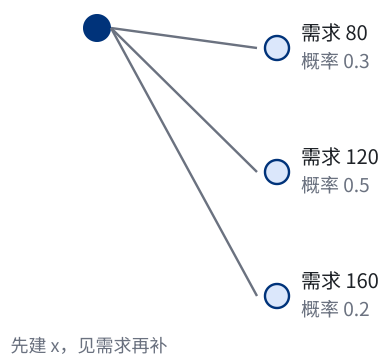
动态规划把

案例：产能扩张的两阶段随机规划

现在决定产能 x (10/单位)，需求有三种情景：低 80 (0.3)、中 120 (0.5)、高 160 (0.2)。不够就外协 (25/单位)，多了就闲置 (2/单位)。

- 随机最优 $x^* = 120$ ，期望成本 1,424。
- 用期望需求 116 当真值去解，再把 116 放进随机环境：期望成本 1,451.6。
- 差额 $VSS = 27.6$ (1.9%)：这就是

第一阶段：建产能 x



情景树把

CHAPTER 16

竞争博弈

当对手也会因你而变，问题从“优化”变成“博弈”。纳什均衡描述互相盯防下的稳态，却未必是社会最优。

博弈 vs 优化

优化假设“世界被动接受你的决策”；博弈里对手也会因你而变。目标不再由你独享，而是多方互动的结果。

一句话：优化找“给定环境最好”；博弈找“互相盯防下的稳态”。

纳什均衡

纳什均衡是一组策略，任何一方单独偏离都不会更好——谁都不想先动，于是稳住。这是博弈的“解”概念。

形式：对每方 i ， $u_i(s_i^*, s_{-i}^*) \geq u_i(s_i, s_{-i}^*)$ 对任意 s_i 成立。

均衡如何求解

双矩阵博弈可用顶点枚举/ Lemke–Howson；大规模则靠迭代算法或学习动力学逼近。

- 最佳响应迭代：双方轮流对对方取最优反应，常收敛到均衡。
- regret 最小化：以无遗憾学习逼近均衡策略。

寡头竞争示例

两家码头争夺同一腹地货源，各自定运价。降价抢量但利润变薄；不降又怕被抢。纳什均衡给出双方都不愿先动的价位。

启示：均衡未必“社会最优”，常存在双方都更赚的“合谋价”，但那不稳定且常违规。

博弈论的局限

均衡可能多个、可能不存在纯策略均衡、且假设“对手完全理性”。现实里人会误算、会情绪化。

工程用法：把博弈当“情景推演”工具，看清对手反应与脆弱点，而非当精确预测器。

博弈要素与纳什均衡

博弈：参与者 i ，策略 S_i ，收益 $u_i(s_1, \cdots, s_N)$ 。没有社会规划者——只有自利。

$$u_i(s_i^*, s_{-i}^*) \geq u_i(s_i, s_{-i}^*) \quad \forall s_i \in S_i$$

At a Nash equilibrium no player benefits from unilateral deviation. Existence needs compact convex S_i and continuous quasi-concave payoffs (Nash's theorem).

古诺 vs 伯特兰

Cournot (quantity)

- Firms choose q_1, q_2 ; price p is determined.
- NE: q_1^*, q_2^* .
- Output above monopoly, below competition.

Bertrand (price)

- Firms choose prices; cheaper serves all.
- NE: p_1^*, p_2^* (even with two firms!).
- The strategic variable reshapes the result.

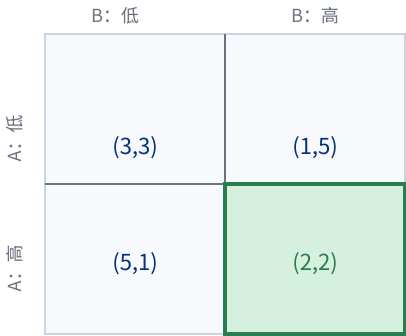
博弈即变分不等式

$$i \quad i^* \quad x_i \quad i^* \quad i \quad i$$

- Collecting all players gives a VI with $x_1 = 1$ — optimization is the special case of one shared objective.
- Congestion games admit a potential function (Rosenthal); steering its minimum reshapes the equilibrium.

收益矩阵与 Nash 均衡

- 非合作博弈三要素：玩家、策略集、收益函数 u_i (全体策略)。
- **Nash 均衡**：无玩家能通过单方偏离改善自身收益。
- 图中 (2,2) 是均衡——两家都提价则落入 (3,3) 但谁都不敢单独提价。
- 工程师常是**规则制定者**：设计让均衡变好的机制，而非自己下场博弈。

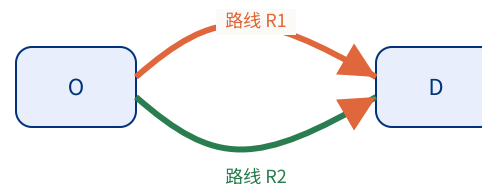


Nash 均衡=(2,2)：任一方单独提价都无利

竞争下该问的不是“什么最优”，而是“什么稳定”。

拥堵博弈与 Wardrop 均衡

- 工程中遍布拥堵博弈：每个主体选资源（路线、频段、服务器），共享者越多质量越差。
- **Wardrop 用户均衡**：无人能通过换路降低自身行程时间——正是拥堵博弈的 Nash。
- 实例：无线频谱共享、云/数据中心负载均衡、电力市场出清。
- 若博弈有势函数（Rosenthal 定理），可改“最小化势函数”把博弈变回优化问题。



拥堵：越多人走→时间越长

司机不断换路，直到 $T_1 = T_2$

Wardrop 用户均衡 = 拥堵博弈的 Nash

均衡可借助势函数“转译”为优化问题——博弈论由此回到本书的优化工具箱。

博弈的基本语言与三种寡头结构

博弈与优化的区别在于：你的结果不只取决于你怎么选，还取决于别人怎么选。

标准型博弈与纳什均衡

定义：由玩家集 N 、策略集 S_i 与收益函数 u_i 组成；均衡是无人能通过单方面改变策略而获益的策略组合。

$$u_i(s_i^*, s_{-i}^*) \geq u_i(s_i, s_{-i}^*) \quad \forall i \in N, \forall s_i \in S_i$$

适用范畴：参与方目标冲突且结果相互依赖：定价、竞标、选址竞争、共享资源。

特点：有限博弈在混合策略下必有均衡（Nash 1950）；但可能多重、也可能帕累托无效（囚徒困境）；一般和博弈的均衡计算是 PPAD-完全的。

例子：两厂商同时定产，收益矩阵中若（低产,低产）互为对方最优反应，则该组合即纳什均衡——谁单独改产都不会更赚。

古诺 / 伯特兰德 / 斯塔克尔伯格

定义：三种经典寡头结构：拼产量、拼价格、拼先后。线性需求 $p = a - bq$ 、边际成本 c 下有闭式解。

$$q_i^C = \frac{a - c}{3b}, \quad p^B = c, \quad q_1^S = \frac{a - c}{2b}, \quad q_2^S = \frac{a - c}{4b}$$

适用范畴：产能与产量竞争（古诺）、同质产品价格战（伯特兰德）、存在先动者或可信承诺（斯塔克尔伯格）。

特点：均衡结果高度依赖“哪个变量是战略变量”——建模时选错竞争变量，算得再准也是在回答另一个问题。

大规模博弈与均衡的统一表达

当玩家多到"单个玩家不影响整体"，均衡就可以写成凸优化；更一般的均衡则统一到变分不等式。

拥堵博弈与 Wardrop 均衡

定义：大量"原子级"用户各自选路；均衡时被使用路径的出行时间相等，且不大于任何未使用路径。它等价于一个凸规划的最优性条件。

$$c_r(x^*) = \min_r c_r(x^*) \quad \forall : x_r^* > 0, \quad \Leftrightarrow \quad \min_x \sum_e \int_0^{x_e} t_e(z) dz \quad \text{s.t. 需求守恒}$$

适用范畴：交通配流、通信网络路由、共享资源排队与拥塞定价。

特点：有势函数，故均衡存在且在严格 monotone 假设下唯一，并可有效计算；但与系统最优不一致，差距用无序代价 PoA 度量，加边甚至可能变差（布雷斯悖论）。

变分不等式 VI

定义：找一点，使算子 在该点与可行集内所有点之间的内积非负。当 时退化为一个优化问题。

$$\text{find } x^* \in K : \quad (x - x^*)^\top F(x^*) \geq 0 \quad \forall x \in K$$

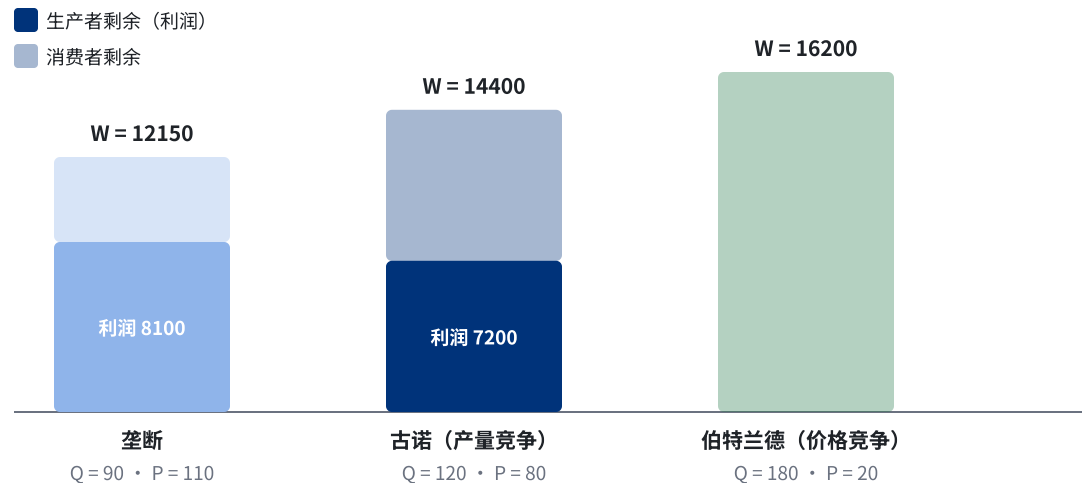
适用范畴：交通、能源与供应链网络均衡、多市场均衡，以及纳什均衡的统一刻画。

特点：比"各自求优化再联立"更通用——不需要每个参与方都有可求导的目标；F 单调时解集为凸且算法收敛，非单调时则很困难。

案例：古诺 vs 伯特兰德（换一个决策变量，结果天差地别）

同样的线性需求 $P = 200 - Q$ ，同样的成本 $c = 20$ ，只把竞争变量从

- 垄断： $Q = 90$ ， $P = 110$ ，利润 8,100。
- 古诺（拼产量）： $q_1 = q_2 = 60$ ， $Q = 120$ ， $P = 80$ ，各赚 3,600。
- 伯特兰德（拼价格）：价格一路压到边际成本 $P = 20$ ，利润归零，消费者剩余 16,200。



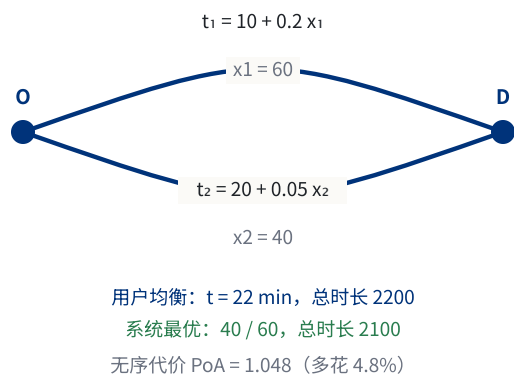
结论取决于

案例：拥堵博弈与布雷斯悖论

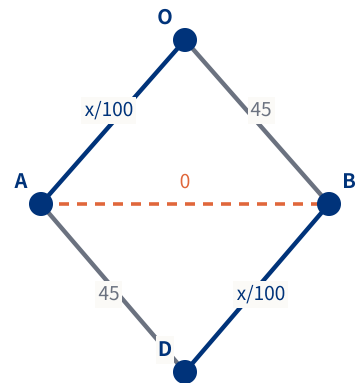
每个人都走对自己最快的路，结果是所有人都不快。左边是两条平行路，右边是著名的四节点网络。

- 用户均衡：两条路时间相等（22 min），流量 60 / 40，总时长 2,200。
- 系统最优：40 / 60，总时长 2,100，无序代价 PoA = 1.048。
- 布雷斯悖论：新增一条零成本捷径后，出行时间从 65 min 涨到 80 min（+23%）。

平行路径：用户均衡 vs 系统最优



布雷斯悖论：新增一条免费捷径



无捷径：2000 / 2000 → 65 min
 有捷径：全体 4000 拥入 → 80 min（+23%）
 新增一条零成本道路，反而让所有人更慢

在拥堵网络里，

第四部分

工程优化方法

问题认清了，该选武器。本篇纵览方法谱系：从经典数学规划，到仿真驱动，再到学习驱动。

本篇逻辑框架

问题认清了，该选武器。本篇纵览方法谱系：从经典数学规划，到仿真驱动，再到学习驱动，呈现从解析到数据的连续光谱。



逻辑主线：概述 → 经典方法 → 仿真驱动 → 学习驱动，沿“解析 → 数据”光谱逐步展开。

CHAPTER 17

概述

方法按“精确—近似”“梯度—无梯度”“确定—随机”多轴排成谱系；选方法先读问题结构。

方法谱系

方法可按多条轴排列成谱系。没有“最好”，只有“最适合当前问题结构”的那一支。

- 横轴：精确 → 近似 → 启发式。
- 纵轴：解析梯度 → 数值梯度 → 无梯度。
- 深度：一次求解 → 迭代学习。

精确 vs 近似

精确法（分支定界、内点）能证最优或给证书，但规模受限；近似/启发式快速度，只给可行或近优。

用精确：问题小、或“最优”是合同/安全硬要求。

用近似：规模大、实时、或只需“够好”。

梯度 vs 无梯度

若目标光滑且梯度可得，梯度法（最速下降、牛顿、拟牛顿）效率极高；若目标黑箱或不可导，只能无梯度搜索。

现实常是“混合”：用代理模型把黑箱变光滑，再上梯度法。

确定性 vs 随机

确定性算法每步可复现；随机算法（SA、遗传、SPSA）用随机性跳出局部，结果带随机性但更易逃谷。

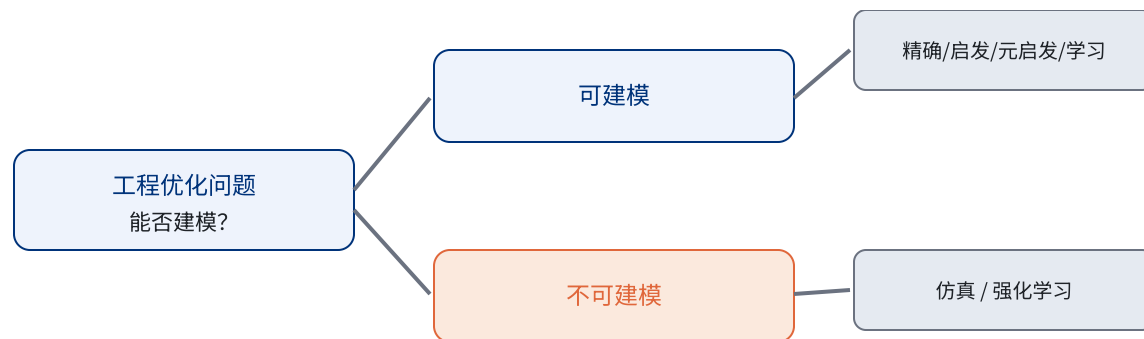
工程取舍：要可复现审计用确定性；要 robustness 探索用随机并多次运行取统计。

选择指南

一张经验性决策树：先看结构，再看规模与时效，最后看可解释性要求。

- 连续+凸 → 梯度/内点，快且优。
- 整数/组合 → MIP 精确，或元启发式兜底。
- 黑箱/昂贵仿真 → 贝叶斯优化。
- 多阶段+不确定 → 近似 DP / 强化学习。

路线图：可否建模



The top split is *modelability*; real problems are hybrids that combine a model with simulation.

方法谱系：简→繁



Start at the simplest rung that still solves your problem; climb only when forced.

决策表：问题类→方法

Problem class	First choice	Modern option
Combinatorial	exact (small), greedy + local	LNS matheuristic; GNN
Multi-objective	weighted sum + exact	NSGA-style meta
Robust	exact + enumeration	scenario sampling; robust RL
Multi-stage	DP / exact	fix-and-relax; RL policy
Bilevel	exact lower level	nested matheuristic
Game	analytical equilibrium	agent-based; multi-agent RL

不同方法的工程适用场景

选方法不是看谁"高级", 而是看问题结构、规模、是否有解析模型、是否在线。下面把每种方法对到具体的工程场合。

Method family	Reach for it when...	Typical engineering scenario
Exact MILP / simplex	size manageable; you need a certificate or audit trail	berth allocation (nightly), feed/blend, network design under $\sim 10^4$ variables
Heuristic / metaheuristic	NP-hard, large, tight deadline, no closed form	vehicle routing, job-shop rescheduling, cutting & packing
Matheuristic	MIP too big but structure is exploitable	rich VRP with side constraints, yard-block allocation, Benders subproblems
Simulation-based optimisation	objective only via a simulator; stochastic & noisy	terminal-layout evaluation, warehouse throughput, traffic micro-simulation
Bayesian optimisation	very expensive black box, only tens-hundreds of evals	solver hyper-parameter tuning, process / material design
RL / learning-based	sequential & online; stable instance family	real-time AGV dispatch, adaptive crane sequencing, congestion control
First-order / ML optimisation	huge scale; 10^{-4} accuracy is enough	model training, large convex programs, cloud-scale logistics

经验法则

先问三件事：① 有可微/可建模的目标吗？② 规模与时限？③ 要证明还是只要好解？精确法给证明但怕规模，启发式/仿真/学习换速度但要实证——按这三问选，而不是按流行度。

三类方法：精确、近似、启发式

先分清"有没有保证"。有保证的方法给你可复核的结论，没有保证的方法给你可用的解——两者都要，但不能互相冒充。

精确算法

定义：在有限步内返回一个可证明的最优解，并同时给出最优性证明（间隙为 0）。

$$x^* \in \arg \min_{x \in \mathcal{F}} f(x), \quad \text{gap} = \frac{f(x^*) - f^*}{|f^*|} = 0$$

适用范畴：线性规划、凸问题、网络流，以及规模可控的混合整数规划（分支定界、割平面）。

特点：结论可信、可复核、可交付审计；最坏情形指数时间，规模一大就可能"算不完"。

启发式

定义：利用问题结构快速构造或改进可行解的经验规则，**不提供**最优性或近似比保证。

$$x = H(I) \quad \text{— 一个规则映射，无 } \alpha \text{ 保证}$$

适用范畴：时间紧、实例大、或目标难以建模；也常用来给精确方法提供初始解（热启动）。

特点：快、易实现、易向现场解释；质量靠实证而非证明，换一批实例可能明显退化。

One spectrum: guarantee decreases as speed increases



方法谱系：从精确（强保证）到启发式（快但无证明），中间是近似（有近似比）。

工程对照

岸桥调度（强 NP 难）：夜间计划（数百项任务）用分支定界求可证明最优；日间重排（数千变量）用遗传/禁忌搜索求快速近优。两者并存，互不替代。

元启发式、数学启发式与超启发式

三者都在"搜索"，但搜索的对象不同：元启发式搜索**解**，超启发式搜索**启发式**，数学启发式把两者与精确求解器缝合起来。

元启发式

定义：与具体问题无关的高层搜索框架，通过在解空间上平衡"探索"与"开发"来迭代改进解。

$$x^{(t+1)} = \text{Modify}(x^{(t)}, N(x^{(t)}), \theta)$$

适用范畴：组合与连续黑箱、目标不可导或不连续、规模从数千到数百万变量；模拟退火、禁忌搜索、遗传算法、粒子群。

特点：不需要梯度与凸性；参数敏感，需调参与多种子。单点法（SA、禁忌）与种群法（GA、PSO）两大类。

数学启发式

定义：把数学规划与（元）启发式组合：用精确方法解子问题，用启发式决定搜索方向或提供初始解。

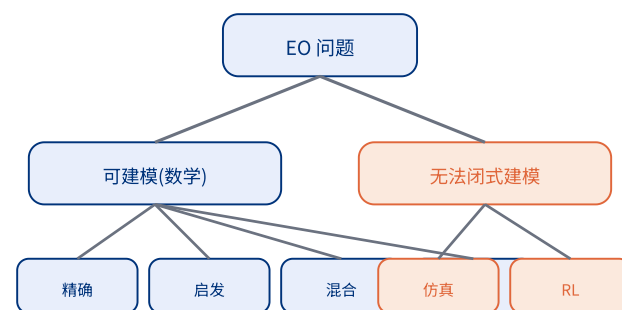
$$\min \{c^T x : x \in F \cap \text{Fix}(S)\} \quad \text{— 固定或松弛一部分变量后精确求解}$$

适用范畴：大规模混合整数规划、带复杂侧约束的车辆路径与排产。

特点：由 LP/MIP 松弛给出下界，因此仍能量化"离最优还有多远"；实现成本高于纯启发式。

路线图：问题可建模性 → 方法族

- 选求解器前先问一个分叉问题：**这问题能写成数学模型吗？**
- **可建模** → 数学优化：精确、启发/元启发、混合启发、端到端学习。
- **无法闭式建模**（随机到达、故障、对手反应）→ 仿真或强化学习。
- 边界情形=部分可建模+部分不确定：用模型+仿真（如鲁棒/多阶段下的情景评估）组合求解。



第一步先问：“这问题能写成数学模型吗？”

边界问题=模型+仿真 组合求解

可建模性这一刀，把整套方法全景分成两 toolbox。

方法谱系：由简到繁的阶梯

- 方法不是竞争者，而是**复杂度的阶梯**：每上一级换一点保证，换更多规模/鲁棒/速度。
- 层级：精确 → 局部搜索 → 元启发 → 混合启发 → 仿真 → 端到端学习 → 强化学习。
- 简单优先**：先用精确或启发证核心可解，再升到混合启发，最后才用仿真/学习。
- 右侧正是本书前几部分“精确方法”失效之处——学习接管序贯、动态、实时问题。



每一步向上都“用能力换保证与工程投入”，别为炫技而爬梯。

CHAPTER 18

经典方法

线性、非线性、整数、凸——四大家族构成了工程优化的数学基础，也对应成熟的求解器生态。

线性规划

LP 是工程优化的工作马：运输、配比、资源分配都常化作 LP。单纯形法实用、内点法有多项式保证。

对偶：每个 LP 有对偶，给出影子价格与灵敏度，是“值不值得多投资资源”的答案。

非线性规划

NLP 允许非线性目标/约束。一阶法（梯度、共轭梯度）与二阶法（牛顿、拟牛顿）利用导数加速。

约束处理：罚函数、障碍法、增广拉格朗日，把约束“揉”进目标或迭代中。

整数规划

变量取整后问题从 P 跳到 NP-hard。分支定界是主力：不断分支、用松弛上界剪枝。

- 松弛：先解连续松弛拿到上界。
- 割平面：加有效不等式收紧松弛。
- 列生成：按需在巨量变量里动态生成。

凸优化

若目标凸、可行域凸，则任何局部最优都是全局最优——这是“可解性”的黄金地带。

凸性判据： $\nabla^2 f(x) \succeq 0$ 且约束函数凹/仿射。凸问题有高效内点法。

求解器生态

不必从零造轮子。商用/开源求解器把上述方法工程化，是工程实践的真实入口。

- 商业：Gurobi、CPLEX、MOSEK（快、稳、贵）。
- 开源：CBC、GLPK、HiGHS、SCIP（免费、可审计）。
- 选择：先问接口/规模/许可，再谈算法。

精确基础与简单启发式

Exact methods

- LP / MILP via branch-and-bound, branch-and-cut, decomposition.
- Return a provable optimum + bound; stall on large discrete problems.

Constructive & local

- Nearest-neighbor, list scheduling, greedy set cover.
- Local search: swap / 2-opt until a local optimum.
- Fast, always feasible, but myopic.

数学启发式：精确力 + 启发搜索

- **Math-first.** Solve the LP/MILP relaxation $\rightarrow$ bound + fractional solution seeds the metaheuristic.
- **RINS.** Fix variables where incumbent and relaxation agree; re-solve the tiny sub-MILP — a standard inside CPLEX/Gurobi.
- **Meta-first.** LNS destroys part of a solution; exact MILP repairs the freed decisions (routing, scheduling, airlines).
- **Net.** Near-optimal solution *with a quality certificate* on large structured problems.

元启发式与工程取舍

Families

- Trajectory: SA, tabu search.
- Population: GA, PSO, ACO.
- Multi-obj: NSGA-II elitist sorting.

Start simple, escalate only as the problem demands.

Four choosing axes

- Instance size & structure.
- Need for a certificate.
- Recurrence & real-time pressure.
- Deployability & trust.

连续优化的三台发动机：单纯形、内点、一阶

同样一个线性或连续问题，可以被三种哲学完全不同的算法求解。选哪个，取决于规模、精度要求，以及你要不要热启动。

单纯形法

定义：沿可行多面体的顶点迭代移动，每次换基使目标不劣化，直到不存在改进方向。

$$r_j = c_j - c_B^T B^{-1} A_j \geq 0 \quad \forall j \Rightarrow x^* \text{ 已找到}$$

适用范畴：线性规划与混合整数规划的 LP 松弛；需要顶点解、基信息与灵敏度分析的场所。

特点：实践极快、可热启动、天然给出对偶与影子价格；最坏情形指数时间。

一阶方法

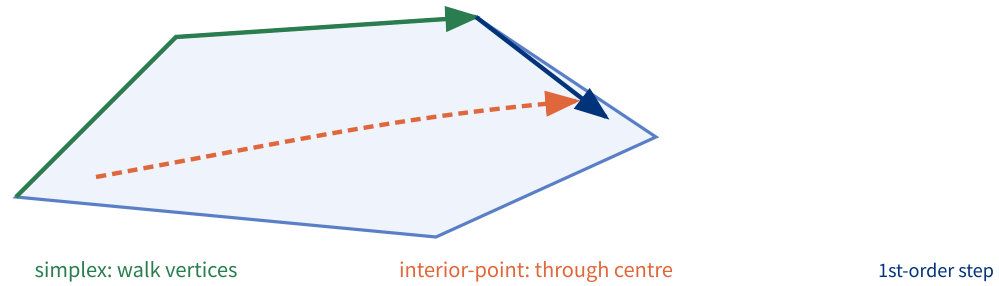
定义：只使用（次）梯度信息的迭代法，单步代价低，能处理大规模乃至非光滑问题。

$$x^{k+1} = \Pi_F(x^k - \eta_k \nabla f(x^k)), \quad f(x^k) - f^* = O(1/k)$$

适用范畴：机器学习、大规模凸问题、精度要求在 10^{-4} 量级而非 10^{-9} 的场景。

特点：单步便宜、内存小、易并行；收敛慢于二阶法，对步长与条件数敏感。

Same feasible region, three philosophies



同一可行域，三种求解哲学：单纯形沿顶点走、内点法穿心而过、一阶法只沿梯度迈小步。

工程对照

日配料用单纯形（顶点解 + 影子价格做灵敏度分析）；全国级物流平衡（ 10^5 变量）用内点法/一阶方法上云求解。同一模型，依规模换引擎。

整数规划的三件武器：分支、割、分支切割

整数性把问题推入 NP-hard。现代求解器靠"分而治之 + 收紧松弛 + 剪枝"这三件事，把指数搜索压缩到可解的规模。

分支定界

定义：把可行域递归地划分为子域，对每个子域用松弛计算下界，剪去不可能改进的子域。

$$z_{LP}(F_i) \geq \bar{z} \implies \text{剪掉子域 } F_i$$

适用范畴：混合整数规划与组合优化——凡是能构造出可快速求解的松弛的问题。

特点：给出最优性与间隙证明；最坏情形指数，强依赖松弛质量与分支策略。

割平面

定义：向松弛中逐步加入"切掉当前分数解、但保留全部整数可行点"的线性不等式，收紧可行域。

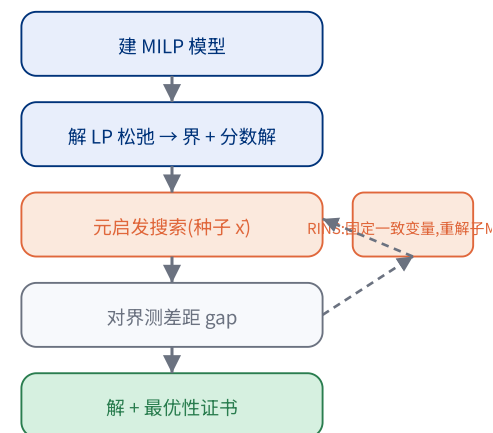
$$\alpha^T x > \beta, \quad \alpha^T x \leq \beta \quad \forall x \in F \cap \mathbb{Z}^n$$

适用范畴：混合整数规划，尤其是含背包、覆盖、流等子结构的问题（覆盖割、Gomory 割、flow cover）。

特点：显著提升下界；割太多会拖慢每次 LP 求解，需要割的筛选与管理。

混合启发(一)：先数学，后元启发

- 混合启发把精确方法与元启发结合，各取所长。
- 先数学**：建 MILP、解 LP 松弛，得到界与分数解。
- 用松弛解**热启动**元启发（种子远优于随机）；松弛界还能给“离最优多远”的证书。
- RINS**：固定 incumbent 与松弛解一致的变量，重解更小子 MILP——现代求解器(CPLEX/Gurobi)标配。



松弛同时提供“种子”与“证书”：纯元启发做不到的，混合启发兼具。

元启发两大家族：轨迹 vs 种群

- 当精确失效、局部搜索被困，**元启发**用“放弃保证”换“搜巨大空间”的能力。
- **轨迹法**：单解被扰动并按规则接受（有时容忍劣步逃局部）——模拟退火、禁忌搜索。
- **种群法**：一组解并行演化——遗传算法、粒子群、蚁群，对崎岖地形更鲁棒。
- **多目标元启发**（如 NSGA-II）维护多样的 Pareto 折中解集，供工程师事后挑选。



多目标版(如 NSGA-II)维护 Pareto 解集

元启发是大规模组合与多目标问题的默认“够好且快”工具。

CHAPTER 19

基于仿真

当目标只是一段昂贵、带噪的仿真，优化必须在“尽量少试”的约束下黑箱搜索——代理与贝叶斯优化登场。

为何需要仿真

许多系统没有闭式目标：离散事件仿真、物理求解器、数字孪生才是“目标函数”。优化只能在仿真端黑箱调用。

特征：每次评估昂贵、带噪、可能失败——这彻底改变了“该用什么优化”。

黑箱优化

只知道“喂入 x ，吐出 $f(x)$ ”，不知导数。必须在尽量少的评估里找到够好的 x 。

- 无梯度：遗传、粒子群、差分进化、Nelder–Mead。
- 噪声处理：多次重复取平均，或用鲁棒性指标。

代理模型

用少量仿真点训练一个便宜的代理（高斯过程、多项式、神经网络），在代理上密集搜索，再回流验证。

核心循环：采样 → 建代理 → 在代理上优化 → 选最不确定/最有望点再仿真。

贝叶斯优化

贝叶斯优化是代理思路的严谨实现：用高斯过程量化“不确定性”，用采集函数平衡探索与利用。

采集函数例：(上置信界)，既看均值也看不确定。

仿真案例

以岸桥调度为例：每次“给定作业顺序→跑离散事件仿真→得总完工时间”要几分钟。用贝叶斯优化约几十次评估就逼近最优顺序，远少于穷举。

要点：把“昂贵仿真”当黑箱目标，优化器负责“下一步该试哪个 x ”最值当。

三种仿真工具

- **Monte Carlo / scenario evaluation.** Draw many scenarios, average the simulated performance — turns a deterministic method into a robust or stochastic one.
- **Discrete-event simulation (DES).** Advances a clock through events (ship berths, crane finishes, truck departs); models queues, resources, interactions.
- **Simulation optimization.** Ranking-and-selection picks the best; ordinal optimization finds a good-enough solution with far fewer runs.

何时走出模型

- **Is a faithful model even available?** Random arrivals, failures, reacting agents → no clean objective exists; simulation is the only honest scorer.
- **How is the simulator coupled to search?** Ranking-and-selection or a surrogate turns raw simulation into a search procedure.
- **What is the budget / is a real system present?** DES pays off when "what-if" matters more than a provable optimum, especially with an existing digital twin.

A terminal is validated by DES *before* concrete is poured — the model-based methods cannot see crane breakdowns or truck bunching.

仿真优化与离散事件仿真

当目标只能"跑一次仿真"才能得到时,优化的对象从 $f(x)$ 变成了 $E G(x, \omega)$ ——无梯度、有噪声、且每一次观测都要花钱。

仿真优化

定义: 目标只能通过一次(昂贵、含噪的)仿真来评估时,在解空间上优化其期望表现。

$$\min_{x \in \mathcal{X}} F(x) = E_{\omega}[G(x, \omega)], \quad \text{观测到 } G(x, \omega_i) = F(x) + \varepsilon_i$$

适用范畴: 排队、生产线、港口与仓储作业、交通流——凡是写不出闭式、但能跑仿真的系统。

特点: 无梯度、有噪声、单次评估昂贵;评估预算通常只有几百到几千次。

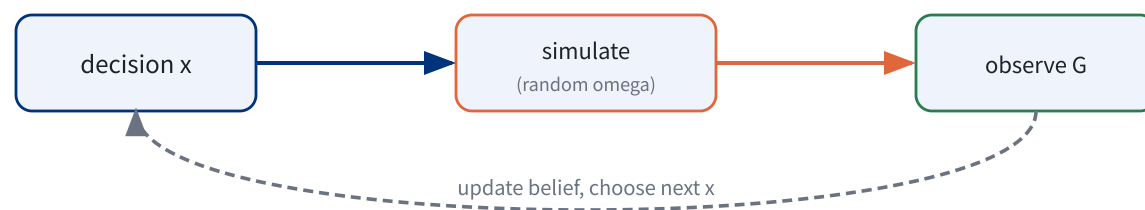
离散事件仿真

定义: 系统状态只在离散事件发生时改变,仿真时钟按事件表跳跃推进,而不是按固定步长。

$$S_{k+1} = \phi(S_k, e_k), \quad t_{k+1} = \min\{t > t_k : \text{下一事件}\}$$

适用范畴: 资源竞争与排队显著的系统——岸桥、AGV、闸口、病床、维修工位。

特点: 能表达复杂逻辑与随机性;单次运行只是一次随机实现,需多次重复才能估计期望。



仿真优化是个闭环：给定决策 x → 跑带噪仿真得到 G → 更新对 $E[G]$ 的信念 → 选下一个 x 。

工程对照

新自动化码头布局无解析目标，于是用离散事件仿真评估岸桥/AGV 争用，给候选布局打分；在数百次仿真上优化 $E[G]$ ，并重复试验以估计期望。

代理模型与贝叶斯优化

评估预算太少，就不能"多试几个点"。代理模型把预算换成"模型"，采集函数再决定下一个点该试在哪。

代理模型

定义：用已评估的少量样本拟合一个便宜的近似模型，在其上做搜索，再用少量真值点校正。

$$\hat{f} = \arg \min_{g \in \mathcal{G}} \sum_i (g(x_i) - y_i)^2 \quad (\text{高斯过程 / RBF / 随机森林})$$

适用范畴：单次仿真耗时从秒级到小时级、维度通常不超过 20-30 的黑箱问题。

特点：把"评估预算"换成"模型精度"；模型偏差会误导搜索，因此必须自适应加点。

贝叶斯优化

定义：以高斯过程为代理、以采集函数为准则，序列化地选择下一个评估点，以最少的次数逼近最优。

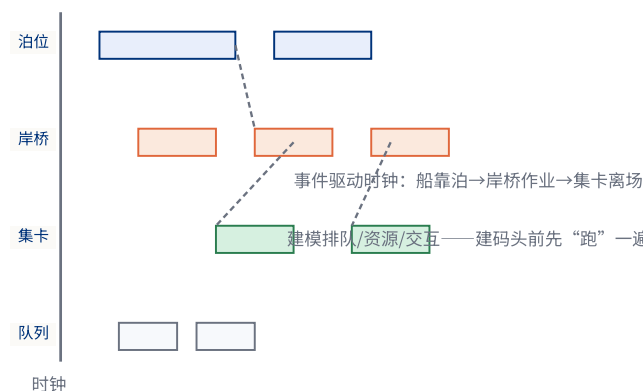
$$x_{n+1} = \arg \max_x \text{EI}(x), \quad \text{EI}(x) = E[\max(0, f_{\min} - \hat{f}(x))]$$

适用范畴：昂贵的黑箱、评估次数极少（几十到几百）、并行批次较小的场景。

特点：样本效率极高；高斯过程在样本数 n 大时代价为 $O(n^3)$ ，维度超过约 20 后性能衰减。

离散事件仿真（DES）：用“跑”来评测

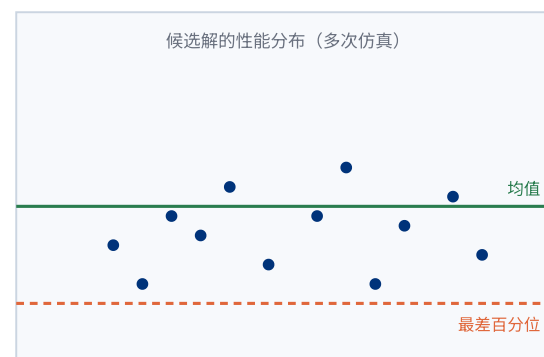
- 有些问题写不出干净的闭式模型：船随机到、机器故障、对手反应。
- **仿真**不靠公式评候选，而是把候选放进系统模型里**运行**看结果。
- **离散事件仿真(DES)**沿事件推进时钟——船靠泊、岸桥完成、集卡离场，刻画排队与资源交互。
- 集装箱码头常在“浇筑混凝土前”用 DES 验证设计——因为模型法看不见故障、集卡聚集与疲劳。



仿真不是“难题”的退路，而是当闭式模型会扭曲现实时的主动选择。

蒙特卡洛与仿真优化

- **蒙特卡洛/情景评估**：对候选抽很多情景（需求冲击、到达时刻、参数误差），平均其仿真表现。
- 优化平均，或优化**最差百分位**——由此把确定性法变成鲁棒/随机法。
- **仿真优化**：排序选择挑最优、序优化用远少于穷举的次数找“够好”解。
- 适用于：系统太复杂/太不确定无法闭式建模；“what-if”评估比可证最优更重要。



蒙特卡洛：抽样多情景取平均/最差 → 把确定性法变鲁棒/随机

把“运行系统”当作目标函数：任何优化器都能借仿真变成鲁棒/随机优化器。

CHAPTER 20

基于学习

把“求解”本身变成可学习的映射：神经求解器与强化学习带来速度，却也带来不可行、脆弱与难审计的新风险。

学习作为优化器

传统优化“每次现算”；学习型方法“先训后推”——把求解经验压缩进模型，遇到新实例直接预测。

适用：同类问题反复出现、单次求解太慢（如实时调度）。

神经求解器

用图神经网络等把组合/连续优化实例编码，训练网络直接输出解。代表：GNN 学 TSP、Transformer 学调度。



用大量样例训练，前向一次即出解

快但需训练数据，且解可能不可行

神经求解器把“优化问题”本身当作可学习的映射：训练一个网络直接从问题实例预测解。

强化学习

对序贯决策（第 15 章），强化学习通过“试错—奖励”学策略，天然适配多阶段、不确定环境。

目标：最大化折扣回报 π_t^t 。策略即“在每个状态怎么动”。

归纳偏置

学习器能否学好，取决于你喂给它的“先验结构”：对称性、稀疏性、物理守恒，都是宝贵的偏置。

- 好偏置：图结构 → GNN；时序 → RNN/Transformer。
- 坏偏置：强行用不合适的架构，样本再多大概率也白搭。

风险与边界

学习型优化快，但带着新风险：解可能不可行、对分布外输入脆弱、且难解释难审计。

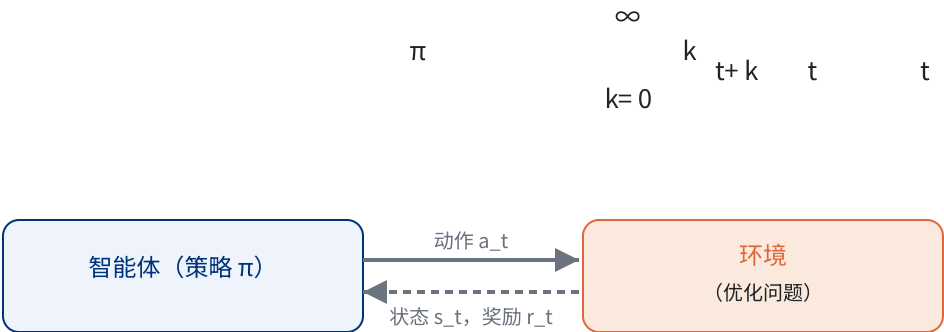
稳妥用法：学习型给“初值/候选”，再交给经典法校验修复。

绝不单独用：安全关键处必须有可行性兜底。

端到端学习（学会优化）

- **Instance → solution mapping.** A GNN encodes the instance; an attention decoder adds elements one by one (e.g. Kool & Blei).
- **Imitation.** Train to reproduce an exact solver / strong heuristic; infer in milliseconds — trading proof for speed.
- **Fit.** Recurring, structured instances (same road network, same layout); real-time control; a slightly-suboptimal answer *now* beats an optimal one tomorrow.

强化学习形式化



何时学习，如何保安全

- **Recurring, structured instances** — otherwise a learned model has nothing to transfer.
- **Real-time / sequential decisions** — E2E gives a one-shot solution; RL gives a reactive policy.
- **A simulator or teacher** — RL needs millions of safe episodes; E2E needs a solver or labelled data.
- **Explainability manageable** — pair with exact/sim when a guarantee or audit is mandatory.

神经求解器与端到端学习

前面所有方法都是"拿到实例再求解"。学习式方法反过来：**先在实例族上训练一次，之后每次求解都只是推理。**

神经求解器

定义：用神经网络把"问题实例"直接映射为"解"，训练一次、推理极快。

$$x = f_{\theta}(I), \quad \min_{\theta} E_{I \sim D}[\ell(f_{\theta}(I), x^*(I))]$$

适用范畴：实例分布稳定、需要毫秒级响应、或存在大量重复子问题（路径重规划、实时调度）。

特点：推理极快、训练成本可被摊销；可行性与最优性无保证，常需修复步骤或仅用作热启动。

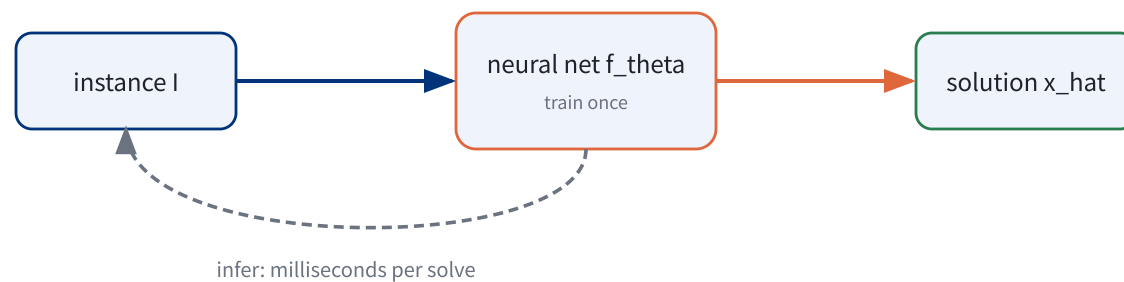
端到端学习（学习如何优化）

定义：把优化算法本身当作可学习的对象，用数据学习更新规则、步长或分支策略。

$$x^{k+1} = x^k + m_{\theta}(\nabla f(x^k), x^k) \quad \text{— 用学习的更新器取代手工规则}$$

适用范畴：存在大量同构问题族，且人工设计的规则明显次优的情形（分支变量选择、割的选择）。

特点：可超越人工启发式；需要大量训练实例与标签，分布漂移时性能会骤降。



学习式求解：实例 $I \rightarrow$ 神经网络 f_{θ} $\rightarrow$ 解 $x_{\hat{}}$ ；训练一次，之后每次求解只是毫秒级推理。

工程对照

实时 AGV 调度：在历史布局上训练 GNN 求解器，之后每次重调度都是毫秒级推理；修复层保证可行性。一次训练，终身摊销。

强化学习与归纳偏置

强化学习把"序贯决策"变成可学习对象；而能不能学到、学到后能不能泛化，很大程度上取决于**归纳偏置**选得对不对。

强化学习

定义：把序贯决策建模为马尔可夫决策过程，智能体通过与环境交互来最大化累计折扣回报。

$$V^*(s) = \max_a [r(s, a) + \gamma \sum_{s'} P(s' | s, a) V^*(s')]$$

适用范畴：序贯与在线决策——调度、库存、路径、控制；环境可仿真或可在线交互。

特点：不需要标注的最优解，只要奖励信号；样本效率低、训练不稳定，仿真到现实的落差是主要风险。

归纳偏置

定义：模型对"解应该长什么样"的先验假设——网络结构、等变性、约束的嵌入方式等。

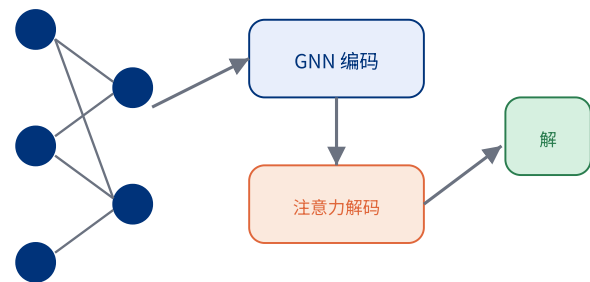
$$\hat{f} = \arg \min_{f \in \mathcal{F}} \text{loss}(f) \quad \text{— 假设空间 } \mathcal{F} \text{ 的选择就是偏置}$$

适用范畴：所有学习方法；对小样本、以及需要外推到更大规模实例的场景尤其关键。

特点：恰当的偏置（置换等变、图结构、可行性修复层）能大幅提升泛化；偏置过强则限制表达力。

端到端学习：实例直译解

- 方法谱系最现代的一端：用模型**学会**把实例直接映射到解。
- **端到端学习**：GNN 编码实例（图/代价矩阵），注意力解码器逐元素构造解——常模仿精确求解器。
- 把“解问题”变成“预测解”，以放弃最优性证明换**毫秒级**推理——适合实时控制。
- 前提：实例**重复且结构化**（同一路网/机群），且需教师或标注数据——否则学无可迁。



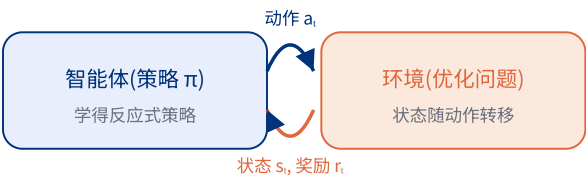
实例→解：图/代价矩阵送入网络，逐元素构造解（毫秒级）

适用于：recurring、结构化、需实时（常以精确解作教师）

学习能力只在“同一问题形状反复出现”时才划算；否则保留搜索。

强化学习：智能体↔环境循环

- 强化学习(RL)是序贯决策的自然方法：智能体与环境交互，试错学策略。
- 在状态 s_t 取动作 a_t ，得奖励 r_t ，转移到 s_{t+1} ；学策略 $\pi(a|s)$ 最大化累积奖励。
- 价值函数满足 **Bellman 方程**；Q学习/DQN/策略梯度从经验中学。
- 最适：决策序贯、环境可仿真、偏好反应式策略——多阶段、鲁棒、竞争博弈、动态组合。



Bellman: Q/V 满足递归; Q学习/DQN/策略梯度从经验中学

最适: 序贯、可仿真、需反应式策略 (多阶段/鲁棒/博弈)

RL 把“试错”变成可部署的反应式策略，补全了本书方法谱图的右端。

第五部分

应用

前面所有概念，最终要落到真实系统。本篇以自动化集装箱码头为贯穿案例，看优化如何贯穿“岸桥—堆场—运输”全链路。

本篇逻辑框架

前面所有概念，最终要落到真实系统。本篇以自动化集装箱码头为贯穿案例，看优化如何贯穿“岸桥—堆场—运输”全链路。

本篇 3 章 · 按阅读顺序

21

自动化集装箱码头

自动化码头

场景建模



22

码头自动化演进

自动化演进

技术演进



23

集装箱码头系统

码头系统

系统集成

逻辑主线：单点场景建模 → 技术演进历程 → 全系统集成，从局部到全局逐层放大。

主线：从单点场景建模、技术演进到全系统集成，由局部到全局逐层放大。

逻辑主线：单点场景建模 → 技术演进历程 → 全系统集成，从局部到全局逐层放大。

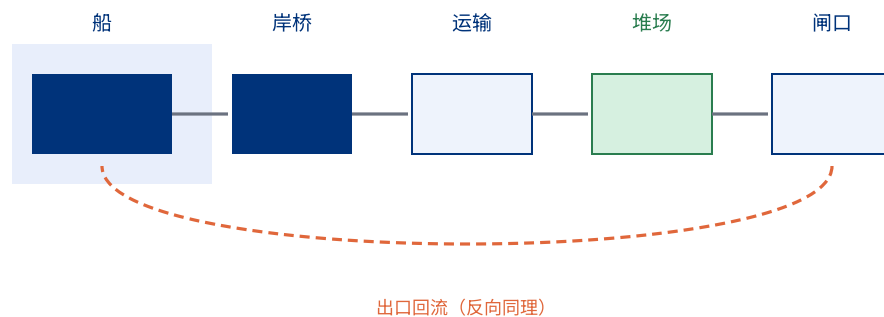
第 21 章

自动化集装箱码头

把前面所有概念装进一个真实系统：泊位、岸桥、堆场、运输相互耦合，几乎覆盖本书每一类问题结构。

什么是自动化集装箱码头

自动化码头指装卸、水平运输、堆存三大环节由可编程、常无人化的设备完成，并由一套软件统一协同。它把“重复确定性劳动”交给机器，把“判断与协调”留给人。

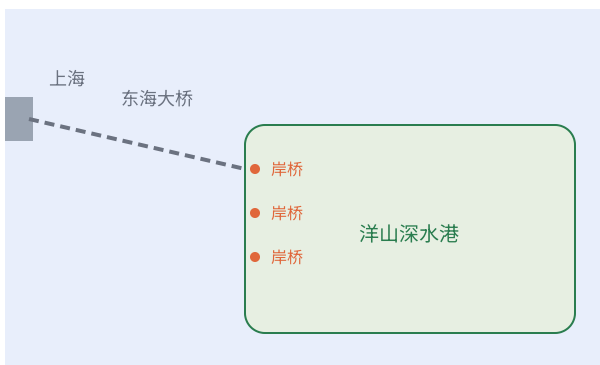


箱流闭环：进口 船→岸桥→运输→堆场，出口反向；任一环节断链都会拖垮整线。

- 为何自动化：24/7 一致作业、零排放、安全，并缓解招工难与岸线/土地稀缺。
- 三类作业：进口卸船、出口装船、中转转水——每类都是一串相互耦合的优化问题。

工程案例：上海洋山深水港四期

洋山四期于 2017-12-10 开港试生产，是全球单体最大、综合自动化程度最高的全自动化集装箱码头，由填海造陆形成，经东海大桥与上海相连。



区位：杭州湾口、东海大桥以南的洋山深水港；水深远期约 -15.5 m。

- 规模：7 个深水泊位，岸线 2350 m，陆域 223 万 m^2 。
- 能力：设计年吞吐初期 400 万 TEU，远期 630 万 TEU。
- 工艺：远程双小车岸桥 + AGV + 自动轨道吊（ARMG），全电力零排放。
- 设备：远景 26 岸桥 / 120 轨道吊 / 130 AGV；“大脑”为上港自研 ITOS。

洋山四期是“工程优化问题的总考场”：在最短岸线与最少用地上，把泊位、岸桥、运输、堆场、闸口协同到小时级乃至秒级——正是前五篇方法要解决的耦合系统。

系统组成

一个自动化码头由三大硬件与一套软件协同：泊位与岸桥负责船边装卸，堆场负责暂存，水平运输负责两者间的转运。

- 泊位/岸桥：决定船期与船边吞吐上限。
- 自动化堆场：AGV/IGV 与堆垛机协同的高密度存储。
- 水平运输：无人小车在岸桥与堆场间穿梭。

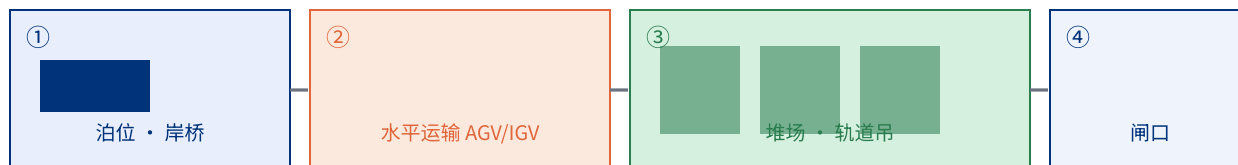
核心调度问题

码头运营是一串相互耦合的调度问题，几乎覆盖了本书每一类结构。

- 岸桥调度：哪台岸桥服务哪些舱位、顺序如何。
- 堆存分配：进口/出口箱落到堆场何处。
- 运输调度：小车路径与任务指派。
- 船舶配载：箱内位置影响翻倒与稳性（见第 11 章鲁棒）。

自动化码头布局与对应的优化问题

把物理布局拆成四个区域，每个区域天然对应一类（或几类）优化问题——布局决定了决策变量与耦合方式。



每个区域 ⇒ 一类（或几类）优化问题

- ① 泊位分配 BAP + 岸桥调度 QCSP：指派/排序；MILP、数学启发、约束规划、GA/禁忌。
- ② AGV/IGV 路径与任务指派、无冲突、充电：路由/指派；最短路、匈牙利、多智能体 RL。
- ③ 堆存位分配 BAS 与翻倒：装箱/指派；MILP、启发式、局部搜索（目标 翻倒）。
- ④ 集卡预约与闸口能力：调度/排队；仿真、MILP、滚动时域。

布局即问题结构：把哪类设备放在哪、彼此如何衔接，直接决定优化模型的变量、约束与耦合强度。

资源与约束

调度本质是在稀缺资源与硬约束下求最优：设备数量、作业时长、安全间距、时间窗都是约束。

硬约束：岸桥不能穿越、小车不可碰撞、船期不可无限拖。

软目标：总完工最短、设备 idle 最少、翻倒最少。

优化建模示例

取“岸桥任务指派”作最小例子：把 m 台岸桥指派到 n 个作业块，使最大完工时间最小。这是一个典型的并行机+负载均衡问题。

模型骨架：
$$C_j = \max_i \{t_{ij}\} \quad i = 1, 2, \dots, m \quad j = 1, 2, \dots, n$$
。即最晚完工，越小越好。

应用效果

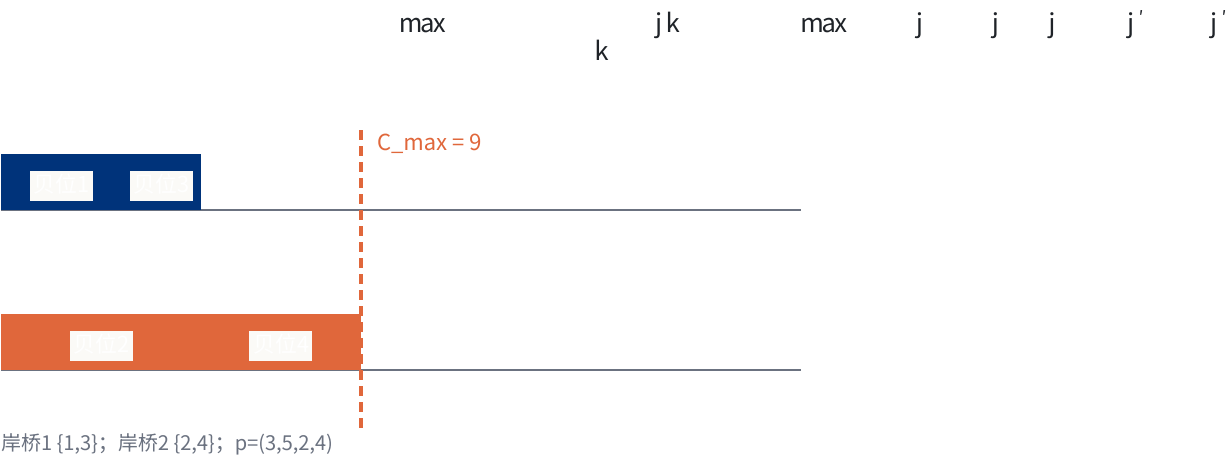
成熟的自动化码头相较传统码头，在人力、安全与夜间作业一致性上收益明显，但前期投资与系统耦合度也更高。

关键不在“全自动”，而在“把对的解在对的时刻送到对的设备”——这正是前面所有方法的总考场。

码头即系统：五类子问题

子问题	决策	原型
泊位分配（BAP）	每艘船分配哪个泊位与时段	指派 / 调度
岸桥调度（QCSP）	每台岸桥的装卸顺序	调度
堆场存储	每个箱分到箱区/贝位	装箱 / 指派
AGV 路径规划	车辆的路径与时刻	路径规划
闸口预约	每辆集卡的时间窗	调度

实例：岸桥调度



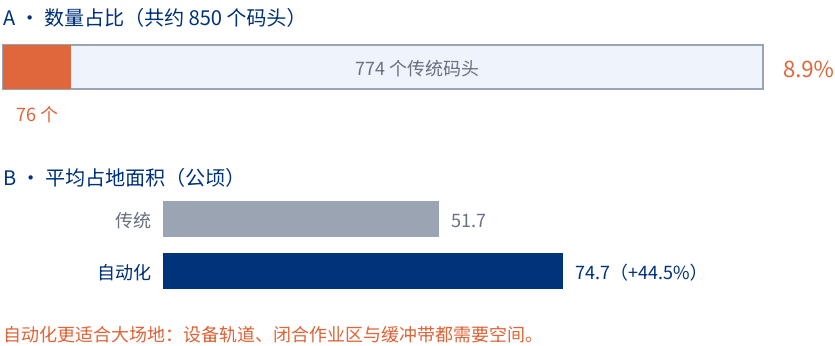
岸桥1 {1,3}; 岸桥2 {2,4}; $p=(3,5,2,4)$

每个问题类都在此现身

问题类	码头表现
组合	BAP、QCSP、堆场装箱、AGV 路径、闸口
鲁棒	抗延误/故障的泊位与岸桥计划
多目标	吞吐、能耗、延误的 Pareto 前沿
多阶段	带补偿的滚动时域重调度
双层	码头价格/能力 → 班轮行为模式
竞争	港间竞争 / 拥堵博弈

全球一览：自动化码头仍是少数派

到 2025 年，全球约 850 个集装箱码头中只有 76 个实现了不同程度的自动化，占比约 8.9%。但它们的平均占地面积明显更大——74.7 公顷对 51.7 公顷，高出 44.5%。



占比与占地：自动化并非普适选择，而是与场地规模强相关的工程决策。

半自动化

仅堆场自动（ARMG），岸桥与水平运输仍由人工驾驶。改造门槛最低。

全自动化

堆场与水平运输均自动，岸桥远程操控。需要重构整个作业逻辑。

完全自动化

含岸桥自动装卸与闸口无人核放，人只在异常时介入。目前样本极少。

海侧：泊位分配与岸桥调度

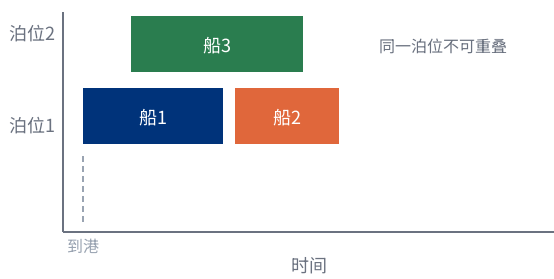
海侧是全码头的节拍源。船一旦压在泊位上，后面的堆场、水平运输、闸口全部被动。泊位分配问题（BAP）已被证明是强 NP 难的。

离散泊位分配的具体数学形式

$$\begin{matrix} & i \in V & i & i & i & & i & i & i & i & i \\ & & & & & & & & & & \\ j & i & & ij & & ib & jb & & ib & & \end{matrix}$$

$b \in B$

- $i b$ 表示船 i 停靠泊位 b ； $i j$ 表示 i 排在 j 之前。
- i 是船的重要度权重（干线大船权重远高于支线小船）。
- 目标是加权在港时间，不是简单的完工时间——这一点决定了排序规则。



泊位—时间平面：每条船占一个矩形，同泊位矩形不得相交。

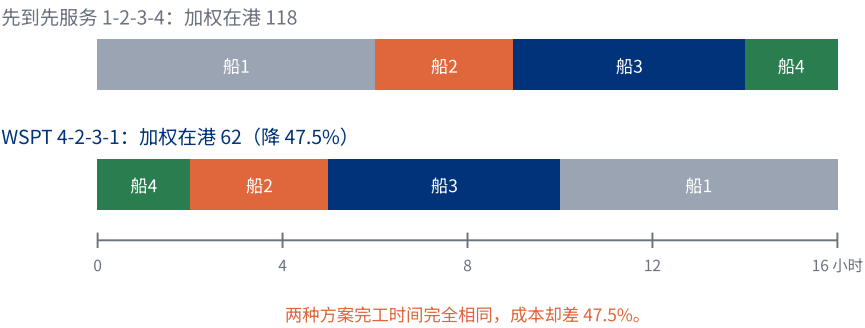
算例：排序本身就创造价值

单泊位、四条船同时到港，作业时长 $p = (6, 3, 5, 2)$ ，重要度 $w = (1, 4, 2, 3)$ 。目标 $\min \sum w_j C_j$ 。总作业量恒为 16 小时，无论怎么排。

船	作业 p_j	权重 w_j	比值 p_j/w_j
1	6	1	6.00
2	3	4	0.75
3	5	2	2.50
4	2	3	0.67

WSPT 规则

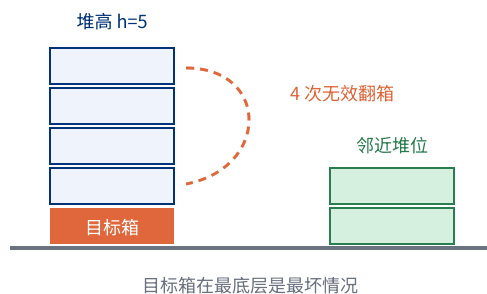
按 p_j/w_j 升序排序即为最优：。这是单机加权完工时间问题的经典结论，可用交换论证证明。



穷举 24 种排列验证：最优 62，最差 132；WSPT 精确命中最优。

堆场：存位分配、预翻箱与翻箱

堆场的困难来自一个物理事实：集装箱是竖着堆的，只能从上面取。取箱顺序与堆放顺序不一致时，就必须先把压在上面的箱子挪走——这就是翻箱。



若目标箱位置在堆中均匀分布，期望翻箱次数为 $(h-1)/2$ ； $h=5$ 时为 2.0 次。

堆场存位分配 YSAP

为每个进场箱选择块、贝、层。目标是最小化后续翻箱与运距的期望值，属于装箱与分配的混合。

预翻箱 Premarshalling

在船到港前的空闲时段重排堆场，使取箱顺序与堆放顺序一致。此时设备是空闲的，重排几乎免费。

工程直觉

翻箱是纯浪费，但它由几小时前的存位决策决定。所以堆场问题的本质是：用当下的信息，为一个还没到来的取箱顺序下注。

水平运输：车队要多大？

AGV 从柴油固定轨道起步，如今已是 GPS 导航、电池驱动、最高 6 m/s 的零排放车辆。但真正的工程问题不是单车性能，而是“配多少台”。

用 Little 法则做一次车队定量

- hs
- m 为岸桥到堆场平均单程距离，

m/s，交接握手

固定耗时

hs

s。
 - 岸桥节拍

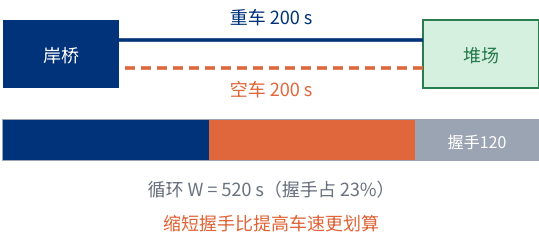
箱/小时

，故每台岸桥需

台 AGV。
 - 26 台岸桥

5 = 130 台

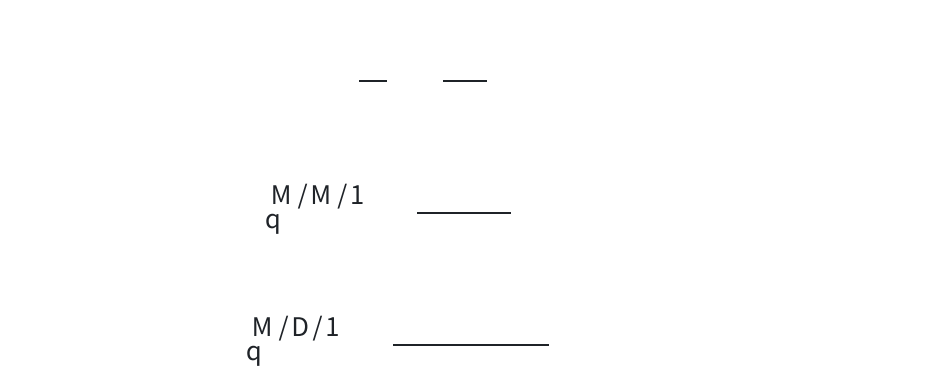
——这恰好等于洋山四期的实际 AGV 保有量。
 - 向上取整带来约 15% 的余量，正好用于充电、故障与交通拥堵。



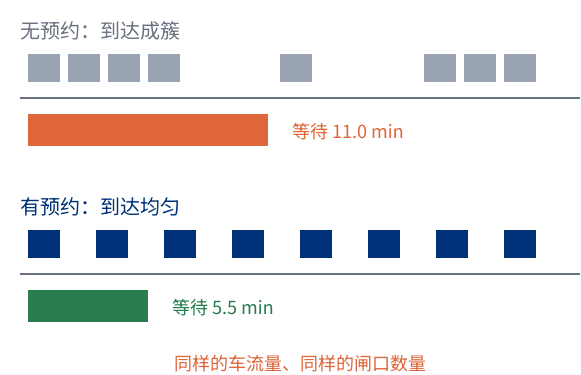
循环时间分解显示：固定的交接时间是车队规模的隐形放大器。

陆侧：闸口预约为什么有效

外集卡在闸口排长队是所有码头的通病。反直觉的是：解决办法往往不是加闸口，而是削平到达的随机性。



- 到达率 λ 车/小时，服务率 μ 车/小时，负荷 ρ 已相当高。
- 预约把服务时间变成确定型（ D ），排队时间从 11.0 分钟降到 5.5 分钟，整整一半。
- 一台闸口设备都没有增加，只是把方差挪走了。



预约系统（TAS）是一个把随机到达转为可控到达的调度问题。

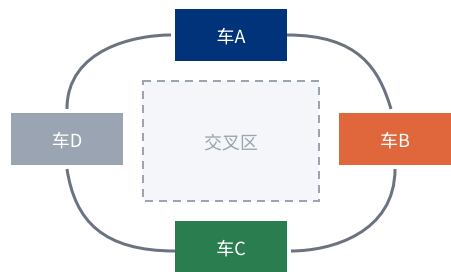
自动化催生的七类新问题

自动化解决了旧问题，同时生产出一批全新的问题。它们在人工码头根本不存在，因为人天生会临机应变。

新问题	为何在人工码头不存在	问题原型
车队死锁避免	司机会互相让路、目视协商	路径规划 + 资源互斥
电池与能量调度	柴油车加油只需几分钟	带充电中断的调度
吊具与车辆交接	人工对位误差可当场修正	同步与耦合约束
实时重调度	班组长口头调整即可	滚动时域优化
感知与箱号识别	人眼读箱号几乎零成本	不确定信息下决策
预测性维护	故障可临时换人换车	随机停机下的维护规划
车队规模与布局共设计	布局定了还能加人加车	战略层双层优化

最典型的新问题：车队死锁

四台 AGV 各自都在等前车让路，形成一个等待环，谁都不动。人工码头几乎不会出现，因为司机会破坏；自动化码头必须由约束在事前禁止它。



等待关系构成有向环 → 系统永久停滞

死锁不是性能问题，而是可行性问题：调度方案在数学上“合法”，在物理上却无法执行。

时空预留的具体约束形式

$$k \in K, \quad k, c, t$$
$$k, c, t \quad k', c', t$$

- k, c, t 表示车 k 在时刻 t 占用路网单元 c 。每个单元每时刻至多一台车。
- 把路网离散成单元后，死锁避免退化为一组线性互斥约束——代价是变量数量与时间步数成正比。
- 这正是第 15 章滚动时域方法必须登场的原因：全时域求解规模无法承受。

从物理分区到问题类：一张对照表

码头的四个物理分区，各自对应一组固定的优化问题原型。看懂这张表，就能在任何码头现场认出“这是第几章讲过的问题”。

物理分区	核心决策	问题类	计算难度
海侧（泊位与岸线）	泊位、靠泊时刻、岸桥台数	分配 + 调度	强 NP 难
岸桥作业面	每台岸桥的作业序列	带不可跨越约束的并行机	NP 难
堆场	存位、预翻箱、翻箱顺序	装箱 + 分配 + 在线决策	NP 难，且信息不完全
水平运输	任务派发、路径、充电	路由 + 资源互斥	NP 难，实时性苛刻
陆侧（闸口与堆场交界）	预约时段、车道分配	调度 + 排队控制	可近似求解

读表要点

四个分区的难度不是均匀的。海侧决定节拍且最难，陆侧最容易改善却常被忽视。优化投入应当先看边际收益，而不是先看技术含量。

放大一层：港口级问题

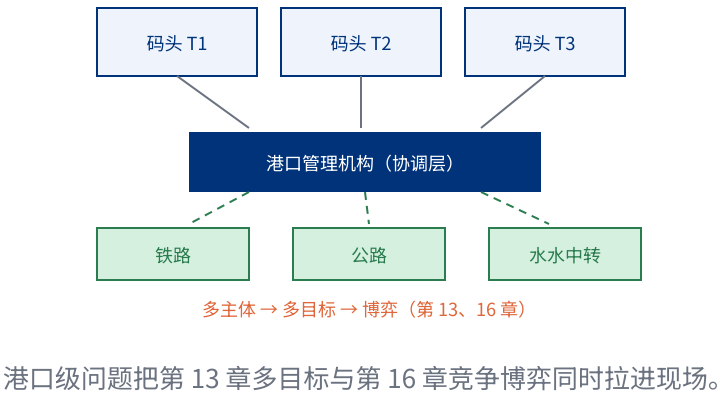
把镜头从码头拉到整个港口，问题的性质会变。此时决策主体不再唯一，各码头之间既合作又竞争，优化模型必须容纳多个目标函数。

泊位共享

相邻码头把泊位资源合池使用。总体效率提升，但需要一套让各方都不吃亏的分摊规则——这是合作博弈，不只是优化。

集疏运协同

水水中转、公路、铁路三种集疏运方式争夺同一批箱量与同一段时间窗。改一种方式的时刻表，会同时改变另外两种的负荷。



港口级问题把第 13 章多目标与第 16 章竞争博弈同时拉进现场。

第 22 章

码头自动化演进

自动化是一条渐进曲线：人从操作者退为监督者，每跨一步，优化问题的边界就向外扩一圈。

从人工到自动

码头自动化不是一夜间发生，而是一条从“人扛”到“人管”的渐进曲线：人从操作者退为监督者与例外处理者。

本质：自动化把“重复确定性劳动”交给机器，把“判断与协调”留给人。

演进各阶段

典型四阶段：机械化 → 单点自动化 → 系统联动 → 自主决策。每跨一步，优化问题的边界就向外扩一圈。

- 阶段一：吊具/拖车替代纯人力。
- 阶段二：单台设备可编程自动运行。
- 阶段三：设备间协同调度（本篇核心）。
- 阶段四：按状态自主重排计划。

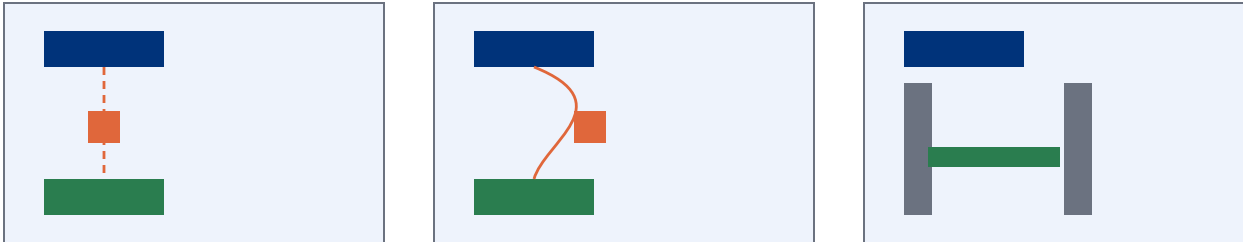
技术驱动

推动演进的是几股技术合力，而非单一发明：

- 感知：激光/视觉让设备“看见”箱与障碍。
- 通信：低时延网络让协同实时化。
- 算法：本书的优化与学习提供“怎么排”的智能。

水平运输技术三方案

自动化码头的水平运输有三条技术路线，设备形态不同，引发的优化问题也不同。



iV：固定导引车道，路径冲突需在线规避（路由+指派） IGV：自由行驶，动态路径更灵活，协同/避碰更难 RMG：轨道吊兼做水平+垂直，无独立运输车，箱区耦合

没有“最优”方案，只有“最适配”：吞吐、用地、投资与现有工艺共同决定选型。

投资与权衡

自动化是高固定成本、低边际成本的选择：前期贵、回本慢，但在吞吐稳定且人工贵的地方划算。

划算场景：吞吐大、地价高、招工难。

慎重场景：吞吐波动大、投资回收期长。

传统 vs 半自动 vs 全自动

自动化不是单点指标跃升，而是在多个维度上重新权衡：用高固定资产投资换低边际成本与一致性。

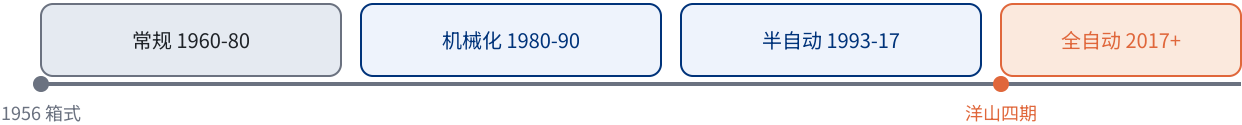
维度	传统	半自动	全自动
每班人数	多（人工）	较少	少（远程）
安全 / 排放	较低 / 柴油	混合	高 / 纯电零排放
用地与泊位	较低	中等	最高（立体堆存）
24/7 一致性	不稳定	较好	稳定
固定资产投资	低	中等	高
回收期	快	中等	慢（常 >6 年）
规划方法	规则 / 人工	MILP + 启发式	MILP + 学习 + 孪生

趋势展望

下一波重点不在“更自动”，而在“更聪明地自动”：用学习预判扰动、用数字孪生预演、用多目标在前沿上动态取舍。

与本书呼应：由粗到精（第 8 章）、多解（第 9 章）、鲁棒（第 11 章）正是“聪明自动化”的方法底座。

自动化四纪元

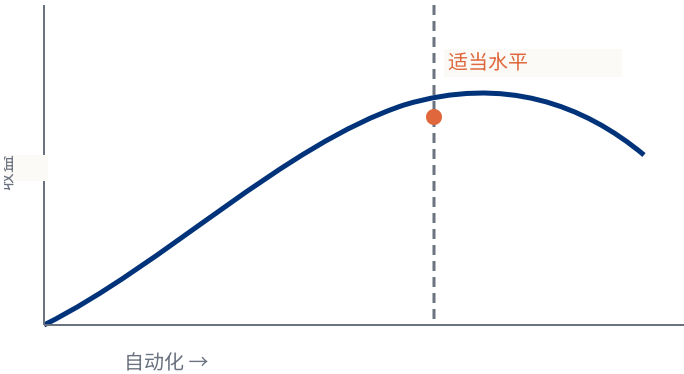


演进即分阶段求解



每一阶段都比前一阶段多出一个维度——结构、不确定性，或相互交织的目标。没有任何人能一步设计到位。

更自动 ≠ 更好：本地上限



需求（经济）

吞吐是否足以摊薄固定成本？

能力

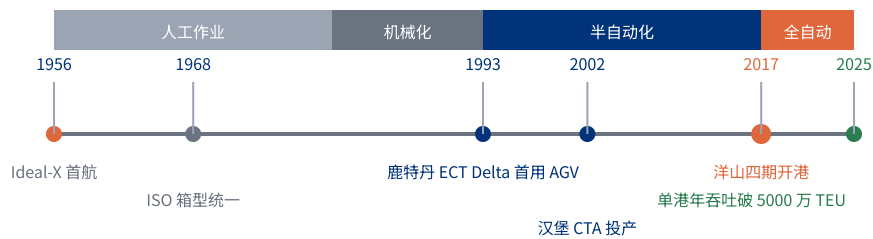
人力、物料、资金。

制度

政治与政策许可。

从一条船到全自动码头：七十年

集装箱运输从 1956 年 McLean 改装的 Ideal-X 号起步。此后每一次跃迁都不是技术单独推动的结果，而是技术、劳工、资本与制度同时对齐的时刻。



注意四条时代带的宽窄：机械化只用了 13 年，半自动化却走了 24 年。

越靠后的跃迁越慢——因为要解决的问题越来越不是技术问题。

每个里程碑之后，都会冒出一批当时无人预料的新优化问题。

七十年里程碑与四条时代带：时间轴上方是阶段，下方是事件。

四个时代，四种问题结构

每个时代的瓶颈不同，因此优化的对象也不同。把这四行读下来，就是一部集装箱码头的优化问题史。

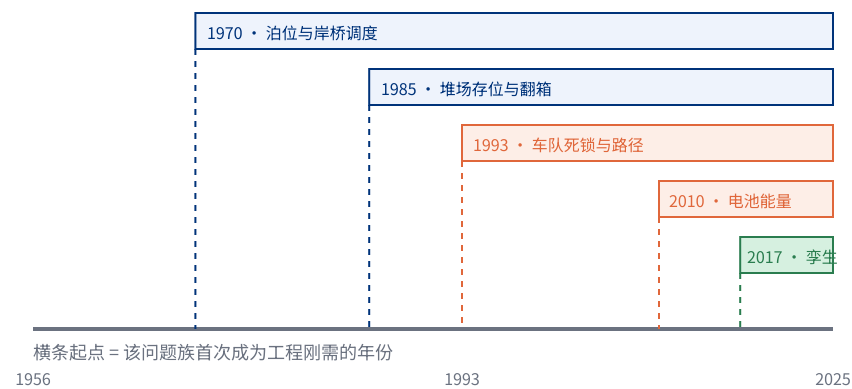
时代	主导瓶颈	优化对象	决策周期
人工作业（1956–1980 年代）	人力与工班组织	工班排班与人力配置	按班（8 小时）
机械化与计算机化（1980–1990 年代）	设备台数与场地空间	设备派工与堆场分区	按小时
半自动化（1993–2017）	人机交接与信息一致性	联合调度与作业耦合	按分钟
全自动化（2017 至今）	异常处理与算法可行性	实时重调度与全局一致性	按秒

演进的真正代价

决策周期从 8 小时压缩到 1 秒，意味着求解时间预算缩小了约 28800 倍。这就是为什么自动化码头必须放弃精确求解，转向启发式与滚动时域。

优化问题不是被设计的，是被逼出来的

没有人在 1970 年坐下来发明“AGV 死锁避免问题”。它是在 1993 年鹿特丹第一次让无人车上路之后，被现场逼出来的。问题族有出生年份。



五个问题族的出生年份：旧问题从未消失，只是被新问题叠加。

对工程师的含义

当你在一个新系统里遇到教科书上找不到的问题时，不要怀疑自己——很可能你正站在一个问题族的出生年份上。

要不要自动化：三个问题与五个因素

自动化不是先进程度的排名，而是一次适配判断。先回答三个问题，再逐项核对五个因素——顺序不能颠倒。

问题一 · 需求

箱量是否足够大且足够稳定？自动化的固定成本必须由持续的高吞吐摊薄，波动的箱量会让重资产变成负担。

问题二 · 能力

本地是否有能维护它的工程师？设备可以进口，运维能力不能。这一条是最常被低估的。

问题三 · 制度

劳工协议、监管框架与地方政治是否容许？技术可行不等于制度可行，后者往往是真正的约束边界。

因素	有利于自动化
经济	高箱量、长期增长可预期
劳工	人工成本高、招工困难
物料与设备	设备供应链与备件可获得
金融	可获得长周期低成本资金
政治	政策支持且劳资关系可协商

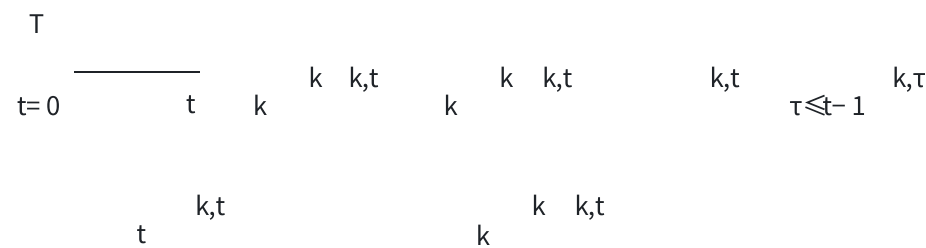
为何有的港口不自动化

并非落后。当人工成本低、箱量波动大、资金昂贵时，不自动化才是最优解。这正是优化的本义：最优取决于约束，而不是取决于潮流。

分阶段自动化：买的是期权，付的是净值

几乎所有码头都选择分阶段改造。这个选择常被说成“稳妥”，但它在财务上是有明确代价的。我们把它算出来。

分阶段投资的具体数学形式



- $k_{k,t}$ 表示阶段 k 在第 t 年启动； $k_{k,t}$ 表示它在第 t 年已在产生效益。
- K 是年度现金上限——正是这条约束把“一次到位”变成了“分三期”。
- 三个阶段：堆场 ARMG（投 120 / 年收 26）、AGV 车队（90 / 20）、远程岸桥（70 / 14）。

方案	15 年净现值 (r=8%)	现金峰值
一次到位（第 0 年全投）	233.6	280
分三期（第 0 / 3 / 6 年）	161.7	120
差异	-71.9（降 30.7%）	-160（降 57%）

结论要读两遍

分阶段牺牲了 30.7% 的净现值，换来现金峰值下降 57% 与随时中止的期权。如果不确定性足够大，这笔交易是划算的；如果需求高度确定，它就是纯损失。

第 23 章

集装箱码头系统

把码头当作一个“优化内核 + 学习外脑 + 孪生沙盘”的闭环系统来看——这正是五篇方法的总考场。

岸桥 / 堆场 / 水平运输

把码头拆成三个环节，每个都是一类经典优化问题；串起来才是系统。



三个环节首尾相接成链；任一环节的瓶颈都会“堵”住整条线，优化必须整体看而非局部最优。

协同与耦合

三环节靠“接口量”耦合：岸桥产出速率决定运输负载，运输速率决定堆场接收节奏。切错接口（第 2 章）会放大拥堵。

启示：码头优化常需分解（第 2、8 章），但协调器必须掌握接口量的真实动力学，否则只是“各扫门前雪”。

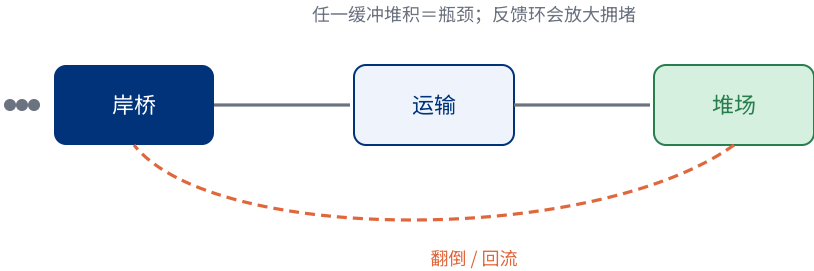
瓶颈分析

定位瓶颈是系统优化的第一步：用吞吐/利用率/{排队长度}判断哪环最“喘”。

- 若岸桥闲、堆场堵：瓶颈在堆场/运输，别盲目加岸桥。
- 若运输空跑多：瓶颈在任务指派不均。

码头即排队网络

把三个环节看成串联的服务台，缓冲堆积处就是瓶颈；翻倒/回流让网络出现反馈环。

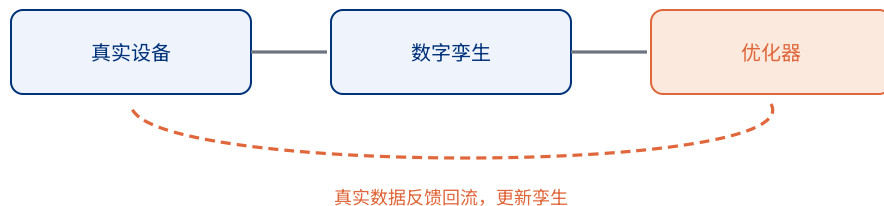


瓶颈不在“设备慢”，常在“衔接处堆积”；优化要盯缓冲而非单台效率。

数字孪生

数字孪生是码头的“可仿真镜像”：实时同步状态，用来预演调度、做第 19 章的贝叶斯式What-if，而不拿真实码头冒险。

闭环：孪生里试解 → 验证可行 → 下发真实设备；真实数据回流再更新孪生。



闭环：孪生里试解 → 优化器求解 → 下发真实设备 → 数据回流再更新孪生。

展望

未来的码头系统会是“优化内核 + 学习外脑 + 孪生沙盘”的闭环：既快又稳，且能在扰动来时自我重排。

而这，正是本书五篇想交给你的全部能力的总和。

集成码头模型



为理解而分解，但为性能而协同优化：局部最优的岸桥序列会堵塞运输、饿死堆场。

自动化栈：五层五问题

- **自动化设备**——被调度、被排序、互不干扰。
- **设备控制**——速度/加速度权衡周期时间与能耗。
- **控制中心（TOS）**——预测、排程、监控、重排。
- **人机**——哪些决策归操作员、哪些归优化器。
- **港口社区**——与船公司、集卡、海关的数据交换。

第 1-3 层构成闭环：TOS 制定计划，设备执行，反馈再重排。

决策时域：战略 · 战术 · 运营

同一座码头在三个时域上反复求解：时域越短、频率越高、越依赖“快而稳”的启发式与学习。



三层用“由粗到精”衔接：战略定边界，战术填计划，运营做秒级修正。

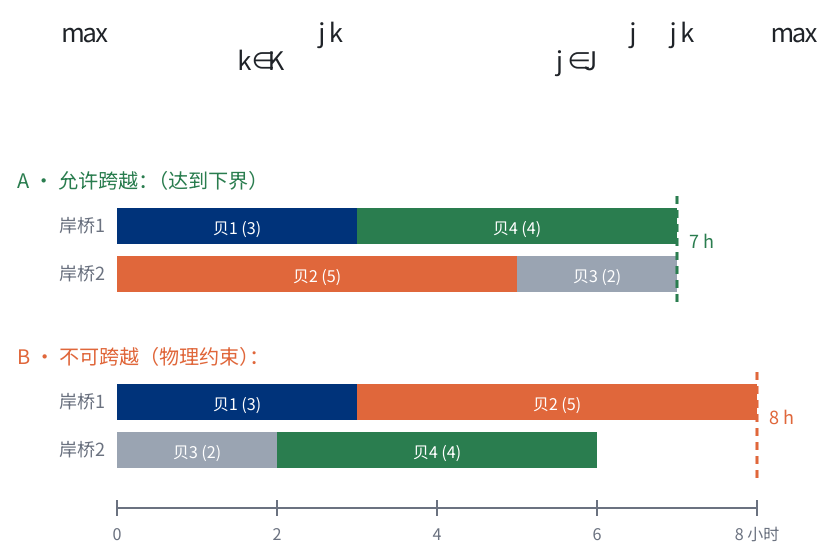
承诺与实绩的落差

- 运营成本下降 **15–35%**（预期 25–55%）。
- 生产率 **下降 7–15%**（预期 +10–35%）；自动化岸桥约 20+ 箱/时，传统码头 30+ 箱/时。
- **61%** 的码头预计回收期超过 6 年；测试耗时 2–37 个月。
- 根本原因：工程师短缺、数据质量差、各自为政、异常处理薄弱——这是 *优化流程* 的失败，而非硬件的失败。

自动化不会消除优化问题——它让优化成为必需。

算例：两台岸桥、四个贝位

船上四个贝位待作业，工作量 $p = (3, 5, 2, 4)$ 小时，两台同型岸桥并行。目标最小化完工时间 $C_{\max}$ 。总工作量 14 小时，理论下界 $\lceil 14/2 \rceil = 7$ 。



穷举 16 种分配方案验证：允许跨越时最优 7（负载 7/7，效率 100%）；施加不可跨越约束后最优 8（负载 8/6，效率 87.5%）。

不可跨越约束的价格 = 1 小时

岸桥装在同一条轨道上，无法互相穿越，因此每台岸桥只能负责一段连续贝位。这条纯物理约束把最优值从 7 推到 8——约束不是装饰，它直接改变最优值。

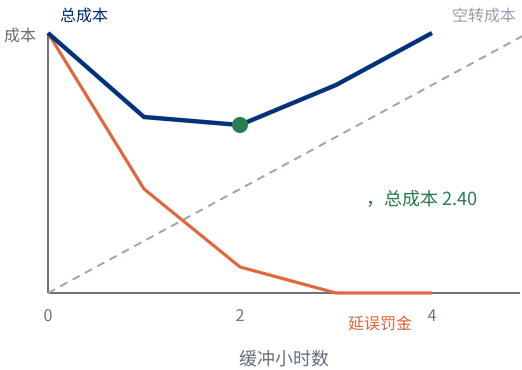
勘误：书稿正文对同一算例给出的方案（岸桥1 作业贝 1、3，岸桥2 作业贝 2、4， $C_{\max} = 9$ ）经穷举验证为次优。允许跨越时最优为 7，施加不可跨越约束后最优为 8。本页数值均由脚本穷举得出。

缓冲时间：留多少才对？

船很少准点。留缓冲会让泊位空转，不留缓冲则延误会顺着计划往后传染。缓冲量本身就是一个可优化的决策变量。

$b \geq 0$ idle del ω ω ω

缓冲 b	空转成本	期望延误	延误罚金	总成本
0	0.0	1.00	4.00	4.00
1	1.0	0.40	1.60	2.60
2	2.0	0.10	0.40	2.40（最优）
3	3.0	0.00	0.00	3.00
4	4.0	0.00	0.00	4.00

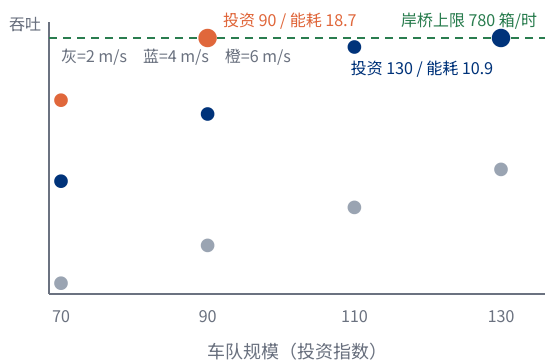


总成本是空转与罚金两条曲线之和，呈 U 形；最优点在两者边际相等处。

- 延误情景 小时，概率 ；
 idle ， del 。
- 最优缓冲 * 小时，总成本 2.40，比零缓冲降 40%。
- 缓冲过头同样有害： 的成本又回到 4.00，与完全不留缓冲一样差。

吞吐、投资、能耗：没有单一赢家

车队规模 $n \in \{70, 90, 110, 130\}$ 与 AGV 速度 $v \in \{2, 4, 6\}$ 组成 12 种配置。穷举后有 10 个非支配解——这意味着无法用一个数字给码头打分。



两个点都达到 780 箱/时的岸桥上限，却分别贵在投资与贵在能耗。

支配关系

配置 被 支配，当且仅当 在吞吐、投资、能耗三项上都不差，且至少一项严格更好。12 个配置中只有 2 个被支配。

关键对比

90 台车跑 6 m/s 与 130 台车跑 4 m/s，吞吐完全相同（都被 780 的岸桥上限截断），但前者省 40 个投资单位、多耗 7.8 个能耗单位。选哪个，取决于电价与资金成本，而不取决于算法。

全时域 24 小时：187200 个二元变量（不可解）

窗口 1（0-4 h）：31200 变量，求解并执行前 2 h

窗口 2（2-6 h）：用最新实况重新求解

窗口 3（4-8 h）：延误、故障、新到船都已并入

窗口 4（6-10 h）：以此类推

0 h 12 h 24 h

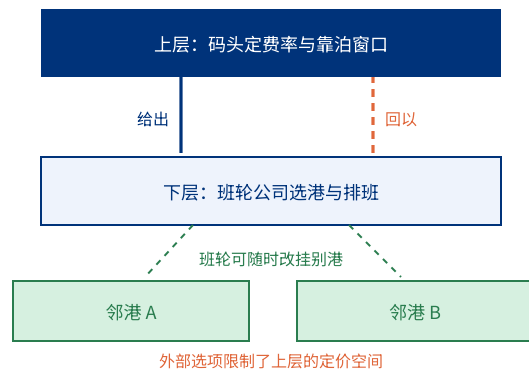
滚动时域放弃了全局最优性——每个窗口的最优拼起来不等于全天最优。但它换来两样东西：可解的规模，以及把新信息吃进模型的机会。在延误频发的码头，后者的价值通常远超前者的损失。

当对手也在优化：双层与港间博弈

码头定价与班期，班轮公司据此选港。码头的最优解取决于班轮的反应，而班轮的反应又取决于码头的决策——这是双层结构，不是单一优化。

$$\begin{array}{c} * \\ y \in \\ * \\ x \in (y) \end{array}$$

- 上层：码头的费率、保证靠泊窗口与承诺作业效率。
- 下层：班轮公司的挂港选择与船期安排，它只关心自己的总成本。
- 难点在于上层目标依赖下层的最优解集，问题一般是非凸的，即便两层各自都是线性的。
- 第 12 章的双层方法与第 16 章的竞争博弈，在这里同时出现在一张合同上。



邻港的存在把双层问题变成博弈：上层不能任意提价，因为下层有退出选项。

一张表收束全书：问题类如何在码头现身

集装箱码头之所以是本书最后一个案例，是因为它把前面所有章节的问题类一次性摆在同一片场地上。

问题类（章）	在码头的具体形态	本篇对应页
确定性组合优化（第 10 章）	泊位分配、岸桥指派、堆场存位	海侧算例、岸桥算例
鲁棒优化（第 11 章）	船期不确定下的缓冲时间设计	缓冲时间算例
双层优化（第 12 章）	码头定价与班轮选港的上下层结构	双层与竞争
多目标优化（第 13 章）	吞吐、投资与能耗的非支配前沿	Pareto 前沿
多阶段决策（第 15 章）	滚动时域重调度与分阶段投资	滚动时域、分阶段代价
竞争博弈（第 16 章）	邻港争抢同一条干线航线	港口级问题、双层与竞争
仿真与学习方法（第 19、20 章）	数字孪生验证与派工策略在线学习	数字孪生

收束

工程优化问题从不以纯净的形态出现。码头把它们全部叠在一起：多个主体、多重目标、真实的不确定性、带噪声的数据，以及不容许失败的安全边界。识别问题类只是起点，判断哪一类此刻主导，才是工程判断。

六篇，一条主线

从“问题是什么”出发，到“如何把它解出来”，再到真实港口——全书始终在训练同一种能力：把混乱的工程现实，变成可建模、可求解、可权衡的结构。

- 第零篇 · 课程简介：建立全局地图与学习路线。
- 第一篇 · 基础：问题、分解、解、方法、权衡——方法论地基。
- 第二篇 · 工程优化：求解流程、由粗到精、多解分析。
- 第三篇 · 工程优化问题：确定/组合、鲁棒、双层、多目标、多阶段、竞争博弈。
- 第四篇 · 工程优化方法：经典数学规划、仿真驱动、学习驱动。
- 第五篇 · 应用：以自动化集装箱码头为贯穿案例，看优化如何贯穿“岸桥—堆场—运输”全链路。

贯穿全书的暗线：不追求“一个最优解”，而追求“在正确的时刻，用正确的方法，得到足够好、且扛得住现实的解”。

你带走的，不是公式

- 问题意识：先问“这是哪一类问题”，再选武器。
- 分解能力：把大系统拆成可建模的子系统，再决定是否协同优化。
- 方法光谱：从精确解、启发式、仿真到学习，知道每条路的边界。
- 多解思维：最优常常不唯一，解集比一个点更有韧性。
- 权衡直觉：吞吐、投资、能耗、风险之间，没有免费午餐。
- 落地视角：真实系统（如自动化码头）是前面所有结构的综合考场。

授人以鱼，不如授人以渔

本讲义的目标，从来不是让你背下某几个算法，而是给你一套“钓鱼的本事”：看清问题结构、选对方法、敢于取舍、容纳不确定性。

鱼

给你一个现成答案：某个公式、某段代码、某种“标准做法”。它解一时之急，却长不出能力。

渔

给你一种思维方式与一套方法工具箱：何时建模、如何分解、怎样在约束与多目标间做工程判断。它让你自己能钓一辈子的鱼。

所以本书反复强调：先分型、再分解、巧权衡、敢用多解——这些才是离开课堂后依然有用的“渔”。

结语

优化，是一种工程教养

当你下次面对一个“排不过来、装不下、来不及”的真实问题时，希望你能想起：先分型，再分解，巧权衡，敢用多解。工具会变，但这种把现实变成可解结构的能力，不会过时。

愿这本讲义，成为你工具箱里那根最趁手的“钓竿”。

谢谢

欢迎提问与交流

周艳杰 · 郑州大学管理学院

计算机与工业工程研究组 (CIEG)

课题组主页

计算机与工业工程研究组 (CIEG)

CIEG 成立于 2020 年 9 月，立足郑州大学管理学院，致力于运筹优化、机器学习与工业工程在真实问题中的交叉研究。

研究方向

港口与物流优化 · 鲁棒与随机优化 · 双层与博弈优化 · 仿真与学习驱动优化 · 工业系统数字孪生。

加入我们

CIEG 长期招收本科、硕士、博士及博士后。无论你来自工业工程、计算机、数学、管理还是其他背景，只要对“用模型解决真实问题”感兴趣，欢迎联系。

主页: <https://ieyjzhou.github.io/lab> (CIEG 课题组主页)